



UNIVERSITÀ DEGLI STUDI DI MILANO

FACOLTÀ DI SCIENZE E TECNOLOGIE

FACOLTÀ DI SCIENZE POLITICHE,
ECONOMICHE E SOCIALI

Master Degree in Data Science for Economics

DIMENSIONALITY REDUCTION AND
CLUSTERING: CONTROLLED
EXPERIMENTS ON SYNTHETIC DATA
AND VALIDATION ON RESIDENTIAL
ENERGY CONSUMPTION DATA

Supervisor: Prof. Silvia SALINI

Co-supervisors: Prof. Stefano MONTANELLI

Thesis by:
Antonio DE PATTO
Student ID: 46430A

Academic Year 2024/2025

A mia sorella Elena

Acknowledgments

Mamma e papà, un ringraziamento speciale va a voi, per avermi sempre dato gli strumenti e la libertà di scegliere la mia strada. La vostra dedizione al lavoro e alla famiglia è stata il mio carburante, la mia motivazione personale per rendervi orgogliosi dei miei successi. Ringrazio mia sorella Elena, a cui dedico questa tesi. Fin da piccolo mi ha seguito, supportato e guidato in ogni scelta. Dalle pagine di quaderno strappate alle serate passate a fare esercizi, non ho mai avuto dubbi che fosse tutto per il mio bene. Un grazie a mia sorella Rossella, da cui ho imparato che nella vita non sempre ci sono regole e orari da rispettare, soprattutto quando si tratta dell'orario di entrata a scuola. Infine, un pensiero di profonda riconoscenza va ai miei zii, cugini e parenti, che anche senza essere citati uno ad uno hanno avuto un'influenza importante. E ancora un grazie sentito va ai miei amici di una vita. Ad Ale, Ricky e Lollo per le mille esperienze, risate, discussioni, partite e viaggi insieme. Al Betti e al Lory per le serate nerd e di completa spensieratezza. A Dany, Ricky e Gio per le grandi bevute e mangiate, poi smaltite giocando a basket. Ringrazio anche Michele, Andrea, Federico, Marta e Laura per avermi accompagnato durante il mio Erasmus e a cui associo infinite risate. Ai miei compagni di università Filippo, Nicolò e Tommaso va il merito di aver reso lo studio più leggero e questa esperienza più condivisa. Vorrei infine menzionare anche Irene, Stefano, Gabriele, Andrea e in generale chiunque ha contribuito in questi anni a rendermi una persona migliore. Concludo ringraziando la professoressa Silvia Salini per la fiducia e il sostegno datomi durante la scrittura di questa tesi, a cui ho dedicato un anno di lavoro e non pochi pensieri. Orgoglioso e felice di questa esperienza, che porto con me gelosamente, ringrazio tutte le persone citate, che stimo vivamente e che hanno reso tutto questo possibile.

Contents

Acknowledgments	iii
1 Introduction	2
1.1 Context	2
1.2 Research Question	3
1.3 Thesis Structure	3
2 Dimensionality Reduction Algorithms	4
2.1 Introduction	4
2.2 Dimensionality Reduction Overview	6
2.3 Principal Component Analysis	8
2.4 t-Distributed Stochastic Neighbor Embedding	15
2.5 Uniform Manifold Approximation and Projection	26
3 Clustering Algorithms	37
3.1 Introduction	37
3.2 Types of Clustering Algorithms	38
3.3 Selecting the Optimal Number of Clusters	41
3.4 Algorithms Presentation	43
3.4.1 k-means Algorithm	43
3.4.2 PAM Algorithm	45
3.4.3 Gaussian Mixture Model Algorithm	48
3.4.4 Birch Algorithm	51
3.4.5 Spectral Clustering Algorithm	54

3.5	Concluding Remarks on Clustering Methods	56
3.6	Validation Measures	58
3.6.1	Internal Evaluation Metrics	58
3.6.2	External Evaluation Metrics	60
4	Methodology and Implementation	62
4.1	Core Idea	62
4.2	Synthetic Dataset Generation Pipeline	63
4.3	Cardinality and Dimensionality of the Dataset	69
4.4	Cluster Cardinality and Standard Deviation	71
4.5	Noise and Correlation Scaling Factor	74
5	Results and Discussion	76
5.1	Procedure	76
5.2	Study on Correlation Scaling Factor	77
5.3	Study on Noise	86
5.4	High-Dimensional Case Study	94
5.5	Final Conclusions	96
6	An Application on a Real Dataset	99
6.1	Introduction	99
6.2	Pre-processing Analysis	100
6.3	Clustering and Results	109
6.4	In-depth Analysis	113
6.4.1	PCA and k-means Configuration	114
6.4.2	UMAP and PAM Configuration	119
6.5	General Conclusions	125
7	Conclusions and Future Work	127
7.1	Final Conclusions	127
7.2	Future work	130

Abstract

Clustering is a fundamental tool in exploratory data analysis, as it helps discover latent patterns and structures that are difficult to find using descriptive statistics alone. However, the quality of the results depends significantly on the characteristics of the dataset, such as dimensionality, sample size, internal correlation and the presence of noise, making it difficult to choose the most suitable algorithm. Furthermore, dimensionality reduction techniques applied as preprocessing can improve clustering performance by deleting redundant information and improving cluster separability. Therefore this work proposes an experimental study in which synthetic datasets are generated by controlling various structural parameters to analyse the behaviour of five clustering algorithms (k-means, PAM, Gaussian Mixture Models, Birch, and Spectral Clustering), both individually and in combination with three dimensionality reduction methods (PCA, t-SNE, and UMAP). The goal is to understand which algorithm performs better and whether the combination with a pre-processing algorithm can improve the results. The same approach is then applied to a real dataset relating to the energy consumption of US households, with the aim of verifying the consistency of the experimental results in an applied context. The results show that the use of dimensionality reduction techniques frequently improves clustering quality, mainly in the presence of high dimensionality. In particular, UMAP appears as the most effective and robust preprocessing method, providing significant benefits on both synthetic and real data, largely when combined with partition-based algorithms such as k-means and PAM.

Chapter 1

Introduction

This chapter introduces the context in which the thesis work is set, defines the research question and describes the overall structure of the study.

1.1 Context

Descriptive analyses and unsupervised methods represent a crucial phase of any data science work and are often necessary for the application of supervised models. Precisely, it allows for the identification of redundant or potentially biasing observations, as well as assessing the presence of outliers, the possible need for transformations or standardization of variables. Among the most relevant techniques in this context is clustering, whose goal is to identify homogeneous groupings of observations based on their similarity. Clustering therefore provides a clear representation of the data and offers the data scientist a preliminary indication of the possible presence of segmentations or latent structures. However, choosing the most appropriate clustering algorithm is not always straightforward, as the performance of different methods depends significantly on the characteristics of the dataset. Factors such as the number of observations, dimensionality, internal correlation between variables and the presence of noise influence the geometric structure of the data and can make one algorithm more or less effective than another. To address this issue, this thesis proposes an experimental analysis based on the generation of synthetic datasets in different configurations, with the aim of studying the behaviour of various clustering algorithms,

including k-means, PAM, GMM, Birch and Spectral Clustering. In parallel, dimensionality reduction techniques are considered, in order to evaluate whether and how these methods can improve the quality of the clustering. This work recalls the study proposed by Grané Chavez, Riani and Salini [1], which introduces a comparative protocol for improving clustering procedures. In relation to this contribution, we will explore the role of dimensionality reduction within clustering algorithms, proposing a reinterpretation and an applicative extension both in simulated contexts and on real data.

1.2 Research Question

How do different clustering algorithms behave under varying dataset conditions (dimensionality, sample size, correlation, noise, and ground-truth number of clusters), and to what extent does dimensionality reduction improve clustering quality on both synthetic and real-world data?

1.3 Thesis Structure

Excluding the Introduction and Conclusions, the thesis is divided into five chapters, organized into two macro-sections. The first, theoretical, includes Chapters 2 and 3: Chapter 2 introduces the background to dimensionality reduction techniques (PCA, t-SNE, and UMAP), while Chapter 3 presents the main clustering algorithms considered in this work (k-means, PAM, GMM, Birch, and Spectral Clustering). The second macro-section is dedicated to the experimental analysis. In particular, Chapter 4 describes the methodological choices adopted, with a focus on the synthetic dataset generation procedure and the selection of experimental parameters. Chapter 5 discusses in depth the results obtained on the synthetic data, evaluating the impact of different configurations and preprocessing techniques. Finally, Chapter 6 applies the same workflow to a real-world dataset on US household energy consumption, to verify the consistency of the results in an applied context.

Chapter 2

Dimensionality Reduction Algorithms

This chapter introduces the theoretical background of dimensionality reduction algorithms that will be used in our work.

2.1 Introduction

Context Unlike supervised learning, whose goal is to predict or classify a set of p variables measured on n observations, unsupervised learning aims to identify interesting relationships or structures within the data itself, such as identifying subgroups among variables or observations. In our case, we will focus exclusively on unsupervised analysis and, in this chapter, we will delve into the topic of dimensionality reduction of a dataset. One problem that can arise when analysing these datasets is indeed the high dimensionality of the variables, a phenomenon commonly known as the "curse of dimensionality", which can be solved by applying unsupervised algorithms.

High dimensional datasets As the number of variables p in a dataset increases, the volume of the feature space grows exponentially. Consequently, even moderately sized datasets become increasingly sparse, and most observations tend to lie near the edges or corners of the high-dimensional space. This sparsity reduces the amount of local information available, making tasks such as clustering, visualization and pattern recognition more difficult.

Illustrative example A clear way to interpret this phenomenon is through a geometric analogy: a one-dimensional line can be sampled effectively with only a few points, whereas a two-dimensional square requires a larger number of samples to achieve comparable coverage of its area. Extending the same reasoning to three dimensions, a cube demands an even greater number of points. As the dimensionality increases further, the sampled space rapidly becomes extremely sparse. Mathematically, if one considers a p -dimensional unit cube, the side length of a sub-cube expected to contain half of the data points is $(0.5)^{1/p}$, which approaches 1 as p grows. Hence, the expected number of points in a fixed volume tends to zero, requiring very large sample sizes to capture local structure.

Challenges The curse of dimensionality manifests in several practical challenges:

1. Data sparsity: as the dimensionality increases, most of the feature space remains empty, making tasks such as clustering or classification increasingly difficult.
2. Model complexity: the number of potential models or functions to consider increases faster than exponentially with p , leading to combinatorial explosion and difficulty in selecting appropriate models.
3. Multicollinearity: in high-dimensional spaces, variables often become highly correlated, or lie on smooth manifolds, complicating regression and non-parametric analysis.
4. Computational cost: processing high-dimensional data requires higher computational resources and processing time.
5. Over-fitting: models may become excessively complex and start fitting noise in the data, thereby losing their generalization capability, especially when p is large relative to the number of observations. ¹

¹Over-fitting occurs when a model learns the training data too precisely, including random fluctuations or noise, which reduces its ability to make accurate predictions on unseen data. An over-fitted model is analogous to an invention that works perfectly in the lab but fails in real-world conditions.

6. Loss of distance meaning: in high-dimensional spaces, distances between points tend to become increasingly similar, making metrics such as the Euclidean distance less informative.
7. Visualization difficulty: exploring and interpreting datasets with many dimensions is challenging, limiting insight from exploratory data analysis.

Together, these aspects illustrate why dimensionality reduction is often necessary for high-dimensional data analysis, particularly in unsupervised learning tasks [2].

2.2 Dimensionality Reduction Overview

To address the challenges set by high-dimensional datasets, we employ dimensionality reduction algorithms that summarize multiple variables into a smaller set of components, preserving as much relevant information as possible.

Linearity of mapping A first fundamental distinction lies in the linearity of the mapping used by the algorithm.

- Linear algorithms: methods such as Principal Component Analysis (PCA) project the data into a lower-dimensional space using linear combinations of the original variables. These methods are computationally efficient and highly interpretable, making them suitable when the underlying relationships between variables are approximately linear. However, they may fail to capture complex nonlinear relationships among features.
- Nonlinear algorithms: approaches such as t-SNE (t-Distributed Stochastic Neighbor Embedding) and UMAP (Uniform Manifold Approximation and Projection) are capable of capturing complex, nonlinear relationships by modelling underlying structures of the data. Nonlinear methods are particularly effective for visualization and cluster detection, though they tend to be less immediately interpretable and more sensitive to parameter choices.

Preservation of pairwise distances Another important distinction focuses on how algorithms preserve distances between data points. Some methods aim to maintain the overall pairwise distance structure amongst all samples, emphasizing large-scale relationships across the dataset like PCA. Others favour the preservation of distances between nearby points, prioritizing local structure over global accuracy like t-SNE and UMAP. [3]

Preprocessing considerations Before applying any dimensionality reduction method, it is essential to ensure that all features are on a comparable scale. When variables are measured in different units or exhibit widely varying magnitudes, those with larger variances can strongly affect the results. To mitigate this issue, data are typically either standardized or normalized.

- Standardization transforms a variable x into a new variable x' with a mean of 0 and a standard deviation of 1, according to the formula:

$$x'_i = \frac{x_i - \mu_x}{\sigma_x}, \quad (2.2.1)$$

where μ_x is the mean of the variable x and σ_x its standard deviation. Standardization is particularly useful when the distribution of the variable is approximately Gaussian and preserves the shape of the original distribution.

- Normalization rescales a variable to a fixed range, typically $[0, 1]$, using the formula:

$$x'_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}, \quad (2.2.2)$$

where $\min(x)$ and $\max(x)$ are the minimum and maximum values of the variable x , respectively. Normalization is especially useful when the features have different units or the data do not follow a Gaussian distribution. [4]

Since PCA, t-SNE, and UMAP rely on distance or variance computations, performing standardization or normalization is crucial to obtain meaningful and unbiased low-dimensional representations.

2.3 Principal Component Analysis

Context When the number of variables considered is large, it is difficult to fully understand the relationships and structures within the data, making meaningful interpretation complex. An intuitive approach might be to examine a series of two-dimensional scatterplots, each representing measurements of the n observations against two of the p variables. However, there are $\binom{p}{2} = \frac{p(p-1)}{2}$ possible combinations of pairs of variables. For example, $p = 10$ would result in 45 distinct plots. For this reason, it is often necessary to reduce the dimensionality of the dataset, trying to minimize information loss. This goal can be achieved using Principal Component Analysis.

Introduction to PCA Principal Component Analysis (PCA) is a widely used unsupervised learning technique for data dimensionality reduction, since it involves only a set of features $X_1, X_2, \dots, X_p$, and no associated response Y . The algorithm's primary goal is to synthesize the information content of large data sets into a small number of uncorrelated variables, called principal components. These principal components are linear combinations of the original variables and are determined to maximize variance compared to other possible linear combinations. In this way, PCA manages to capture and preserve most of the information contained in the original dataset, while simultaneously reducing data complexity.

Algorithm Presentation

Covariance and correlation matrices PCA can be performed starting from the covariance matrix or the correlation matrix. In the first case, each principal component is expressed as a linear combination of the deviations from the mean of the p variables. In the second case, the components are obtained as linear combinations of the standardized deviations of the variables. Since PCA is based on maximizing variance and correlation coefficients, the correlations between the variables should be sufficiently high to allow them to be reduced to a smaller number of components. When this occurs, PCA allows us to identify a small set of dimensions that effectively

summarize the available information. More precisely, starting from a data matrix of size $n \times p$, containing only quantitative variables, PCA allows us to replace the original p (correlated) variables with a new set of variables, called principal components, which have the following properties:

- are uncorrelated with each other, so their loading vectors are orthogonal
- are ordered in decreasing order of the variance they explain

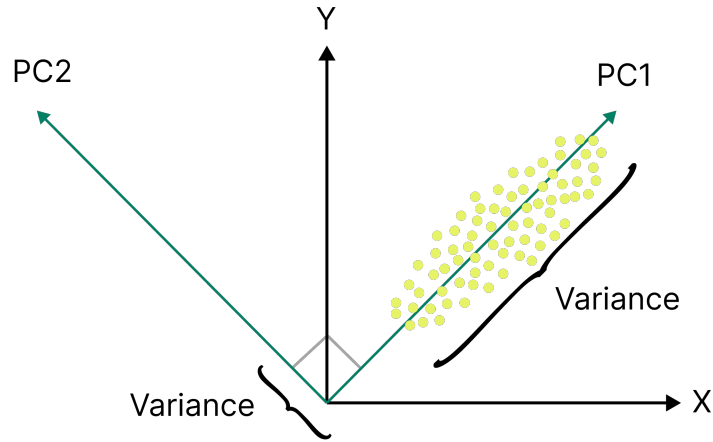


Figure 1: PCA visualization using two Principal Components that maximize variance

First principal component The first principal component of a set of variables $X_1, X_2, \dots, X_p$ is the normalized linear combination of the variables:

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \dots + \phi_{p1}X_p \quad (2.3.1)$$

that achieves the highest variance. Here, normalized means that the coefficients satisfy the constraint $\sum_{j=1}^p \phi_{j1}^2 = 1$, where the elements $\phi_{11}, \dots, \phi_{p1}$ are referred to as the loadings (weights) of the first principal component. This constraint is imposed to avoid trivial solutions, since arbitrarily large coefficients would otherwise yield unbounded variance. Moreover, because PCA focuses on variability, we assume that each variable in X has been centred, i.e., each column has zero mean. We then seek the linear combination of the observed values in the form:

$$z_{i1} = \phi_{11}x_{i1} + \phi_{21}x_{i2} + \cdots + \phi_{p1}x_{ip} \quad (2.3.2)$$

that maximizes the sample variance, subject to the constraint $\sum_{j=1}^p \phi_{j1}^2 = 1$. In other words, the loading vector of the first principal component solves the following optimization problem:

$$\begin{aligned} & \underset{\phi_{11}, \dots, \phi_{p1}}{\text{maximize}} && \frac{1}{n} \sum_{i=1}^n \left(\sum_{j=1}^p \phi_{j1} x_{ij} \right)^2 \\ & \text{subject to} && \sum_{j=1}^p \phi_{j1}^2 = 1. \end{aligned} \quad (2.3.3)$$

The equation can be rewritten as $\frac{1}{n} \sum_{i=1}^n z_{i1}^2$, where $z_{11}, \dots, z_{n1}$ represent the scores of the first principal component. This problem can be solved using an eigenvalue decomposition: the loading vector $\boldsymbol{\phi}_1 = (\phi_{11}, \phi_{21}, \dots, \phi_{p1})^\top$ defines a direction in the space of variables along which the data vary most. Equivalently, the score can be written as $z_{i1} = x_i^\top \boldsymbol{\phi}_1$. By projecting the points $x_1, \dots, x_n$ onto this direction, we obtain the scores $z_{11}, \dots, z_{n1}$ of the first principal component.

Second principal component Now the second principal component Z_2 can be computed. This is the linear combination of $X_1, \dots, X_p$ with the maximum variance among all those that are uncorrelated with Z_1 . The scores of the second principal component $z_{12}, z_{22}, \dots, z_{n2}$ can be expressed as:

$$z_{i2} = \phi_{12}x_{i1} + \phi_{22}x_{i2} + \cdots + \phi_{p2}x_{ip}, \quad (2.3.4)$$

where $\boldsymbol{\phi}_2 = (\phi_{12}, \phi_{22}, \dots, \phi_{p2})^\top$ is the loading vector of the second principal component. Constraining Z_2 to be uncorrelated with Z_1 is equivalent to imposing that the direction $\boldsymbol{\phi}_2$ be orthogonal to $\boldsymbol{\phi}_1$. To determine $\boldsymbol{\phi}_2$, a similar problem to the previous one is solved, adding the orthogonality constraint. Once the principal components have been calculated, they can be plotted against each other to obtain a low-dimensional visualization of the dataset, as shown in Figure 1. [5]

Variance explained We are interested in knowing the proportion of variance explained (PVE) by each principal component. The total variance present in a dataset after the variables have been centred to have mean zero is defined as:

$$\sum_{j=1}^p \text{Var}(X_j) = \sum_{j=1}^p \frac{1}{n} \sum_{i=1}^n x_{ij}^2 \quad (2.3.5)$$

The variance explained by the m -th principal component is:

$$\frac{1}{n} \sum_{i=1}^n z_{im}^2 = \frac{1}{n} \sum_{i=1}^n \left(\sum_{j=1}^p \phi_{jm} x_{ij} \right)^2 \quad (2.3.6)$$

Therefore, the proportion of variance explained by the m -th principal component is given by:

$$\text{PVE}_m = \frac{\sum_{i=1}^n z_{im}^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2} = \frac{\sum_{i=1}^n \left(\sum_{j=1}^p \phi_{jm} x_{ij} \right)^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2}. \quad (2.3.7)$$

The PVE of each principal component is a positive quantity. To compute the cumulative PVE of the first M principal components, we simply add the individual PVEs. In total, there are $\min(n-1, p)$ principal components, and their combined PVEs equal one. The total variance of the data can be decomposed into the variance explained by the first M principal components plus the mean squared error of the M -dimensional approximation, as follows:

$$\sum_{j=1}^p \frac{1}{n} \sum_{i=1}^n x_{ij}^2 = \sum_{m=1}^M \frac{1}{n} \sum_{i=1}^n z_{im}^2 + \frac{1}{n} \sum_{j=1}^p \sum_{i=1}^n \left(x_{ij} - \sum_{m=1}^M z_{im} \phi_{jm} \right)^2. \quad (2.3.8)$$

Since the first term is fixed, maximizing the variance of the first M principal components is equivalent to minimizing the mean squared error of the M -dimensional approximation, and vice versa. This explains why the principal components can be viewed both as maximizing variance and as minimizing reconstruction error. Furthermore, Equation 2.3.8 shows that the cumulative proportion of variance explained by the first M principal components can be written as:

$$1 - \frac{\sum_{j=1}^p \sum_{i=1}^n \left(x_{ij} - \sum_{m=1}^M z_{im} \phi_{jm} \right)^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2} = 1 - \frac{\text{RSS}}{\text{TSS}}, \quad (2.3.9)$$

where TSS^2 represents the total sum of squares of the elements of X , while RSS^3 represents the residual sum of squares of the M -dimensional approximation provided by the principal components. We can interpret the PVE as the analogue of R^2 for the approximation of X obtained via the first M principal components.

Number of components Choosing the optimal number of components can be done in several ways and there is no universally accepted objective rule for establishing the optimal number. To this end, three main criteria can be adopted:

1. Percentage of variance explained: a number of components is chosen that explains a sufficiently high percentage of the total variance. As p increases, the total variance increases, so it may be reasonable to consider a lower threshold for explained variance. Typically, the extracted components are required to explain between 95% of the mean variance of each of the initial p variables, corresponding to a threshold of:

$$0.95^p \times 100 \quad (2.3.10)$$

For example, for $p = 5$ we obtain 77.38%, for $p = 10$ 59.87%.

2. Kaiser Criterion: we retain all principal components whose eigenvalue is greater than 1. Since the eigenvalue represents the variance explained by a component and, in the case of standardized variables, each variable has unit variance, we retain a component only if it explains more variance than a single original variable.

²TSS = $\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2$.

³RSS = $\sum_{j=1}^p \sum_{i=1}^n \left(x_{ij} - \sum_{m=1}^M z_{im} \phi_{jm} \right)^2$.

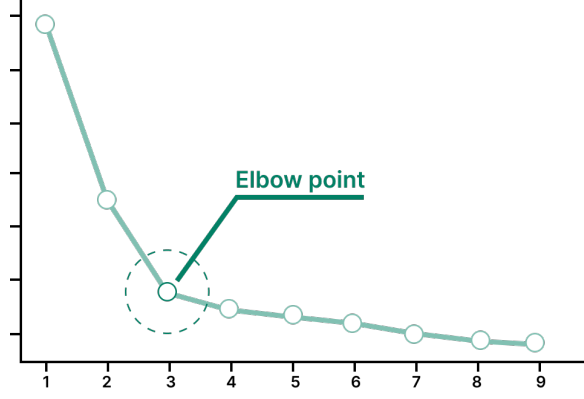


Figure 2: Scree plot example with elbow method

3. **Scree plot criterion:** a graph of the eigenvalues is constructed as a function of the component index $m = 1, 2, \dots, p$. If the first M components capture most of the relevant structure while the remaining $(p - M)$ components mainly represent noise, a sharp change in the slope of the eigenvalue curve is observed between M and $M + 1$, forming the so-called elbow. This point suggests the appropriate number of components to retain. [6]

Algorithm Summary

Algorithm 1: Principal Component Analysis (PCA)

Data: Data matrix $X \in \mathbb{R}^{n \times p}$,

Target dimensionality M (number of principal components)

Result: Score matrix $Z \in \mathbb{R}^{n \times M}$

begin

 Compute the sample mean vector: $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$

 Center the data: $\tilde{\mathbf{x}}_i = \mathbf{x}_i - \boldsymbol{\mu}$, $i = 1, \dots, n$

 Form the centred matrix $\tilde{X}$

 Compute the covariance matrix: $C = \frac{1}{n-1} \tilde{X}^\top \tilde{X}$

 Compute eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$ and eigenvectors $\boldsymbol{\phi}_1, \dots, \boldsymbol{\phi}_p$ of C

 Select the first M eigenvectors: $\boldsymbol{\Phi}_M = [\boldsymbol{\phi}_1, \dots, \boldsymbol{\phi}_M]$

 Project the data onto the principal components:

$$Z = \tilde{X} \boldsymbol{\Phi}_M$$

end

Weaknesses and conclusions Now that we have introduced PCA and outlined the main steps in its operation, we can analyse the algorithm’s main issues. This section will be of great help in understanding whether the issues listed above will be met personally during the analyses.

- **Linearity assumption:** PCA assumes that the relationships between variables are linear. In the presence of nonlinear or complex structures, this technique may fail to capture the most significant variations in the data, generating suboptimal results.
- **Sensitivity to variable scale:** PCA is strongly influenced by the data’s measurement scale. Variables with larger variances can dominate the analysis if proper normalization is not applied, leading to biased results.
- **Sensitivity to outliers:** Since PCA is based on variance, the presence of outliers can significantly alter the principal components and compromise the reliability of the results.
- **Computational cost:** PCA’s running time increases cubically with the number of dimensions in the dataset. This makes it computationally expensive when working with large data or a large number of variables.

Overall, PCA remains a widely adopted and effective dimensionality reduction technique thanks to its simplicity, interpretability, and ability to remove redundancy in high-dimensional data. When the underlying structure of the dataset can be reasonably approximated by linear relationships and the data are properly preprocessed, PCA provides a compact representation that can be extremely beneficial for subsequent analyses. However, its reliance on variance and linear projections limits its ability to capture complex nonlinear patterns, suggesting that alternative approaches may be more suitable in scenarios where the data exhibit strong nonlinear structures or manifold-like geometries.

2.4 t-Distributed Stochastic Neighbor Embedding

Context The fundamental idea behind t-Distributed Stochastic Neighbor Embedding (t-SNE) is to map high-dimensional data points into a lower-dimensional space, typically two or three dimensions, while preserving the local relationships among them. The algorithm begins by converting pairwise distances between data points in the high-dimensional space into conditional probabilities that represent similarities. It then constructs an analogous probability distribution in the lower-dimensional space. The goal of t-SNE is to minimize the divergence between these two distributions, typically measured using the Kullback–Leibler (KL) divergence, through an iterative optimization process based on gradient descent. By doing so, t-SNE effectively preserves local structure and cluster relationships, making it particularly powerful for visualizing complex, high-dimensional datasets and uncovering meaningful patterns that may be obscured in the original feature space. [7]

Comparison with PCA PCA is a well-established technique for dimensionality reduction and visualization; however, it is a linear method, which limits its ability to capture complex, non-linear relationships within the data. A key limitation of PCA is its tendency to map dissimilar data points far apart without adequately preserving local neighborhood structures. In datasets where the data lie on a non-linear manifold, similar points should remain close together even after projection, something linear methods often fail to achieve. In contrast, t-Distributed Stochastic Neighbor Embedding (t-SNE) is a non-linear dimensionality reduction technique based on probabilistic similarity measures and a KL-divergence minimization objective. It aims to uncover the underlying manifold structure of high-dimensional data by preserving local relationships between points and clustering structure among data rather than global variance. Moreover, unlike PCA, the distances between clusters in a t-SNE embedding should be interpreted with caution, since the method primarily preserves local neighborhood structure rather than global geometry.

Algorithm Presentation

The foundations of t-SNE To gain a deeper understanding of how t-SNE works, it is essential to start from the algorithm on which it was originally built: Stochastic Neighbor Embedding. SNE begins by converting the high-dimensional Euclidean distances between data points into conditional probabilities that represent similarities. The similarity of a data point x_j to another data point x_i is defined as the conditional probability $p_{j|i}$, which represents the probability that x_i would choose x_j as its neighbor if neighbors were selected proportionally to their probability density under a Gaussian distribution centred at x_i .

Similarity in high-dimensional space For nearby data points, the value of $p_{j|i}$ is relatively high, whereas for widely separated points, $p_{j|i}$ becomes almost infinitesimal, depending on the variance of the Gaussian σ_i . Mathematically, the conditional probability $p_{j|i}$ is defined as:

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}, \quad p_{i|i} = 0. \quad (2.4.1)$$

where σ_i is the variance of the Gaussian distribution centred on data point x_i .

Similarity in low-dimensional space For the low-dimensional counterparts y_i and y_j of the high-dimensional data points x_i and x_j , it is possible to compute an analogous conditional probability, denoted as $q_{j|i}$. The variance of the Gaussian distribution used to compute these conditional probabilities is set to $\frac{1}{\sqrt{2}}$ ⁴. Hence, the similarity of the mapped point y_j to y_i is modelled as:

$$q_{j|i} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq i} \exp(-\|y_i - y_k\|^2)}, \quad q_{i|i} = 0. \quad (2.4.2)$$

⁴Adjusting the variance of the low-dimensional Gaussians only affects the scale of the resulting map. It is important to note that using a single, shared variance for all data points in the low-dimensional embedding removes the self-modelling property of the data. This is because, unlike in the high-dimensional space where each Gaussian has its own variance σ_i , the low-dimensional space assumes uniform variance across all points.

If the low-dimensional points y_i and y_j accurately reflect the similarities between the high-dimensional data points x_i and x_j , then the conditional probabilities $p_{j|i}$ and $q_{j|i}$ will be approximately equal. Therefore, the primary objective of SNE is to find a low-dimensional representation that minimizes the discrepancy between $p_{j|i}$ and $q_{j|i}$.

SNE cost function A natural measure for the mismatch between original and low-dimensional conditional probabilities is the Kullback–Leibler (KL) divergence, which quantifies the difference between the two probability distributions. The SNE algorithm minimizes the sum of the KL divergences across all data points using a gradient descent optimization method.⁵ The cost function C of the SNE algorithm is defined as:

$$C = \sum_i \text{KL}(P_i \| Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}} \quad (2.4.3)$$

where P_i represents the conditional probability distribution over all other data points given data point x_i , and Q_i represents the corresponding conditional probability distribution over all other map points given map point y_i .

Kullback–Leibler divergence Because the Kullback–Leibler (KL) divergence is not symmetric, different types of errors in the pairwise distances within the low-dimensional map are not penalized equally. In particular, there is a large penalty when widely separated map points are used to represent nearby high-dimensional points (i.e. when a small $q_{j|i}$ is used to model a large $p_{j|i}$). Conversely, using nearby map points to represent distant data points results in only a small cost. In other words, the SNE cost function places greater emphasis on preserving the local structure of the data in the low-dimensional representation for reasonable values of the Gaussian variance σ_i in the high-dimensional space.

⁵Gradient descent is an iterative optimization algorithm used to minimize an objective function $J(\theta)$, parameterized by $\theta \in \mathbb{R}^d$, by updating the parameters in the direction opposite to the gradient $\nabla_{\theta} J(\theta)$. The learning rate η determines the size of the steps taken along the surface of the function, guiding the algorithm "downhill" towards a local minimum. In this way, by progressively reducing the value of the objective function, gradient descent also allows us to reduce the error between predicted and actual values in machine learning models, improving their predictive performance. [8]

Variance The remaining parameter to be determined in SNE is the variance σ_i of the Gaussian distribution centred on each high-dimensional data point x_i . It is not likely to have a unique value of σ_i because in dense regions a smaller value is usually more appropriate than in sparser regions. Each particular value of σ_i defines a probability distribution P_i over all other data points. The entropy associated with the distribution increases as σ_i increases, reflecting greater uncertainty in neighborhood relationships. SNE determines the optimal value of σ_i for each data point through a binary search procedure, such that the resulting distribution P_i achieves a user-specified perplexity, defined as:

$$\text{Perp}(P_i) = 2^{H(P_i)}, \quad (2.4.4)$$

where $H(P_i)$ denotes the Shannon entropy ⁶ of P_i , measured in bits:

$$H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}. \quad (2.4.5)$$

Perplexity Perplexity can be interpreted as a measure of the effective number of neighbors for each data point we are considering when building the similarity map and controls how the algorithm balances local versus global relationships in the data. In practice, SNE performance is relatively stable across different perplexity values, which typically range from 5 to 50. Setting a low perplexity value causes the algorithm to consider only a few neighbors when assessing similarity, making it focus primarily on local relationships. As a result, SNE becomes sensitive to small differences between observations and aims to preserve tight clusters of very similar profiles. Conversely, using a high perplexity value leads the algorithm to take more neighbors into account, improving its ability to capture global relationships. In this case, individual clusters may appear less compact, but the overall structure of cell types tends to be clearer.

⁶Shannon entropy is defined as the average rate at which information is produced by a stochastic source of data, where higher entropy indicates greater information content from new values. It is capable of detecting nonlinearity aspects in model series, contributing to a more reliable explanation regarding the nonlinear dynamics of different points of analysis, which in turn enhances the comprehension of the nature of complex systems characterized by complexity and non equilibrium [9]

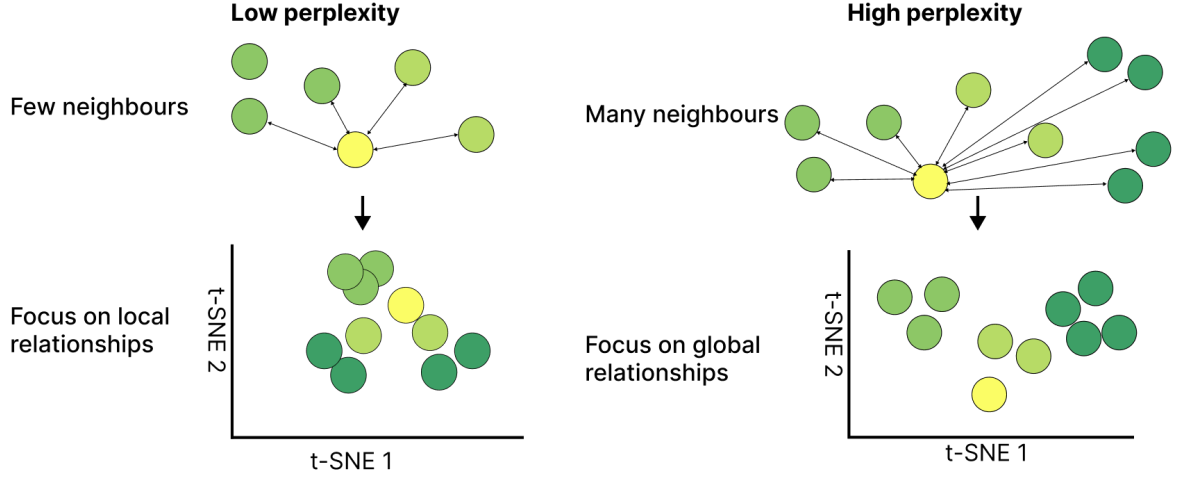


Figure 3: How perplexity affects low-dimensional representation.

In our case, when applying t-SNE to our dataset, we will use the default perplexity value of 30, which provides a good balance between capturing local and global data structure.

Cost function minimization The cost function previously seen is minimized using a gradient descent method, that takes the following form:

$$\frac{\partial C}{\partial y_i} = 2 \sum_j ((p_{j|i} - q_{j|i}) + (p_{i|j} - q_{i|j})) (y_i - y_j) \quad (2.4.6)$$

Fundamentally, the gradient can be interpreted as the net force generated by a set of springs between the map point y_i and all other points in the map y_j . All springs exert a force in the direction $(y_i - y_j)$. The spring between y_i and y_j attracts or repels the points of the map depending on whether the distance between the two is too small or too large to correctly represent the similarities between the two observations in high-dimensional space. The force deployed by the spring between y_i and y_j is also proportional to its hardness, which is given by the gap $(p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j})$ between the similarities of pairs of points in the original data and in the map. This drives the points toward a configuration that better preserves the original similarities, gradually reducing the mismatch between the two spaces.

Gradient descent process The gradient descent process begins by sampling map points randomly from an isotropic Gaussian distribution with a small variance, centred at the origin. To speed up the optimization and reduce the risk of converging to undesirable local minima, a relatively large momentum term is introduced into the gradient. In other words, the current gradient is added to an exponentially decreasing combination of previous gradients to determine the change in coordinates of the map points at each iteration of the optimization process. Mathematically, the momentum gradient update is expressed as:

$$Y(t) = Y(t-1) + \eta \frac{\partial C}{\partial Y} + \alpha(t)(Y(t-1) - Y(t-2)) \quad (2.4.7)$$

where $Y(t)$ represents the solution at iteration t , η is the learning rate, and $\alpha(t)$ denotes the momentum term at iteration t . In the early stages of optimization, Gaussian noise is added to the map points after each iteration. Gradually reducing the noise variance helps the algorithm avoid poor local minima and improves the global structure of the map. However, this approach requires careful selection of both the initial noise level and its decay rate.

From SNE to t-SNE Although SNE produces reasonably good visualizations, it suffers from two main disadvantages: a cost function that is difficult to optimize and a phenomenon known as the crowding problem⁷. To overcome these limitations, a modified algorithm called t-Distributed Stochastic Neighbor Embedding (t-SNE) was introduced. The cost function used in t-SNE differs from that of SNE in two key ways:

1. It employs a symmetrized version of the SNE cost function, which leads to simpler gradients and more stable optimization,

⁷The crowding problem in SNE and t-SNE arises because, when projecting data from a high-dimensional space to a lower-dimensional one (typically two dimensions), the available space in the map is insufficient to accurately represent the distances between moderately distant points in the original space. In SNE, this causes points to become overly concentrated near the centre, making clusters difficult to separate. t-SNE addresses this issue by using a Student's t-distribution with heavier tails to model distances in the low-dimensional space, which allows for greater separation between clusters and reduces the crowding effect.

2. It replaces the Gaussian distribution with a heavy-tailed Student's t-distribution when computing the similarities between points in the low-dimensional space.

These modifications improve the preservation of local neighborhood structure and lead to clearer visual separations in the embedding.

Symmetric SNE Alternatively, instead of minimizing the sum of Kullback–Leibler (KL) divergences between the conditional probabilities $p_{j|i}$ and $q_{j|i}$, one can minimize a single KL divergence between a joint probability distribution P in the high-dimensional space and a corresponding joint probability distribution Q in the low-dimensional space:

$$C = \text{KL}(P\|Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (2.4.8)$$

where

$$q_{ij} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq l} \exp(-\|y_k - y_l\|^2)}, \quad q_{ii} = 0. \quad (2.4.9)$$

$$p_{ij} = \frac{\exp(-\|x_i - x_j\|^2/2\sigma^2)}{\sum_{k \neq l} \exp(-\|x_k - x_l\|^2/2\sigma^2)}, \quad p_{ii} = 0. \quad (2.4.10)$$

However, this approach can create problems when a high-dimensional data point x_i is an outlier, meaning that all pairwise distances $\|x_i - x_j\|^2$ are large. For such an outlier, the values of p_{ij} become extremely small for all j , so the corresponding low-dimensional point y_i has very little influence on the cost function. As a result, its position in the map is poorly determined relative to the other points. To address this issue, we define the joint probabilities p_{ij} in the high-dimensional space as symmetrized conditional probabilities:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}. \quad (2.4.11)$$

where n is the number of data points. This variant of SNE is referred to as symmetric SNE, since it satisfies $p_{ij} = p_{ji}$ and $q_{ij} = q_{ji}$ for all pairs (i, j) . The main advantage of the symmetric formulation is that it produces a simpler and more computationally efficient gradient:

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j), \quad (2.4.12)$$

Introduction to t-SNE Since symmetric SNE matches the joint probabilities of pairs of data points in the high-dimensional and low-dimensional spaces, rather than their absolute distances, it provides a natural way to alleviate the crowding problem. The idea is to use different probability distributions in the two spaces to handle the mismatch in dimensionality. In the high-dimensional space, distances are converted into probabilities using a Gaussian distribution. In the low-dimensional map, however, we can use a probability distribution with much heavier tails than a Gaussian to perform this conversion. This choice allows moderate distances in the high-dimensional space to be represented by much larger distances in the map, thereby eliminating the undesired attractive forces between map points that correspond to moderately dissimilar data points.

Student's t-distribution In t-SNE, we use a Student's t-distribution⁸ with one degree of freedom (equivalent to a Cauchy distribution) as the heavy-tailed distribution in the low-dimensional space. Using this distribution, the joint probabilities q_{ij} are defined as:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}. \quad (2.4.14)$$

We choose a Student's t-distribution with a single degree of freedom because it possesses a particularly useful property: the term $(1 + \|y_i - y_j\|^2)^{-1}$ approximates an inverse-square law for large pairwise distances $\|y_i - y_j\|$ in the low-dimensional map.

⁸The Student's t-distribution is a probability distribution that arises when estimating the mean of a normally distributed population when the sample size is small and the population standard deviation is unknown. It is defined by its probability density function (PDF):

$$f(t|\nu) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi} \Gamma(\frac{\nu}{2})} \left(1 + \frac{t^2}{\nu}\right)^{-\frac{\nu+1}{2}}, \quad (2.4.13)$$

where ν denotes the degrees of freedom and $\Gamma(\cdot)$ is the gamma function. The t-distribution has heavier tails than the normal distribution, which makes it more robust to outliers and large deviations.

This makes the map's representation of joint probabilities nearly invariant to changes in scale for distant points.⁹ The gradient of the Kullback-Leibler divergence between P and the Student-t based joint probability distribution Q is given by:

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j) (1 + \|y_i - y_j\|^2)^{-1} \quad (2.4.15)$$

t-SNE algorithm The t-SNE gradient exhibits two important characteristics that significantly improve upon those of SNE. First, t-SNE introduces a strong repulsive force between dissimilar data points that are incorrectly modelled as being too close in the low-dimensional space. While SNE also incorporates repulsion between dissimilar points, its effect is much weaker and often overshadowed by strong attractive forces between similar points.

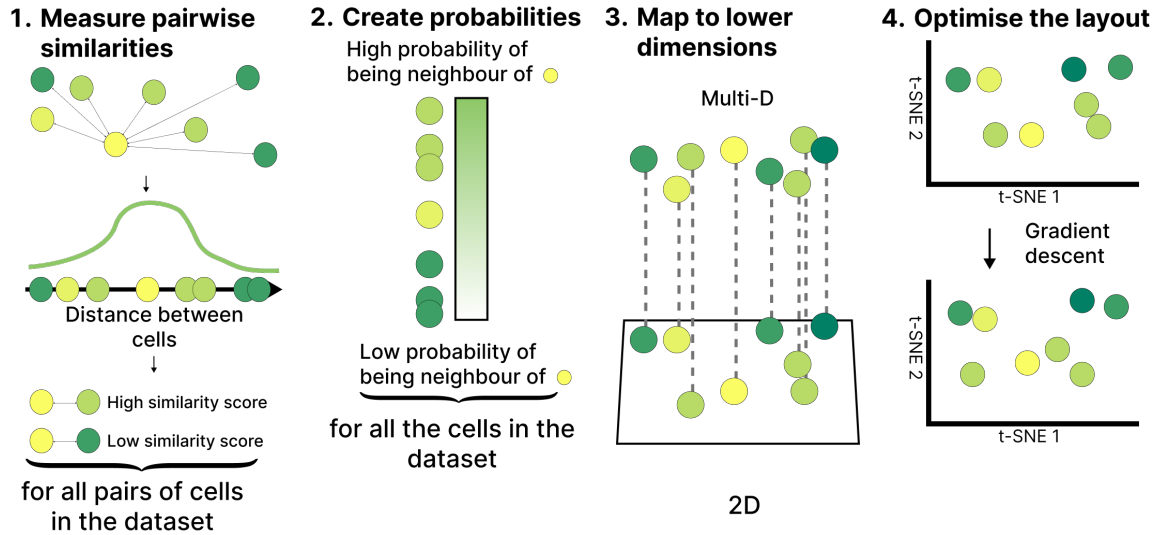


Figure 4: t-SNE projection of high-dimensional data into 2D space.

Second, although t-SNE introduces these strong repulsions, they do not approach infinity. This ensures numerical stability during optimization and prevents points from

⁹A theoretical justification for the selection of the Student t-distribution is that it is closely related to the Gaussian distribution, as the Student t-distribution is an infinite mixture of Gaussians. A computationally convenient property is that it is much faster to evaluate the density of a point under a Student t-distribution than under a Gaussian because it does not involve an exponential, even though the Student t-distribution is equivalent to an infinite mixture of Gaussians with different variances.

diverging excessively in the low-dimensional map. As a result of these properties and due to the approximate scale invariance of the Student’s t-distribution, the optimization of the t-SNE cost function is substantially easier and more stable than that of the original SNE formulation. This leads to clearer visualizations and faster convergence while maintaining both local and global structure in the resulting embeddings.

Algorithm Summary

Algorithm 2: t-Distributed Stochastic Neighbor Embedding (t-SNE)

Data: Dataset $X = \{x_1, x_2, \dots, x_n\}$,

Cost function parameter: perplexity $Perp$,

Optimization parameters: number of iterations T , learning rate η , momentum $\alpha(t)$

Result: Low-dimensional data representation $Y^{(T)} = \{y_1, y_2, \dots, y_n\}$

begin

 Compute pairwise affinities $p_{j|i}$ with perplexity $Perp$

 Set $p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$

 Sample initial solution $Y^{(0)} = \{y_1, y_2, \dots, y_n\}$ from $\mathcal{N}(0, 10^{-4}I)$

for $t = 1$ **to** T **do**

 Compute low-dimensional affinities q_{ij}

 Compute gradient $\frac{\partial C}{\partial Y}$

 Update map points:

$$Y^{(t)} = Y^{(t-1)} + \eta \frac{\partial C}{\partial Y} + \alpha(t) (Y^{(t-1)} - Y^{(t-2)})$$

end

end

Weaknesses and conclusions Although t-SNE has proven to be an effective method for data visualization, it presents three main weaknesses:

1. Dimensionality reduction beyond visualization: t-SNE is primarily designed for visualizing data in two or three dimensions. Its performance for higher-dimensional embeddings ($d > 3$) remains uncertain, as the heavy tails of the Student’s t-distribution dominate a larger portion of the probability mass in higher dimensions, leading to representations that do not preserve local structure of the data as well. For such cases, using t-distributions with more degrees

of freedom could return better results since by increasing the degrees of freedom the tails of the t-distribution gets lighter.¹⁰

2. Curse of intrinsic dimensionality: because t-SNE relies mainly on local Euclidean distances, it can struggle with datasets that have high complex nonlinear manifolds. In such cases, the local linearity assumption may be violated. A partial solution is to apply t-SNE to features extracted from deep nonlinear models such as auto-encoders¹¹, which can capture manifold structure more effectively before visualization. Nevertheless, it remains impossible to fully represent highly complex manifolds in only two or three dimensions.
3. Non-convexity of the cost function: the t-SNE cost function is non-convex, meaning that optimization may converge to different local minima depending on initialization and parameter choices. While this may lead to variations between runs, empirical results show that the quality of visualizations is generally robust. Moreover, convex alternatives often produce less meaningful embeddings and are computationally expensive for large-scale data. Thus, despite its non-convexity, t-SNE remains a practical and powerful approach for visual data analysis [11].

Overall, t-SNE remains one of the most effective techniques for visual exploration of high-dimensional data, as it can reveal meaningful local structures and cluster-like patterns in low-dimensional space. However, due to its sensitivity to hyperparameters, non-convex optimization, and reduced stability beyond 2 or 3 dimensional embeddings, its effectiveness may decrease in more complex settings, and the resulting representations should be interpreted with caution when used as a preprocessing step.

¹⁰With infinite degrees of freedom, the Student-t distribution is equal to the Gaussian distribution.

¹¹In machine learning, an auto-encoder is an unsupervised learning algorithm in which the input is equal to the output, with the goal of reconstructing the original data with minimal distortion. This technique is widely used to compress data, reducing storage space, speeding up computation times, improving performance by removing redundant variables, visualizing high-dimensional data, and filtering noise from the original data. During training, the auto-encoder learns to represent the data using latent variables, which are not directly observable but are essential for describing the data distribution: this representation, called latent space, captures only the essential information needed for accurate reconstruction of the input. [10]

2.5 Uniform Manifold Approximation and Projection

Context UMAP is based on the idea that high-dimensional data often lie on a lower-dimensional manifold ¹², a curved surface that describes the intrinsic structure of the data. The underlying principle of UMAP is to identify a compact representation that preserves the local geometry of the data. The first step of the algorithm consists of constructing a high-dimensional weighted graph in which each point is connected to its nearest neighbors. In this graph, the strength of the connection between two points represents the probability or degree of mutual membership in the same neighborhood. In this way, UMAP ensures that the local structure of the data is preserved, balancing it with global information about the overall distribution, often resulting in improved preservation of global organization compared to t-SNE.

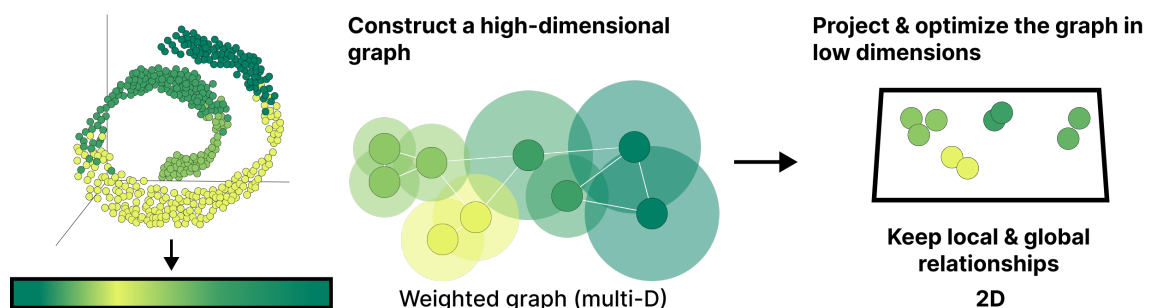


Figure 5: UMAP dimensionality reduction from p dimensions to two dimensions.

Next, UMAP optimizes the layout of a low-dimensional graph, attempting to maintain the connections defined in the high-dimensional weighted graph. Although the principle is similar to that of t-SNE, the mathematical formulation is different: t-SNE uses probability distributions to model the relationships between points, while UMAP relies on topological concepts, derived from the study of shapes and connections, to

¹²A manifold is an abstract topological space in which the neighborhood of each point is a subset of the Euclidean space, however the global structure of a manifold may be more complicated. For example, a line and a curve are one dimensional manifolds, since the neighborhood of each point on the one dimensional manifold is a line segment. A plane or the surface of a ball are instead two dimensional manifolds. The neighborhood of each point on the two dimensional manifold is a flat region. Informally, a manifold is a smooth low dimensional surface embedded in a higher dimensional space. [12]

preserve the manifold structure. The goal of this phase is to maintain consistency between local neighborhoods while preserving the global relationships present in the data. As a final result, for each point in the dataset, two coordinates are obtained, representing the position of each point in the reduced two-dimensional space.

Algorithm Presentation

Comparison with t-SNE The t-SNE algorithm is one of the most effective techniques for nonlinear dimensionality reduction. However, UMAP has emerged as a strong alternative, offering comparable representation quality while providing superior computational efficiency and better preservation of global data structure. Unlike t-SNE, which updates all data points at each optimization step, UMAP optimizes the embedding using stochastic updates on sampled point pairs, allowing it to scale efficiently to very large datasets and higher-dimensional embeddings. Conceptually, the two methods differ in how they model the data relationships. t-SNE relies on minimizing the Kullback–Leibler divergence between pairwise probability distributions, which emphasizes local neighborhoods, whereas UMAP constructs a topological representation of the data manifold through a weighted graph, capturing both local and global structure. As a result, UMAP tends to preserve the overall organization of clusters and their relative positions more effectively, while t-SNE focuses on fine local detail at the expense of global coherence. Nevertheless, it is important to note that, since both algorithms project data into a lower-dimensional space, the resulting coordinates and distances are not directly interpretable, unlike in linear techniques such as PCA. Another important distinction concerns stability. Although both algorithms are stochastic, UMAP yields more consistent projections across multiple runs and parameter variations. This is largely due to its initialization strategy: t-SNE begins from a random embedding, while UMAP can use a spectral embedding initialization that provides a deterministic starting point. Ultimately, UMAP’s stability and scalability make it more suitable for exploring large and complex datasets, whereas t-SNE remains advantageous when the focus is on visualization of local structures. It should

be noted, however, that in complex datasets, UMAP may sometimes struggle to separate a dense, compact cluster embedded within a larger, sparse cluster, whereas t-SNE can occasionally resolve such nested structures more clearly. [13, 3]

A computational view of UMAP UMAP can be viewed, from a computational perspective, as a sequence of operations for constructing and manipulating weighted graphs, thus falling into the category of k-nearest neighbor learning algorithms. The algorithm is divided into two main phases: in the first, a weighted graph based on the nearest neighbors is constructed, applying a transformation of the edge weights based on local distances and correcting any intrinsic asymmetries or distortions; in the second phase, a layout of the graph is computed in a low-dimensional space, optimizing an objective function that preserves the topological characteristics of the graph itself. These procedures are based on the following fundamental axioms:

1. There exists a manifold on which the data are approximately uniformly distributed.
2. The underlying manifold is locally connected.
3. The primary goal is to preserve the topological structure of that manifold. ¹³

These assumptions guide the construction of the neighbor graph and its low-dimensional embedding.

Graph construction The first phase of UMAP consists of constructing a weighted graph of k-neighbors. Let $X = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, with $\mathbf{x}_i \in \mathbb{R}^p$, be the input dataset, with a metric ¹⁴ or dissimilarity measure $d : X \times X \rightarrow \mathbb{R}_{\geq 0}$. Given a hyper-parameter k , for each point $\mathbf{x}_i$, the set of its k nearest neighbors $\{\mathbf{x}_{i_1}, \dots, \mathbf{x}_{i_k}\}$ is computed with

¹³The key purpose is to preserve neighborhood relationships between point. If multidimensional points are close to each other, the points representing them in the low dimensional space should also be close. In some cases, it is like unfolding a nonlinear manifold. [12]

¹⁴In most applications, the Euclidean distance is used as the metric d , defined as $d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_2 = \sqrt{\sum_{l=1}^p (x_{il} - x_{jl})^2}$, where p is the dimensionality of the data points.

respect to the metric d . This calculation is performed by using the Nearest Neighbor Descent algorithm ¹⁵. For each point $\mathbf{x}_i$, we define the local parameters ρ_i and σ_i as:

$$\rho_i = \min\{d(\mathbf{x}_i, \mathbf{x}_{i_j}) \mid 1 \leq j \leq k, d(\mathbf{x}_i, \mathbf{x}_{i_j}) > 0\} \quad (2.5.1)$$

and σ_i is the value such that:

$$\sum_{j=1}^k \exp\left(-\frac{\max(0, d(\mathbf{x}_i, \mathbf{x}_{i_j}) - \rho_i)}{\sigma_i}\right) = \log_2(k) \quad (2.5.2)$$

The choice of ρ_i arises from the local connectivity constraint, which guarantees that every point $\mathbf{x}_i$ is connected to at least one other point with an edge of weight 1. In other words, even if a point is isolated or located in a low-density region, UMAP requires that it have at least one significant connection to another point in the graph. This ensures that the resulting weighted graph is locally connected at $\mathbf{x}_i$, which improves robustness in high-dimensional settings, where neighborhood relationships can become less informative. The selection of the σ_i parameter acts as a smoothed normalization factor. Together with ρ_i , it determines the effective scale of distances in the local neighborhood of $\mathbf{x}_i$, adapting to the density of points. Intuitively, one can think of this as defining a local geometry for each point: in dense regions, distances are compressed, while in sparse regions, they are expanded, so that the manifold’s local structure is faithfully captured. This local geometry, which specifies how distances are measured in the neighborhood of each point, is what UMAP approximates as a local Riemannian metric ¹⁶. We therefore define a weighted directed graph $G = (V, E, \omega)$,

¹⁵The idea behind Nearest Neighbor Descent is to start from an initial estimate of the neighbor graph—usually chosen randomly—and progressively improve that estimate through an iterative process. In each iteration, for each element i , not only the current neighbors of i but also the neighbors of its neighbors are considered as candidate neighbors. For example, if an element i has j as its neighbor, then the neighbors of j become potential new neighbors of i . The term descent recalls the analogy with gradient descent: the sum of the distances between connected nodes in the graph can be interpreted as an objective function to be minimized. As the graph is updated and better neighbors are found, these overall distances tend to decrease, progressively improving the quality of the neighbor graph. [14]

¹⁶A Riemannian metric on a smooth manifold M is a smoothly varying inner product g_p on each tangent space $T_p M$. In the context of UMAP, the parameters (ρ_i, σ_i) provide a discrete approximation of this concept, defining the scale of distances in each point’s local neighborhood. [15]

where:

- the vertices V correspond to the points of the dataset X ;
- the directed edges correspond to $E = \{(\mathbf{x}_i, \mathbf{x}_{i_j}) \mid 1 \leq j \leq k, 1 \leq i \leq n\}$ and represent the connections between pairs of points that are close together;
- weights are defined as:

$$\omega(\mathbf{x}_i, \mathbf{x}_{i_j}) = \exp \left(-\frac{\max(0, d(\mathbf{x}_i, \mathbf{x}_{i_j}) - \rho_i)}{\sigma_i} \right) \quad (2.5.3)$$

For each point $\mathbf{x}_i$, UMAP defines a local weighted graph consisting of $\mathbf{x}_i$ and its outgoing edges to its nearest neighbors. Each local graph represents the neighborhood structure around $\mathbf{x}_i$ and encodes the relative strength of connections to nearby points through the edge weights. The value $\omega(\mathbf{x}_i, \mathbf{x}_{i_j})$ can be interpreted as the probability or strength of connection between $\mathbf{x}_i$ and $\mathbf{x}_{i_j}$, reflecting how strongly the two points are related in the local geometry of the data. By combining all these local graphs, UMAP constructs a global weighted structure that captures both the local and overall relationships among data points. Let $\mathbf{A}$ denote the weighted adjacency matrix of the resulting directed graph. To obtain an undirected representation, UMAP applies the following symmetrization rule:

$$\mathbf{B} = \mathbf{A} + \mathbf{A}^T - \mathbf{A} \odot \mathbf{A}^T, \quad (2.5.4)$$

where $\odot$ denotes the element-wise product. Interpreting A_{ij} as the probability that a directed edge exists from $\mathbf{x}_i$ to $\mathbf{x}_j$, the entry B_{ij} represents the probability that at least one of the two directed edges between $\mathbf{x}_i$ and $\mathbf{x}_j$ exists. The initial graph $G = (V, E, \omega)$ is directed and weighted because the local parameters (ρ_i, σ_i) are computed independently for each point. Consequently, the connection strength from $\mathbf{x}_i$ to $\mathbf{x}_j$, given by $\omega(\mathbf{x}_i, \mathbf{x}_j)$, may differ from the reverse connection $\omega(\mathbf{x}_j, \mathbf{x}_i)$. This asymmetry reflects the fact that the neighborhood of $\mathbf{x}_i$ is not necessarily the same as that of $\mathbf{x}_j$; a point can be among the k -nearest neighbors of another without the

reverse being true. Through the symmetrization step defined above, UMAP transforms this directed structure into an undirected weighted graph, where each edge weight B_{ij} captures the combined probability of connection between $\mathbf{x}_i$ and $\mathbf{x}_j$, regardless of direction. In essence, B_{ij} quantifies the likelihood that a meaningful relationship exists between the two points, accounting for both possible directions of connection. The resulting matrix $\mathbf{B}$ defines the adjacency of the final undirected graph that UMAP uses to approximate the manifold’s structure, ensuring that local geometric information is preserved while maintaining a globally symmetric and coherent representation of the dataset.

Graph layout The second phase of the algorithm consists of computing a graph layout in a low-dimensional space. Let $Y = \{\mathbf{y}_1, \dots, \mathbf{y}_n\}$, with $\mathbf{y}_i \in \mathbb{R}^d$, denote the low-dimensional embedding of the dataset. This approach relies on attractive forces applied to the edges and repulsive forces between the vertices, which are iterated until a stable equilibrium is reached. This process is equivalent to solving a non-convex optimization problem. Convergence to a local minimum is guaranteed by gradually reducing the strength of the attractive and repulsive forces. The attraction between two vertices i and j , with coordinates $\mathbf{y}_i$ and $\mathbf{y}_j$ in the embedding space, is defined as:

$$-2ab \cdot \frac{\|\mathbf{y}_i - \mathbf{y}_j\|_2^{2(b-1)}}{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|_2^2)} \cdot \omega(\mathbf{x}_i, \mathbf{x}_j) \cdot (\mathbf{y}_i - \mathbf{y}_j) \quad (2.5.5)$$

where a and b are hyperparameters that control the shape of the attraction function. Due to computational constraints, repulsive forces are calculated by sampling. Each time an attractive force is applied to an edge, a sample of other vertices that are not connected in the original graph is selected to apply the repulsive force:

$$\frac{2b}{(\epsilon + \|\mathbf{y}_i - \mathbf{y}_j\|_2^2) \cdot (1 + a\|\mathbf{y}_i - \mathbf{y}_j\|_2^{2b})} \cdot (1 - \omega(\mathbf{x}_i, \mathbf{x}_j))(\mathbf{y}_i - \mathbf{y}_j) \quad (2.5.6)$$

where ϵ is a small positive value to avoid division by zero. These sampled points are known as negative samples, and they efficiently simulate repulsive interactions without computing all pairwise non-neighbor forces. Therefore, for each pair of nearby points

the attractive force is applied, while for the repulsive forces some non-nearby points (negative samples) are randomly selected to be pushed away in the reduced space. To improve both stability and convergence speed, UMAP employs a spectral initialization of the embedding, based on the eigenvectors of a normalized Laplacian derived from the graph G . In simple terms, spectral initialization uses the properties of the Laplacian matrix to obtain an initial low-dimensional layout that already reflects the overall structure of the data. The Laplacian encodes how each data point connects to its neighbors, and its eigenvectors provide a smooth, geometry-aware representation of the graph. By projecting the data according to these eigenvectors, UMAP starts the optimization from a configuration that is already consistent with the intrinsic relationships among points, rather than from a random arrangement. The normalized Laplacian serves to balance the influence of nodes with different degrees, preventing highly connected points from dominating the layout. This normalization ensures that distances and connections are treated consistently across the graph, making the spectral initialization both stable and representative of the data’s true geometry. As a result, the algorithm converges faster and avoids poor local minima compared to random initialization. The optimization process can be interpreted as minimizing the cross-entropy divergence between the weighted graph G , constructed from the original data, and an equivalent weighted graph H , induced by the embedded points $\{\mathbf{y}_i\}_{i=1}^n$ in the reduced space. Specifically, the forces acting on the points $\mathbf{y}_i$ and $\mathbf{y}_j$ are derived from the gradient of the edge-wise cross-entropy between the two graphs. The goal is to position the embedded points such that the graph H most closely approximates G in terms of edge existence probabilities. In other words, UMAP seeks to make the low-dimensional weighted graph H reproduce the topological relationships encoded in G as faithfully as possible. Since G captures the intrinsic structure of the data manifold, minimizing their divergence ensures that the resulting embedding preserves both local and global topology. The final configuration therefore represents a low-dimensional projection that accurately reflects the overall geometric organization of the dataset.

Hyperparameters The UMAP algorithm uses two main hyperparameters:

1. k : the number of neighbors to consider when approximating the local metric
2. min-dist: the desired minimum distance between neighboring points in the embedding space

Number of neighbors k The k parameter can be interpreted as the local scale at which the manifold is approximated as locally flat by averaging information from its k -nearest neighbors. In other words, k controls the balance between capturing local structure and representing the global structure of the data. Small values of k allow for a more precise description of local details and small variations in the manifold, but risk fragmenting the space into many small, connected components, thus losing an overview of the global structure.

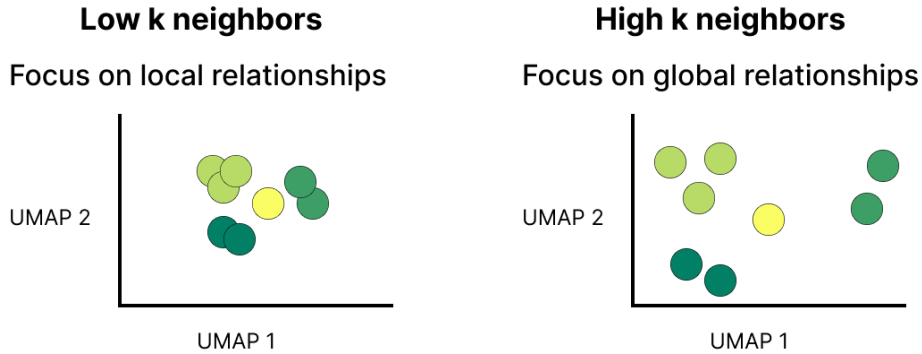


Figure 6: Local versus global structure preservation through k neighbors parameter

Conversely, large values of k favour a more coherent and continuous representation of the global structure, but tend to flatten or average out local subtleties. In such cases, features that occur at scales smaller than that defined by k -neighbors are lost, while large-scale structures that cannot be reconstructed by combining local sets of this size may be poorly represented. When constructing the graph in high-dimensional space, UMAP assigns each point a different radius to determine which other points are considered connected. The size of this radius depends on the point's distance from its k th nearest neighbor, so that each point is connected to approximately the

same number of neighbors. UMAP then fuzzes the graph, progressively reducing the probability of connection as the distance between points increases. In this context, a low value of k neighbors better preserves local relationships: very similar points tend to form compact clusters. In contrast, a high value of neighbours k leads to greater global connectivity and clusters are less tightly packed. In this study, the k parameter is set to the default value of 15.

Minimum distance Unlike the number of neighbors, the $min - dist$ parameter directly affects the final embedding result, as it controls the construction of the simplicial fuzzy set from the low-dimensional representation. In practice, $min - dist$ plays a role analogous to the nearest neighbor distance, used to ensure local connectivity. It determines how close the points can be arranged in the reduced space.

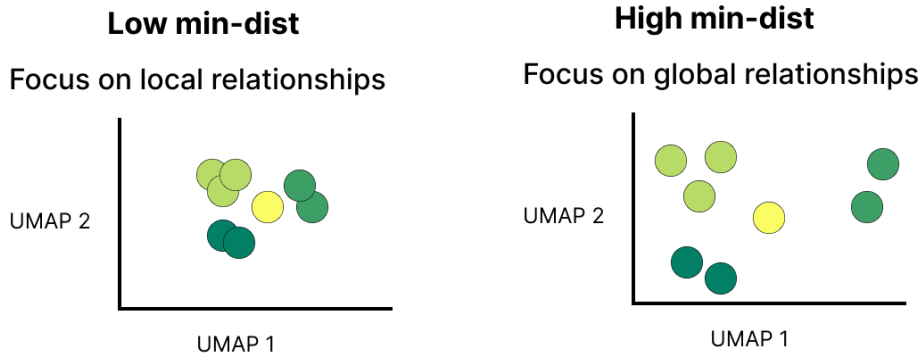


Figure 7: Local versus global structure preservation through $min - dist$ parameter

Low values of $min - dist$ allow the points to aggregate into denser regions, more faithfully reproducing the structure of the original manifold. However, this can lead to more crowded visualizations and potential overlapping problems. Conversely, high values of $min - dist$ force the algorithm to distribute the points more widely in the embedding space, improving visual readability and highlighting the global relationships between points. In short, $min - dist$ can be considered an essentially aesthetic parameter, which governs the appearance of the final projection. Its influence is relevant as it controls the balance between the fidelity of the local structure and the clarity of the global structure. In our work minimum distance is set to default value of 0.1.

Algorithm Summary

Algorithm 3: Uniform Manifold Approximation and Projection (UMAP)

Data: Dataset $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ with $\mathbf{x}_i \in \mathbb{R}^p$,
Hyperparameters: number of neighbors k , minimum distance min_dist , target dimension d ,
Optimization parameters: number of epochs T , learning rate η , negative sample rate n_{neg}

Result: Low-dimensional data $Y^{(T)} = \{\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n\}$ with $\mathbf{y}_i \in \mathbb{R}^d$

begin

 Construct k-nearest neighbor graph for X

for each point $\mathbf{x}_i$ **do**

 Compute local metric ρ_i (distance to nearest neighbor)

 Compute σ_i such that $\sum_{j \in N(i)} \exp\left(-\frac{\max(0, d(\mathbf{x}_i, \mathbf{x}_j) - \rho_i)}{\sigma_i}\right) = \log_2(k)$

end

 Compute high-dimensional graph weights:

$$w_{ij} = \exp\left(-\frac{\max(0, d(\mathbf{x}_i, \mathbf{x}_j) - \rho_i)}{\sigma_i}\right)$$

 Symmetrize: $p_{ij} = w_{ij} + w_{ji} - w_{ij} \cdot w_{ji}$

 Initialize $Y^{(0)} = \{\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n\}$ randomly or with spectral embedding

for $t = 1$ **to** T **do**

for each edge (i, j) with weight $p_{ij} > 0$ **do**

 Sample n_{neg} negative samples $\{k_1, k_2, \dots, k_{n_{neg}}\}$

 Compute attractive force for (i, j) :

$$F_{attr} = -2ab \cdot \frac{\|\mathbf{y}_i - \mathbf{y}_j\|_2^{2(b-1)}}{1 + \|\mathbf{y}_i - \mathbf{y}_j\|_2^2} \cdot p_{ij} \cdot (\mathbf{y}_i - \mathbf{y}_j)$$

 Compute repulsive forces for negative samples:

$$F_{rep} = \sum_{k \in \text{negative samples}} \frac{2b}{(\epsilon + \|\mathbf{y}_i - \mathbf{y}_k\|_2^2) \cdot (1 + a\|\mathbf{y}_i - \mathbf{y}_k\|_2^{2b})} \cdot (\mathbf{y}_i - \mathbf{y}_k)$$

 Update embeddings: $Y^{(t)} = Y^{(t-1)} + \eta \cdot (F_{attr} + F_{rep})$

end

end

end

Weaknesses and conclusions Although UMAP is an extremely powerful algorithm for both visualization and dimensionality reduction it has inevitable tradeoffs:

- Limited interpretability of results: like other nonlinear dimensionality reduction techniques, such as t-SNE, UMAP lacks the clear interpretability of linear techniques such as Principal Component Analysis. The dimensions of the embedding space have no specific meaning and because UMAP is based on distances between observations rather than on the original features, it does not provide equivalents to the 'factor weights' found in linear techniques.
- Assumption of a manifold structure: UMAP relies on the assumption that the data are distributed across a lower-dimensional manifold. With small or noisy datasets, the algorithm may detect spurious structures. Robustness increases with increasing data, but caution is needed with small samples.
- Limited preservation of global structure: because UMAP prioritizes the accurate representation of local relationships, the global structure may be less faithful. While UMAP maintains better global consistency than techniques like t-SNE, it is not ideal when the goal is to preserve distances at all scales.
- Data with variable density: UMAP tends to even out the distribution of points, assuming a homogeneous density in space. If the original distances accurately reflect the intrinsic structure of the dataset, and this has very dense regions alongside sparse areas, UMAP may distort these proportions.
- Computational approximations: to improve efficiency, UMAP introduces several approximations that may affect results on small datasets. In particular, the use of approximate nearest-neighbor searches and negative sampling during optimization can result in suboptimal embeddings.

Overall, UMAP provides a strong balance between scalability and the ability to capture meaningful structures in high-dimensional data. Nevertheless, since it relies on manifold assumptions, local neighborhood modelling, and approximate optimization procedures, its outputs may require careful tuning and interpretation, especially when dealing with noisy, small, or highly heterogeneous datasets.

Chapter 3

Clustering Algorithms

This chapter introduces the theoretical background behind the clustering algorithms used in combination with pre-processing methods.

3.1 Introduction

Context Clustering is a family of unsupervised learning techniques aimed at discovering hidden structures within a dataset. Its goal is to group observations into different clusters such that elements within the same group share strong similarities, while those in different groups are clearly dissimilar. Despite its conceptual simplicity, clustering includes a wide range of approaches, each suited to different data characteristics and analytical goals. Nevertheless, selecting the most suitable algorithm remains a key challenge, since different methods may capture different data structures.

Main applications Clustering algorithms are widely used in numerous fields to extract patterns and knowledge from large amounts of heterogeneous data. By revealing natural groupings and underlying structures, they provide a fundamental tool for exploratory data analysis and support a wide range of applications in areas such as biology, marketing, image processing, and social sciences. For instance, clustering is employed for automatic spam detection, recommendation systems, customer segmentation for targeted marketing strategies and the organization of visual content based on perceptual similarities.

Challenges Although clustering is a powerful tool, its application to large datasets presents significant challenges. In many cases, algorithms compute the similarity between all pairs of observations, with a complexity that increases quadratically with the number of examples, indicated as $O(n^2)$. This makes these methods impractical on datasets containing millions of instances. In this scenario, the dimensionality reduction techniques presented in the previous chapter play a crucial role: they allow the relevant information to be considered into a lower-dimensional space, making the application of clustering algorithms more efficient and scalable. [5]

Notation Throughout this chapter, we adopt the following notation:

- n : number of observations,
- p : number of variables,
- K : number of clusters,
- $k \in \{1, \dots, K\}$: cluster index

3.2 Types of Clustering Algorithms

As stated in the Introduction, one of the main goals of this work is to analyse how different clustering algorithms perform under distinct experimental conditions, using datasets that differ in number of observations, number of variables, number of clusters, within-cluster correlation and noise level. This comparison will help highlight the strengths, limitations, and most suitable application contexts for each class of methods. In particular, we consider five clustering algorithms: k-means, PAM (Partitioning Around Medoids), GMM (Gaussian Mixture Model), Birch, and Spectral Clustering. Each of them belongs to a different macro-category of clustering techniques, characterized by distinct theoretical assumptions and operational principles. In this section, we provide a general overview of these algorithms in order to clarify their structural differences, thus enabling a more effective analysis of their behaviour in the following sections.

Partition-based clustering: Partition-based clustering algorithms are built on the idea of dividing data into non-overlapping partitions, identifying a representative central point, called a centroid, for each cluster. The goal is to assign each observation to the cluster whose centroid is closest, according to a predefined distance measure. These algorithms generally have low computational complexity and high efficiency, making them particularly suitable for large datasets. However, they have some limitations: they are not suitable for non-convex data, are sensitive to the presence of outliers, and require a priori definition of the number of clusters. This category includes k-means and PAM. Although they belong to the same family, it is important to make a distinction. k-means creates clusters from randomly initialized centroids, which are then iteratively updated as the average of the points assigned to each cluster. PAM, on the other hand, uses medoids, which are real points from the dataset that represent the centre of the cluster. The use of medoids makes PAM more robust to the presence of outliers, although at the cost of greater computational complexity.

Model-based clustering: Model-based algorithms assume that the data is generated by a combination of underlying probabilistic models and attempt to identify the parameters that best describe this distribution. The goal is to estimate the most appropriate probabilistic model for each cluster, often using maximum likelihood methods. Among these, Gaussian Mixture Models are a prime example: each cluster is described by a Gaussian distribution characterized by a mean and a covariance matrix, and the algorithm estimates the probability of each point belonging to each cluster. These methods are particularly flexible, as they allow for modelling clusters of varying shapes and sizes, but they also present some critical issues: high computational complexity, dependence on the choice of initial parameters, and sensitivity to statistical assumptions that do not always reflect the true distribution of the data.

Compression-based clustering: Compression-based clustering algorithms are based on the idea of reducing the size and complexity of the data before proceeding to

the clustering phase. In other words, the data is represented in a more compact form through a structure that preserves the main geometric and statistical characteristics of the dataset. This compressed representation allows for more efficient clustering, maintaining good quality results even with very large or high-dimensional datasets. A representative example is the Birch (Balanced Iterative Reducing and Clustering using Hierarchies) algorithm. Birch builds a tree structure, called a CF-Tree (Clustering Feature Tree), in which each node stores a set of synthetic statistical information about the data. This structure is updated steadily, allowing the data to be processed in a single step and dynamically adapted to new points. After the compression phase, a traditional clustering algorithm like k-means is applied to the centroids stored in the CF-Tree. Compression-based algorithms therefore offer a particularly efficient solution for clustering large amounts of data, with an acceptable trade-off between accuracy and speed.

Graph-based clustering: Graph-based clustering algorithms represent data as a graph, where each observation constitutes a node and connections between nodes are established based on a similarity measure. The central idea is that the cluster structure can be revealed by analysing the graph’s topological properties, such as connectivity or relationships between groups of nodes. A very popular approach within this category is Spectral Clustering, which uses tools from linear algebra to identify a projection of the data into a space in which the cluster structures are more evident. Specifically, a similarity matrix between points is computed, from which the Laplacian ¹ of the graph is derived. The spectral decomposition of this matrix provides the eigenvectors needed to reduce dimensionality and complexity. Subsequently, a traditional clustering algorithm like k-means is applied to the transformed space. These methods are particularly effective at identifying clusters with nonlinear shapes and complex structures.

¹The Laplacian of a graph is defined as $L = D - W$, where W is the similarity matrix between nodes and D is the diagonal degree matrix, with $D_{ii} = \sum_j W_{ij}$. The Laplacian encodes the structure of the graph and is fundamental because its eigenvectors describe the graph’s connectivity.

However, they present some critical issues: building the similarity matrix can be computationally expensive on very large datasets, and the quality of the result depends on the choice of similarity function and neighborhood parameter used to construct the graph.[16, 17]

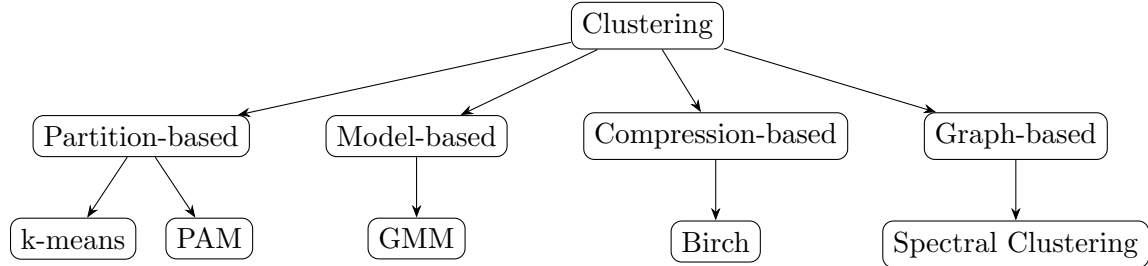


Figure 8: Hierarchical overview of clustering algorithms

3.3 Selecting the Optimal Number of Clusters

The main challenge in hard clustering ² lies mainly in choosing the optimal number of clusters, as it directly impacts the quality of the information extracted and the practical utility of the identified clusters. To address this issue, two main methods are used, called the Elbow method and the Silhouette score method.

Elbow method This involves running the clustering algorithm for a range of cluster numbers and graphically representing the sum of squared distances to the cluster centroids as a function of the number of clusters.

$$W = \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \mu_k\|^2 \quad (3.3.1)$$

where:

- K represents the total number of clusters;

²Hard clustering refers to a clustering paradigm in which each data point is assigned exclusively to one and only one cluster. Formally, it corresponds to a deterministic partition of the dataset, defined by a mapping $c : X \rightarrow \{1, \dots, K\}$ that assigns every observation to exactly one cluster with no overlap among clusters. This approach contrasts with soft or fuzzy clustering, where points may belong to multiple clusters with varying degrees of membership. [18]

- C_k denotes the set of all points belonging to cluster k ;
- $\mathbf{x}$ is a point within cluster C_k ;
- μ_k is the centroid of cluster k ;
- $\|\mathbf{x} - \mu_k\|^2$ calculates the squared Euclidean distance between a point $\mathbf{x}$ and the centroid μ_k , which quantifies how far the point is from the centroid.

W measures how tight the clusters are, with lower values indicating that points are closer to their respective centroids. The point at which the decrease in inertia visibly slows, forming a sort of elbow in the graph, is often considered the optimal number of clusters. The graphical concept is the same as that presented for the best choice of the number of principal components in the case of PCA in Figure 2. However, the elbow point is not always clearly recognizable, making this criterion partly subjective.

Silhouette score This measures how similar each point is to the elements of its own cluster compared to the elements of other clusters. The silhouette score $s(i)$ ranges from -1 to 1 , where a high value indicates that point i is well matched to its own cluster and poorly matched to neighboring clusters, a value near 0 suggests that i lies between two clusters while a negative value indicates that i may have been assigned to the wrong cluster. The formula for the Silhouette Score for a single data point is:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (3.3.2)$$

where:

- $s(i)$ is the silhouette score for a single data point i ;
- $a(i)$ is the average distance of data point i to all other points in the same cluster, measuring how well i fits within its own cluster;
- $b(i)$ is the smallest average distance of i to all points in any other cluster, minimized over all other clusters, representing how close i is to its nearest neighboring cluster.

For each value of K the average silhouette score is computed across all data points. The value of K that maximizes this average silhouette score is considered the optimal number of clusters. This method is also widely used to evaluate clustering quality.

3.4 Algorithms Presentation

We now present in detail the algorithms that will be used in our analyses, with the aim of highlighting their strengths and limitations. This section is particularly important, as it will provide a fundamental reference when we later analytically evaluate the performance of the different clustering approaches.

3.4.1 k-means Algorithm

k-means is a partition-based clustering algorithm that divides a dataset into K groups by assigning each observation to the closest centroid. The main idea is to minimize the within-cluster sum of squares (WCSS), defined as the sum of squared Euclidean distances between each point and the centroid of its assigned cluster. This minimization is carried out through an iterative procedure consisting of two steps: assigning points to the current centroids and updating the centroids based on the assigned points, until convergence is reached. k-means is widely used in various applications due to its simplicity and computational efficiency. However, it is sensitive to the initial choice of centroids and may converge to a local minimum rather than the global optimum, making careful initialization important.

Initialization The first step is to choose K initial centroids $(\mu_1, \mu_2, \dots, \mu_K)$. The optimal number of clusters can be determined using methods such as the elbow method or the silhouette score. Once K is chosen, the centroids can be initialized randomly or, more efficiently, using the *k-means++* method, which improves the probability of converging to a better solution. The procedure consists of selecting the first centroid randomly and then choosing subsequent centroids with probability proportional to the squared distance from the closest centroid already selected.

Assigning points to the closest centroid Each point $x_i \in \mathbb{R}^p$, for $i = 1, \dots, n$, is assigned to the cluster C_k whose centroid μ_k is closest in terms of Euclidean distance:

$$d(x_i, \mu_k) = \sqrt{\sum_{j=1}^p (x_{ij} - \mu_{kj})^2}. \quad (3.4.1)$$

The overall within-cluster dispersion is computed as the sum of squared Euclidean distances:

$$W(C) = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2. \quad (3.4.2)$$

Thus, the criterion is minimized by assigning the n observations to the K clusters in such a way that the dissimilarity of the observations from their cluster centroids is minimized. The objective of k-means can therefore be written as:

$$C^*, \{\mu_k^*\} = \arg \min_{C, \{\mu_k\}} W(C) = \arg \min_{C, \{\mu_k\}} \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2. \quad (3.4.3)$$

Updating centroids After assigning points to clusters, each centroid is updated as the mean of the points assigned to that cluster:

$$\mu_k^{(t+1)} = \frac{1}{|C_k^{(t)}|} \sum_{i \in C_k^{(t)}} x_i. \quad (3.4.4)$$

This ensures that, given the current cluster assignments, the updated centroid minimizes the within-cluster sum of squared distances.

Iteration until convergence The assignment and update steps are repeated until convergence occurs. This can be verified by checking that the variation of centroids between two successive iterations is less than a threshold ε :

$$\max_{1 \leq k \leq K} \|\mu_k^{(t)} - \mu_k^{(t-1)}\| < \varepsilon. \quad (3.4.5)$$

Each iteration decreases the value of the objective function, thereby guaranteeing

convergence to a local minimum. For this reason, it is common to run k-means multiple times with different random initializations and select the solution corresponding to the lowest final value of $W(C)$, reducing the sensitivity of the algorithm to initialization.

Limitations Despite its high computational efficiency, k-means has several limitations. A first critical issue is the implicit assumption that clusters are convex, homogeneous, and approximately spherical. When the dataset contains nonlinear structures, irregular shapes, or clusters with varying densities, the algorithm tends to produce suboptimal partitions. Another significant limitation concerns its sensitivity to outliers: even a small number of anomalous points can substantially shift the centroids, leading to distorted or misleading results.

k-means as a starting point Although k-means remains an effective and conceptually simple method, these limitations make it unsuitable for datasets characterized by complex structures or high levels of noise. In the following sections, we will therefore discuss clustering algorithms that address some of these issues and can generally improve upon the performance of k-means. Among these, we will examine PAM, a variant of k-means that uses actual data points (medoids) instead of the mean to represent clusters, making it more robust to outliers. We will also consider methods such as Birch and Spectral Clustering, which include strategies to better capture structural information from the data before clustering, thereby mitigating several weaknesses of k-means and enhancing its effectiveness in challenging scenarios. [17, 19, 20]

3.4.2 PAM Algorithm

The PAM or k-medoids algorithm is a clustering technique closely related to k-means. While k-means represents each cluster by its centroid, so the average of all points within the cluster, k-medoids instead selects one of the actual data points, known as the medoid, as the cluster center. This makes PAM a more robust alternative to k-means, being less sensitive to noise and outliers and creating more stable and interpretable clusters.

Algorithm overview Objects temporarily defined as medoids are inserted into a set S of selected objects. If O is the set of all objects, the set $U = O \setminus S$ contains the unselected objects. The goal of the algorithm is to minimize the average dissimilarity of the objects from their closest selected object, i.e. the medoid. Equivalently, we can minimize the sum of the dissimilarities between each object and the closest selected object. The algorithm has two phases:

1. Build: in this first phase, an initial set of K objects is selected to become the medoids and initialize set S
2. Swap: in the second phase, an attempt is made to improve the clustering quality by swapping selected objects with unselected objects.

For each object j , we maintain two numbers: D_j , that is the dissimilarity between j and the closest object in S , and E_j , the dissimilarity between j and the second-closest object in S . These numbers must be updated whenever the sets S and U change. Note that $D_j \leq E_j$ and that $j \in S$ if and only if $D_j = 0$.

Build phase The Build phase involves the following steps:

1. Initialize S by adding to it an object s for which the sum of the distances to all other objects is minimal. We are therefore choosing the most central point of the distribution, that is, the one that is on average closest to the others.
2. Consider an object $i \in U$ as a candidate for inclusion in the set S of selected objects. We now have to evaluate if the clustering would improve if we added i to S as a new medoid.
3. For an object $j \in U \setminus \{i\}$, compute:

$$D_j = \min_{s \in S} d(j, s), \quad (3.4.6)$$

as the dissimilarity between j and its closest selected medoid s picked in phase

1. This tells us how well the remaining points are covered by the current set S .

4. If $D_j > d(j, i)$, object j will contribute to the decision to select object i as a better medoid. This means that j would be closer to i than to any medoid already chosen, so j suggests that i should be selected. We define:

$$C_{ji} = \max\{D_j - d(j, i), 0\}. \quad (3.4.7)$$

as the measure of how much j 's coverage would improve if i were added to S .

5. Calculate the total gain obtained by adding i to S as:

$$g_i = \sum_{j \in U} C_{ji}. \quad (3.4.8)$$

6. Choose the object i that maximizes g_i and update $S := S \cup \{i\}$ and $U := U \setminus \{i\}$.

These steps are repeated until K objects are selected.

Swap phase The Swap phase attempts to optimize the set of selected objects and therefore the quality of the clustering. This is done by considering all pairs $(i, h) \in S \times U$ and calculating the effect T_{ih} on the sum of the dissimilarities between the objects and the closest selected object, caused by swapping i and h . If T_{ih} will be negative the swap will improve the clustering, while having a positive T_{ih} will lead us to reject the swap. The calculation of T_{ih} involves determining the contribution K_{jih} of each object $j \in U \setminus \{h\}$ to the swapping of i and h . We have that either $d(j, i) > D_j$ or $d(j, i) = D_j$, where D_j is the distance of j to its assigned medoid.

1. K_{jih} is computed by considering the following cases:

(a) If $d(j, i) > D_j$, meaning that the actual medoid of j is not i , we have two subcases:

- i. If $d(j, h) \geq D_j$, then $K_{jih} = 0$;
- ii. If $d(j, h) < D_j$, then $K_{jih} = d(j, h) - D_j$, resulting in a negative contribution to T_{ih} .

In both subcases, we can write:

$$K_{jih} = \min\{d(j, h) - D_j, 0\}. \quad (3.4.9)$$

(b) If $d(j, i) = D_j$, meaning that the actual medoid of j is i , we have two subcases:

- i. If $d(j, h) < E_j$, where E_j is the dissimilarity between j and the second closest selected object, then $K_{jih} = d(j, h) - D_j$;
- ii. If $d(j, h) \geq E_j$, then $K_{jih} = E_j - D_j$.

In both subcases, we can summarize:

$$K_{jih} = \min\{d(j, h), E_j\} - D_j. \quad (3.4.10)$$

2. Compute the total result of the swap as:

$$T_{ih} = \sum_{j \in U \setminus \{h\}} K_{jih}. \quad (3.4.11)$$

3. Select the pair $(i, h) \in S \times U$ that minimizes T_{ih} .

4. If $T_{ih} < 0$, perform the swap, update D_j and E_j for every object j , and return to Step 1. If $\min(T_{ih}) > 0$, the objective value cannot be further decreased and the algorithm ends. [21]

3.4.3 Gaussian Mixture Model Algorithm

Algorithms like k-means show satisfactory performance in relatively simple scenarios and continue to be fundamental tools for preliminary data analysis. However, they exhibit significant limitations when dealing with data characterized by complex structures or markedly multimodal distributions, often generating suboptimal clustering results. To address these difficulties, the Gaussian Mixture Model (GMM) was introduced, a probabilistic model capable of representing data as a combination of multiple Gaussian distributions.

Probabilistic density modelling A Gaussian Mixture Model (GMM) is a probability density function represented as a weighted linear combination of multiple Gaussian distributions³. Formally, it can be written as:

$$p(\mathbf{x} \mid \Psi) = \sum_{k=1}^K w_k g(\mathbf{x} \mid \mu_k, \Sigma_k) \quad (3.4.13)$$

where $\mathbf{x} \in \mathbb{R}^p$ is a continuous vector of dimension p , w_k are the weights of the mixture model that represent the probability that a randomly selected data point was generated by the Gaussian component under consideration. In this framework, each Gaussian component $g(\mathbf{x} \mid \mu_k, \Sigma_k)$ can be interpreted as a cluster, whose shape and dispersion are defined by its mean and covariance matrix, while the corresponding weight w_k reflects the prior probability and relative importance of that cluster within the overall data distribution. The set of model parameters is denoted by:

$$\Psi = \{w_k, \mu_k, \Sigma_k\}_{k=1}^K \quad (3.4.14)$$

To implement a GMM, it is necessary to establish the number of Gaussian components to use and apply an estimation algorithm, usually Expectation-Maximization (EM), specifying the desired shape of the covariance matrices.

EM algorithm The EM algorithm is widely used to iteratively calculate the maximum likelihood estimate (MLE)⁴ of the parameters of a probabilistic model. In the

³The Gaussian, or normal, distribution is a continuous probability distribution defined by mean μ and variance σ^2 , with density

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad (3.4.12)$$

which describes a random variable symmetric about the mean and completely determined by its first two moments

⁴The maximum likelihood estimator (MLE), denoted by $\hat{\Psi}$, is the value of the parameter vector Ψ that maximizes the likelihood function $L(\Psi)$: [22]

$$\hat{\Psi} = \arg \max_{\Psi} L(\Psi). \quad (3.4.15)$$

context of GMMs, it is used to determine the Ψ parameters from the observation of a dataset X . The likelihood of the model is defined as:

$$L(\Psi) = \prod_{i=1}^n p(\mathbf{x}_i \mid \Psi) \quad (3.4.16)$$

and the maximum likelihood is obtained by solving:

$$\frac{\partial L(\Psi)}{\partial \Psi} = 0 \quad (3.4.17)$$

which in practice takes the most convenient form of log-likelihood:

$$\frac{\partial \log L(\Psi)}{\partial \Psi} = 0 \quad (3.4.18)$$

The EM algorithm iteratively calculates the optimal parameters Ψ^* using the following steps:

1. Initialization: initial selection of the $\Psi^{(0)}$ parameters. A common practice is to initialize component means using centroids obtained through k-means clustering.
2. E-Step: compute the responsibilities, i.e., the conditional probabilities that each data point belongs to each Gaussian component, based on the current parameter estimates.
3. M-Step: updates the parameters (means, covariances, and mixture weights) by maximizing the expected log-likelihood, using the responsibilities computed in the E-step.
4. Convergence check: evaluate the log-likelihood at each iteration. Convergence is achieved when the change in log-likelihood becomes less than a fixed threshold (typically 10^{-3}) or when a maximum number of iterations is exceeded (usually 100).

These steps are repeated until a local maximum of the log-likelihood function is

reached. Since the EM tends to converge to a local optimum, the choice of initial conditions has a significant impact on the performance of the model. A common practice is to use the k-means centroids as initial values for the means μ_k . Another popular strategy is to run the algorithm multiple times with different random initializations, finally selecting the solution with the highest likelihood. [23]

3.4.4 Birch Algorithm

Traditional clustering algorithms, such as k-means, are often inefficient for large datasets, especially when computational resources are limited. Birch (Balanced Iterative Reducing and Clustering using Hierarchies) addresses this issue by first creating a compact summary of the data that preserves essential information. The clustering process is then applied to this reduced representation rather than to the full dataset. Before detailing the Birch algorithm, two key concepts need to be introduced: the Clustering Feature (CF) and the CF-Tree.

Clustering Feature Birch summarizes large datasets into smaller, denser regions, called Clustering Features. Formally, a CF is defined as an ordered triplet:

$$CF = (N, LS, SS) \tag{3.4.19}$$

where:

- N is the number of points belonging to the cluster;
- LS is the linear sum of the points in the cluster;
- SS is the sum of the squares of the points.

Each CF can represent a single cluster or a set of other CFs, thus allowing a hierarchical representation of the data. The information contained in a CF allows statistics such as the mean and variance of the cluster to be easily calculated. For example, the mean can be computed as:

$$\mu = \frac{LS}{N} \quad (3.4.20)$$

while the variance, computed component-wise, is given by:

$$\sigma^2 = \frac{SS}{N} - \left(\frac{LS}{N}\right)^2 \quad (3.4.21)$$

CF-Tree The CF-Tree represents the hierarchical and compact structure used by Birch to summarize the dataset. It is a balanced tree in which each leaf node contains sub-clusters. Each internal node of the tree contains:

- a set of references to its child nodes;
- a CF entry representing the sum of the Clustering Features of its children.

In this way, each internal node summarizes the statistical information of the sub-clusters located at lower levels of the tree, allowing for a hierarchical and compact representation of the entire dataset. Each leaf node has a maximum number of CF entries it can contain, determined by a threshold parameter T . If a node exceeds this threshold, it is split, ensuring that the tree structure remains balanced and efficient.[24]

Algorithm presentation The process of inserting a new element into the CF-Tree is described as follow:

1. Identification of the appropriate leaf: the algorithm begins with an initial threshold value, analyses the data, and progressively inserts the points into the tree. Starting from the root, we recursively descend along the CF-Tree, selecting, at each level, the closest child node according to the chosen distance metric, typically the Euclidean distance. If available memory runs out before the data scan is complete, the threshold is increased and a new, more compact CF-Tree is rebuilt. A good initial threshold can significantly reduce the number of reconstructions required. However, if the threshold is too high, the CF-Tree will be less detailed than the available memory allows.

2. Leaf modification: once a leaf node is reached, the closest CF element is identified and checked to see if it can absorb the new CF without violating the threshold condition. If it can, the CF entry is updated; otherwise, a new CF entry is added to the leaf. If the leaf does not have enough space for the new entry, the node is split. This split is done by choosing the two elements furthest from each other as seeds and redistributing the remaining entries based on their distance. Since some clustering algorithms work best with a certain number of objects within a certain range, very dense sub-clusters can be grouped into larger clusters, resulting in an overall more compact CF-Tree.
3. Modifying the path to the leaf: since each non-leaf node is itself a Clustering Feature representing the sum of the CFs of its children, inserting a CF entry into a leaf updates the CF information of all nodes along the path to the root. In the event of a split, a new entry is inserted into the parent node, pointing to the newly created node. If the parent node also runs out of space, it is also split, propagating the split upward to the root. If the root is split, the height of the tree increases by one level.
4. Refinement and merge phase: node splits are determined by the size of the memory pages, not the clustering properties of the data. In the presence of data with a highly unbalanced distribution, this can compromise the quality of the clustering and reduce space efficiency. A simple additional merging step often improves these aspects, increasing space utilization and postponing any future splits. It is important to note that, up to this point, although the tree may have been reconstructed multiple times, the original data is scanned only once. The final step (step 4) involves additional passes through the data to correct any inaccuracies due to the fact that the clustering algorithm was applied to an approximate summary of the data. This step also allows for the identification and possible elimination of outliers. [25]

Once the CF-Tree has been constructed and refined, the leaf nodes contain a compact representation of the dataset in the form of Clustering Features. These sub-clusters can then be used as input for a traditional clustering algorithm, such as k-means, to obtain the final clusters. This approach allows Birch to efficiently handle very large datasets by reducing the number of data points involved in the final clustering step, while preserving the statistical properties and structure of the original data. Additionally, this final step enables the identification of outliers and further refinement of the cluster boundaries.

3.4.5 Spectral Clustering Algorithm

Spectral clustering is a graph-based clustering method that exploits connectivity relationships between data points to identify meaningful groupings. The basic idea is to represent the dataset as a graph, where data points correspond to nodes and their similarities are modelled by weighted edges. From this representation, the algorithm uses the eigenvalues and eigenvectors of the graph Laplacian matrix to project the data into a lower-dimensional embedding space, where group structures are more easily identifiable. Once this embedding is obtained, a traditional clustering method such as k-means can be applied to partition the data into homogeneous sets.

Data representation using similarity graphs Let us consider points $x_1, \dots, x_n$ and a similarity measure $s_{ij} \geq 0$ between pairs of points (x_i, x_j) . A natural representation consists of constructing a similarity graph:

$$G = (V, E), \tag{3.4.22}$$

where each vertex $v_i \in V$ represents a data point x_i , and each edge (v_i, v_j) is weighted by $w_{ij} = s_{ij}$. Although several approaches exist for constructing the graph, one of the most widely used in practice is the m -Nearest neighbors (m -NN) method. This is because it produces a sparse graph that is computationally efficient and captures the local structure of the data, making it a standard choice in many spectral

clustering implementations.⁵ [26]. Once formed, the graph is represented by the weighted adjacency matrix

$$W = (w_{ij})_{i,j=1,\dots,n}, \quad (3.4.23)$$

from which the degree matrix D is derived. It is defined as a diagonal matrix with entries

$$d_i = \sum_{j=1}^n w_{ij}. \quad (3.4.24)$$

Graph Laplacians The mathematical tool underlying spectral clustering is the Laplacian matrix, which allows local properties of the graph to be expressed through linear operators on vector spaces. The unnormalized Laplacian is defined as:

$$L = D - W. \quad (3.4.25)$$

The Laplacian matrix L possesses several fundamental properties that make it particularly suitable for clustering. First, L is symmetric and positive semidefinite, which guarantees real eigenvalues and orthogonal eigenvectors. Moreover, if the graph is connected, its smallest eigenvalue is equal to zero and the corresponding eigenvector is the constant vector of ones. Finally, all eigenvalues of L are real and nonnegative, reflecting the intrinsic structure of the graph and enabling the use of spectral information to reveal cluster boundaries.

Two normalized versions of the Laplacian are also widely used:

$$L_{\text{sym}} = D^{-1/2} L D^{-1/2} = I - D^{-1/2} W D^{-1/2}, \quad (3.4.26)$$

$$L_{\text{rw}} = D^{-1} L = I - D^{-1} W, \quad (3.4.27)$$

⁵ m -NN links each data point to its m nearest neighbors based on a specified distance metric, such as the Euclidean distance. Consequently, a graph is created where the edges connect each node to its m closest points.

known respectively as the Symmetric Laplacian and the Random-Walk Laplacian. In particular, L_{sym} is symmetric positive semidefinite and has non-negative eigenvalues. A key aspect of spectral clustering is that the eigenvectors associated with the smallest eigenvalues encode the structure of the clusters present in the graph: ideally, they are approximately constant within each cluster.

Normalized spectral clustering algorithm One of the most widely used formulations is based on the matrix L_{sym} and can be summarized as follows:

1. Construct the similarity graph and the weighted adjacency matrix W .
2. Compute the normalized Laplacian L_{sym} .
3. Compute the first K eigenvectors $u_1, \dots, u_K$ associated with the smallest eigenvalues.
4. Form the matrix $U \in \mathbb{R}^{n \times K}$ containing the eigenvectors as columns.
5. Normalize each row of U to unit norm, obtaining the matrix T .
6. Interpret the i -th row t_i as a new representation of the point x_i .
7. Apply k-means to the points t_i to obtain the final clusters.

The core of the algorithm lies in the spectral transformation of the data: the eigenvectors of the Laplacian provide an embedding in which the cluster structure is often significantly more evident than in the original space.

3.5 Concluding Remarks on Clustering Methods

In conclusion, different clustering algorithms have complementary strengths and limitations (Table 1). Partition-based methods such as k-means and PAM are efficient and suitable for large datasets, but require a fixed number of clusters and are sensitive to outliers. Model-based methods such as Gaussian Mixture Models are more flexible

and allow for estimating membership probabilities, but they rely on data assumptions and are more complex. Compression-based algorithms such as Birch handle very large datasets well thanks to data synthesis, with possible loss of information. Finally, graph-based methods such as Spectral Clustering identify clusters with complex shapes, but require greater computational cost and a careful choice of parameters.

Category	Advantages	Limitations	Representative Algorithms
Partition-based Clustering	Low computational complexity and high efficiency. Easy to implement and suitable for large datasets.	Sensitive to outliers. Can converge to local optima and requires a priori specification of the number of clusters.	k-means, PAM
Model-based Clustering	Allows modelling of non-spherical clusters and estimating the membership probability of each point. Can flexibly represent clusters of different shapes and sizes.	High computational complexity and dependence on initialization. Results are sensitive to assumptions about the underlying data distribution.	Gaussian Mixture Models (GMM)
Compression-based Clustering	Highly efficient on large datasets. Incrementally summarizes data and reduces dimensionality while preserving main statistical properties.	Possible information loss during data compression; performance depends on the hierarchical structure and threshold parameters used.	Birch
Graph-based Clustering	Effective for detecting clusters with complex, non-linear shapes. Uses similarity graphs and spectral properties to reveal hidden structures.	Construction of the similarity graph can be computationally expensive. Sensitive to the choice of similarity function and neighborhood parameters.	Spectral Clustering

Table 1: Summary of the main categories of clustering algorithms, with advantages, limitations, and representative algorithms.

3.6 Validation Measures

To quantitatively evaluate clustering results, a variety of metrics have been developed, categorized into internal and external measures. Internal metrics evaluate the quality of clusters using only data-driven properties such as compactness and separation; examples include the Silhouette Score, the Davies-Bouldin Index (DBI), and the Calinski-Harabasz Index (CH). External metrics instead require ground-truth labels and quantify how well the clustering matches known classes, with widely used measures such as the Adjusted Rand Index (ARI) and the Normalized Mutual Information (NMI). Overall, internal metrics assess the dataset’s intrinsic structure, while external metrics measure clustering accuracy when reference labels exist.

3.6.1 Internal Evaluation Metrics

Silhouette score The Silhouette Score is one of the most widely used internal metrics for evaluating the quality of clusters generated by unsupervised clustering algorithms. It measures how well clusters are separated and provides an indication of the overall validity of the clustering solution. The Silhouette Score for a single data point is calculated by considering two quantities: the average distance between the point and other points belonging to the same cluster, denoted $a(i)$, and the average distance between the point and the points of the closest cluster, denoted $b(i)$. The Silhouette Score value for a point i is therefore defined as:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (3.6.1)$$

The Silhouette Score assumes ranges from -1 to 1 . Values close to 1 indicate well-defined clusters, where the point is much closer to its own cluster members than to those of neighboring clusters. A value close to 0 suggests that the clusters are overlapping or poorly distinct, while values close to -1 indicate that the point may have been assigned to the wrong cluster. The overall Silhouette Score of the clustering solution is obtained by averaging the Silhouette scores of all points in the dataset.

Davies-Bouldin Index The Davies-Bouldin Index (DBI) is a metric used to evaluate the quality of a clustering solution. It simultaneously measures the compactness of clusters and their separation from each other, with the aim of identifying configurations in which clusters are clearly distinct from each other. The index formula is as follows:

$$DBI = \frac{1}{K} \sum_{i=1}^K \max_{j \neq i} \left(\frac{\text{avg radius}_i + \text{avg radius}_j}{\text{distance}(c_i, c_j)} \right) \quad (3.6.2)$$

where:

- K is the number of clusters,
- c_i and c_j are the centroids of clusters i and j ,
- avg radius_i and avg radius_j represent the average distances from the centroid to the points belonging to clusters i and j ,
- $\text{distance}(c_i, c_j)$ is the distance between the centroids of the two clusters.

The Davies-Bouldin Index is based on Euclidean distance and therefore assumes that clusters have a roughly spherical shape; therefore, it may not be suitable for cluster structures with irregular shapes. Furthermore, the formula implicitly assumes that clusters are similar in size, making it less effective when there are large imbalances in cluster size. Lower DBI values indicate better clustering performance, meaning that clusters are more compact and well-separated while higher DBI values suggest that clusters are less distinct or more dispersed.

Calinski-Harabasz Index The Calinski-Harabasz Index (CH), also known as the Variance Ratio Criterion, is an internal metric used to evaluate the quality of a clustering solution based on the ratio of the dispersion between clusters to the dispersion within clusters. The index is defined as:

$$CH(K) = \frac{B(K)(n - K)}{W(K)(K - 1)} \quad (3.6.3)$$

where K represents the number of clusters and n the total number of samples. $B(K)$ indicates the inter-cluster dispersion, while $W(K)$ represents the intra-cluster dispersion. They are computed respectively as:

$$B(K) = \sum_{k=1}^K a_k \|\bar{x}_k - \bar{x}\|^2 \quad W(K) = \sum_{k=1}^K \sum_{C(j)=k} \|x_j - \bar{x}_k\|^2 \quad (3.6.4)$$

where $a_k = |C_k|$ denotes the number of points belonging to cluster k . A high value of $B(K)$ indicates good separation between clusters, while a low value of $W(K)$ suggests high internal compactness. Consequently, a higher value of the CH Index corresponds to more effective clustering, characterized by dense and clearly separated clusters. The index is particularly useful in selecting the optimal number of clusters, as it tends to favour partitions that maximize the distance between centroids and minimize internal variance [27, 28]

3.6.2 External Evaluation Metrics

Adjusted Rand Index The Adjusted Rand Index (ARI) is a widely used external metric to assess the similarity between two clustering solutions by measuring the degree of agreement between the actual labels and those assigned by the algorithm. Unlike the traditional Rand Index, the ARI corrects the obtained value by taking into account the concordances expected by pure chance. Its formulation is:

$$ARI = \frac{RI - \text{Expected}(RI)}{\text{Max}(RI) - \text{Expected}(RI)} \quad (3.6.5)$$

where the Rand Index (RI) expresses the proportion of pairs of data points that are correctly classified as belonging to the same or different clusters. The term $\text{Expected}(RI)$ represents the expected value of the RI under random assignment, while $\text{Max}(RI)$ normalizes the index to the range -1 to 1 , making the ARI more interpretable. Despite its usefulness, the ARI has some limitations. It is sensitive to the presence of unbalanced clusters, potentially favouring larger ones. Furthermore, it assumes knowledge of the true number of clusters, making it less reliable when

this information is unknown or when the algorithm produces a different number of clusters. Correcting for chance relies on the assumption of completely random assignments, which may not reflect the nature of the data. Because the ARI operates through pairwise comparisons, it does not capture more complex relationships between clusters. Finally, it necessarily requires ground-truth labels, which are unavailable in many unsupervised learning scenarios. Nevertheless, the ARI remains a popular and interpretable metric for assessing clustering quality when true labels are available.

Normalized Mutual Information Normalized Mutual Information (NMI) is a widely used external metric to evaluate the degree of agreement between real labels and those produced by a clustering algorithm. It measures the extent to which the two partitions share mutual information, offering a balanced evaluation that takes into account both the homogeneity and completeness of the clustering solution. NMI is defined as:

$$NMI = \frac{2 I(Y; C)}{H(Y) + H(C)} \quad (3.6.6)$$

where Y represents the set of real labels, C is the set of labels assigned by clustering, $I(Y; C)$ is the mutual information between the two partitions, while $H(Y)$ and $H(C)$ indicate their respective entropies. The NMI assumes values between 0 and 1: a value of 0 indicates no relationship between clustering and ground truth, while a value of 1 indicates a perfect match. The NMI is widely used because, thanks to normalization, it allows comparison between solutions obtained on datasets of different sizes, and because it provides a balanced representation of the quality of the estimated partition. However, it has some limitations: in particular, it assumes that each cluster corresponds predominantly to a single class, a condition that does not always reflect the nature of real data. Despite this, the NMI remains a robust and easily interpretable metric for assessing the alignment between clustering and class structure when real labels are available. [16]

Chapter 4

Methodology and Implementation

The aim of this chapter is to provide a clear, transparent, and well-motivated description of the methodological choices and experimental workflow adopted.

4.1 Core Idea

The central idea behind this work is to analyse the performance of dimensionality reduction and clustering algorithms, both individually and in combination. To systematically evaluate how these algorithms perform under different conditions, it is necessary to have controlled scenarios that allow selectively varying the factors that most influence the behaviour of the models, such as the number of observations, the number of variables, the presence of correlations between features, the level of separability between clusters, and the presence of noise or outliers. This requirement led to the design of a function dedicated to the generation of synthetic datasets, in which each parameter can be explicitly adjusted. This function allows for the construction of datasets with known and scalable structural characteristics, making it possible to identify which aspects most impact the performance of the different algorithms, as well as highlight the strengths and weaknesses of individual methodologies. In the following sections, we will analyse in detail how the generation function is able to construct datasets and manipulate the inner properties of the data. All the mechanisms described will be reported in full in code form in the Appendix 7.2.

4.2 Synthetic Dataset Generation Pipeline

Figure 9 illustrates the general operation of the synthetic dataset generation function. The process begins with the initialization of user-defined parameters, which control the statistical and structural characteristics of the final dataset. Later, cluster centroids are generated, enforcing minimal separation between them in the feature space. Starting from each centroid, the observation distribution phase is then launched, which represents the core of the generation process, as it controls both the number of samples assigned to each cluster and the statistical modelling of their spatial dispersion.

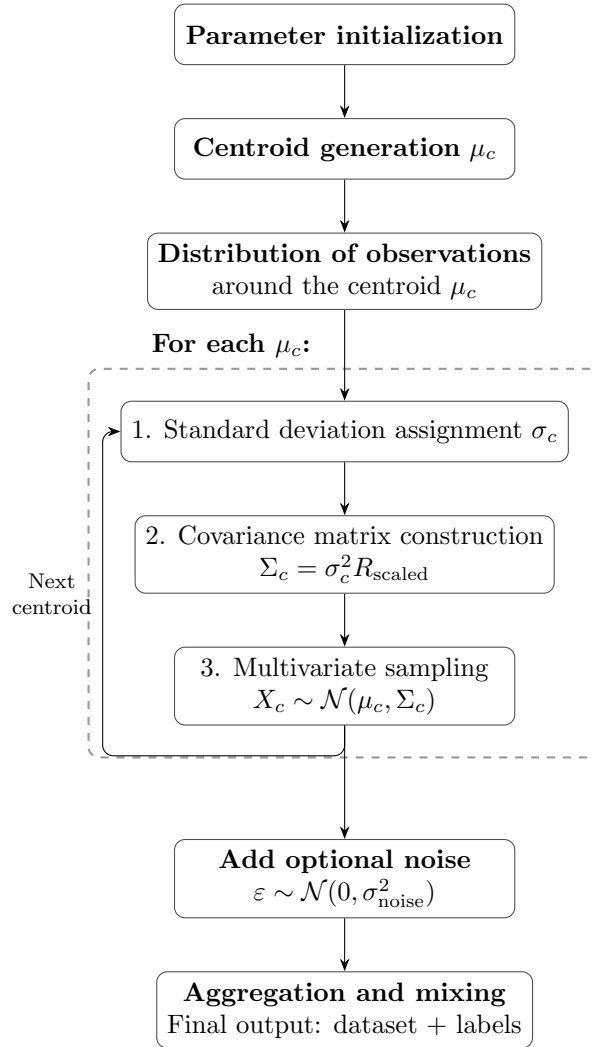


Figure 9: Complete workflow of the dataset generation function.

Specifically, each centroid is given a specific standard deviation. This value, along with the user-selected correlation scaling factor, is used to construct a covariance matrix. This matrix defines the dependence structure between features and is used as a parameter for a multivariate normal distribution with mean equal to the centroid considered. Sampling from the multivariate normal distribution is repeated iteratively for each cluster, producing a set of correlated observations associated with their respective centroid. At the end of the generation process, optional Gaussian noise can also be added to the data to increase the dataset’s realism. Finally, all generated observations are aggregated, labelled, and shuffled, returning the final dataset ready for analysis.

Generating separate centroids Starting with centroids creation, they are generated using a random sampling procedure with a distance constraint, the goal of which is to ensure minimal separation between clusters. This condition is essential: if two centroids are too close, primarily in the case of clusters characterized by high variance, the overlapping distributions would make it difficult to correctly distinguish latent structures. Under these conditions, any failure of clustering methods would be due to the poor quality of the synthetic dataset and not to the performance of the algorithm. To avoid this situation, a minimum separation distance between centroids is defined, thus ensuring a controlled level of separability. This distance is expressed as a multiple of the maximum standard deviation between clusters:

$$d_{\min} = \text{separation factor} \cdot \sigma_{\max}. \quad (4.2.1)$$

This formulation allows the separation to be proportional to the actual scale of the data. Generation proceeds by sampling each centroid from a uniform distribution on a fixed arbitrary bounded sampling domain:

$$[-10, 10]^p, \quad \text{with } p = \text{number of features}. \quad (4.2.2)$$

The first centroid is accepted immediately, while a sample-and-reject mechanism is applied to all subsequent ones. This means that each new candidate is compared

with the centroids already accepted, verifying the constraint:

$$\|\mu_c - \mu_{c'}\|_2 \geq d_{\min} \quad \forall c' < c. \quad (4.2.3)$$

If this condition is not satisfied, the sample is discarded and a new one is generated from a uniform distribution, up to a maximum of 1000 attempts. This limit prevents infinite loops if the imposed constraints are too restrictive with respect to the sampling space. This procedure ensures that the centroids are chosen randomly but consistently with the assumption of well-separable clusters, a necessary condition for a reliable evaluation of clustering and dimensionality reduction algorithms.

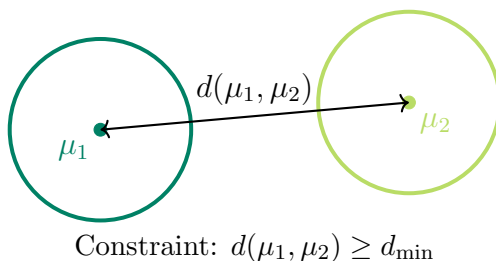


Figure 10: Centroids generation with minimum distance constraint

Distribution of observations Once the cluster centroids have been generated, the number of observations associated with each cluster needs to be chosen. This step defines the cardinality of each cluster prior to sample generation and is entirely controlled by the user. The function supports three alternative distribution strategies, which are automatically selected based on the format of the input provided.

1. Uniform distribution (default): if no custom distribution is specified, the total number of samples n is distributed as evenly as possible across the K clusters. Each cluster is initially assigned:

$$b = \left\lfloor \frac{n}{K} \right\rfloor \quad (4.2.4)$$

observations, while any remaining samples are assigned one by one to the first clusters, ensuring that the total number of generated observations is exactly

equal to n .

2. Percentage-based distribution: if a list of real values $p_i \in [0, 1]$ summing to one is provided, these values are interpreted as proportions of the total dataset size. The number of observations assigned to the i -th cluster is computed as:

$$n_i = \lfloor n \cdot p_i \rfloor. \quad (4.2.5)$$

Any residual difference

$$d = n - \sum_i n_i \quad (4.2.6)$$

resulting from rounding is distributed among the clusters with the largest fractional remainders, thus preserving the relative proportions specified by the user.

3. Explicit distribution: if the input consists of integer values, these are interpreted as exact cluster sizes. In this case, the validity condition

$$\sum_i n_i = n \quad (4.2.7)$$

must be satisfied; otherwise, an error is raised.

By supporting these three mechanisms, the function allows the user to model a wide range of experimental settings, ranging from perfectly balanced datasets to highly skewed cluster configurations.

Cluster standard deviation The internal dispersion of observations in a cluster is described by the standard deviation, which represents the square root of the variance. It determines the compactness of the cluster: low values generate denser groups, while high values produce more diffuse ones. In the adopted model, each cluster can be given its own standard deviation, allowing for the modelling of groups with different levels of cohesion. The cluster geometry is also influenced by the correlation between features, which determines the orientation and distension of the distribution's isosurfaces. Clusters characterized by low correlation exhibit nearly circular shapes,

while high correlations generate stretched and oriented ellipses. The interaction between standard deviation and correlation therefore defines the anisotropic structure of clusters, as illustrated in Figure 11.

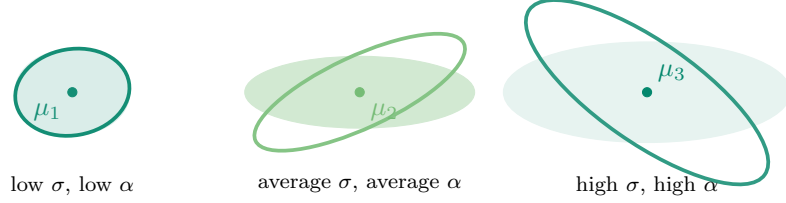


Figure 11: How σ (standard deviation) and α (correlation scaling factor) control distention and orientation. Ellipses represent isosurfaces of the $\mathcal{N}(\mu, \Sigma)$ distribution.

The first cluster exhibits a low standard deviation and almost zero correlation, producing a compact shape close to circularity. The second exhibits a high correlation but moderate dispersion, resulting in a more elongated ellipse oriented along the main direction of the correlated components. Finally, the third cluster combines high standard deviation and marked anisotropy, giving rise to a large and highly directional distribution. These configurations illustrate how the joint modulation of dispersion and correlation determines the geometric organization of the clusters.

Generating the covariance matrix A key component of the proposed data generation procedure is the construction of the covariance matrix, which controls both the dispersion of the observations within each cluster and the linear dependence structure among the features. The idea is to start from the formula that relates covariance, variance, and correlation. We know that, starting from two random variables X and Y , their linear correlation is defined as

$$\text{Corr}(X, Y) = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}, \quad (4.2.8)$$

So, we first start from generating randomly a symmetric and positive definite matrix Σ_{base} , whose dimension is equal to the number of features of the dataset. This matrix provides a valid but otherwise unconstrained covariance structure, serving as

a neutral starting point. To ensure that the resulting matrix captures only the linear dependence between features and is independent of their individual variances, Σ_{base} is transformed into a correlation matrix via the normalization

$$R_{ij} = \frac{\Sigma_{\text{base}, ij}}{\sqrt{\Sigma_{\text{base}, ii}} \sqrt{\Sigma_{\text{base}, jj}}}, \quad (4.2.9)$$

which takes up the general formula indicated previously in Equation 4.2.8. We finally obtain a matrix with unit diagonal and off-diagonal elements representing pure pairwise correlations. The desired correlation intensity is applied by explicitly separating the diagonal and off-diagonal components of the correlation matrix. Specifically, the identity matrix I is first subtracted from R in order to isolate the off-diagonal terms, which encode the pairwise linear dependencies between features. These off-diagonal elements are then uniformly scaled by a user-defined factor α , while the diagonal is subsequently reintroduced to preserve unit variance. To ensure that the resulting matrix remains a valid correlation matrix, the scaling factor is restricted to the interval $\alpha \in [0, 1]$, which guarantees positive definiteness by construction.¹ This procedure can be expressed as:

$$R_{\text{scaled}} = \alpha(R - I) + I. \quad (4.2.10)$$

As a result, the diagonal entries of R_{scaled} remain equal to one, whereas the strength of the off-diagonal correlations is continuously modulated. Finally, the correlation matrix is converted back into a covariance matrix by reintroducing the desired scale through the cluster-specific standard deviation σ , resulting in

$$\Sigma_{\text{final}} = \sigma^2 R_{\text{scaled}}. \quad (4.2.11)$$

¹The restriction $\alpha \in [0, 1]$ ensures that the scaled correlation matrix $R_{\text{scaled}} = \alpha(R - I) + I$ can be expressed as a convex combination $R_{\text{scaled}} = (1 - \alpha)I + \alpha R$ of two positive definite matrices, namely the identity matrix I and the original correlation matrix R . Since convex combinations of positive definite matrices remain positive definite, this constraint guarantees the mathematical validity of the resulting matrix as a correlation matrix and, consequently, as a covariance matrix suitable for multivariate normal sampling.

The resulting covariance matrix is guaranteed to be positive definite, exhibits the prescribed marginal variances, and encodes a controllable correlation structure. As a consequence, it directly determines the shape, orientation, and distension of each cluster in feature space, enabling the generation of synthetic datasets with non-spherical, correlated clusters that more closely resemble real-world data.

Construction of the final dataset The synthetic dataset is finally obtained by integrating all the previously described mechanisms into a single, consistent generation process. After the cluster-specific parameters have been defined, observations are independently drawn for each group as realizations of multivariate normal distributions centred at the corresponding centroids and shaped by their associated covariance matrices. We recall that cluster centres are generated in order to satisfy a minimum separation constraint, ensuring that clusters remain well-separated even for larger values of dispersion. Moreover, the covariance structure of each cluster is built by combining a valid positive definite matrix with a tunable correlation scaling factor α , while the overall spread is regulated by the standard deviation σ . The samples generated across all clusters are subsequently aggregated and randomly permuted, ensuring that no artificial structure is introduced by the order in which the data are produced. At this stage, an optional additive Gaussian noise can be applied to the observations, providing a controlled way to increase variability while preserving the underlying cluster geometry. The resulting dataset combines well-defined ground-truth cluster structure with adjustable levels of noise and statistical complexity, making it particularly suitable for the systematic evaluation of clustering and dimensionality reduction techniques under controlled and reproducible conditions.

4.3 Cardinality and Dimensionality of the Dataset

This analysis considers three macro-scenarios that reflect realistic data analysis situations, characterized by different ratios between the number of observations and the number of variables.

2.500 observations and 15 variables The first scenario represents a context in which the number of observations is relatively small compared to the number of variables. A 15-dimensional dataset, while not extremely high in terms of number of dimensions, may already exhibit some effects attributable to the curse of dimensionality discussed in chapter 2, such as a reduced discriminative power of distances and greater instability of the identified clusters. In such a context, it is reasonable to hypothesize that dimensionality reduction techniques could be useful for synthesizing information into a more compact space. At the same time, projecting into lower-dimensional spaces inevitably introduces information loss, which could affect some latent structures relevant to clustering. [29]

10.000 observations and 5 variables At the other extreme, consider a dataset with many observations but few variables. In this case, a high observation-to-variable ratio could theoretically favour more stable estimates of latent structures and a reduced influence of noise. The need for dimensionality reduction seems less critical, although such techniques remain useful for evaluating possible benefits on the final quality of the clustering. From a computational view, however, the large number of observations directly impacts computational costs, especially for distance-based algorithms such as k-means and PAM. From a statistical perspective, a large sample with the same dimensionality should allow for more robust estimates of distances and centroids compared to less populated contexts.

5.000 observations and 10 variables The third scenario represents an intermediate situation, with a more balanced balance between the number of observations and the number of variables. This case allows us to observe the behaviour of the algorithms in a less extreme configuration, serving as a reference for comparing how performance varies with the combined change in dimensionality and sample size. Overall, the comparison between the three scenarios allows us to analyse how different configurations can influence the performance of the considered algorithms. [30, 31, 32]

4.4 Cluster Cardinality and Standard Deviation

In addition to the three macro-situations described above, three further sub-scenarios are considered for each configuration, differing in the number of intrinsic clusters and in the distribution of observations in terms of cardinality and dispersion. In each case, observations are distributed around a set of centroids according to a hierarchical and proportional structure: there is always a main cluster, characterized by the greatest number of observations and a higher standard deviation, followed by groups of progressively smaller clusters, each with a cardinality equal to half of the previous one. In a real-world application context, this main cluster can be interpreted as the most populated and, at the same time, the most heterogeneous class, while subsequent clusters represent increasingly specific and less variable sub-populations. Clusters with the same cardinality are always generated in pairs and differ only in standard deviation, allowing us to evaluate the algorithms' behaviour when faced with groups of equal size but different density. Dispersion also follows a fixed structure: the main cluster has a standard deviation of 5, while subsequent pairs are characterized by values of 3.5 and 2, respectively. This setting allows us to analyse the algorithms' ability to distinguish clusters that differ in both size and internal variability. Finally, to ensure a controlled and comparable level of separation between groups, the minimum distance between centroids is set equal to twice the maximum standard deviation considered, i.e. 10. This parameter remains unchanged across all simulations, thus isolating the effect of cardinality and dispersion on the performance of the analysed algorithms.

2.500 observations and 15 variables For a dataset consisting of 2.500 observations and 15 variables, three different configurations are considered, characterized by 3, 5, and 7 clusters, respectively. In all configurations, the distribution of observations follows the proportional logic described previously. In the case of 3 clusters, the main cluster contains 1.250 observations, equal to half the dataset, and is characterized by a standard deviation of 5. The remaining observations are divided into two clusters

of equal cardinality, each composed of 625 observations, but with different standard deviations, equal to 3.5 and 2, respectively. In the case of 5 clusters, the main cluster is composed of 1.000 observations. This is followed by two clusters of 500 observations each, and finally two clusters of 250 observations. Within each pair of clusters with equal cardinality, the standard deviations are assigned 3.5 and 2, respectively, maintaining the same modular structure. Finally, in the case of 7 clusters, the main cluster contains 900 observations, followed by two clusters of 450 observations, two clusters of 225 observations, and two clusters of 125 observations. Even in this scenario, pairs of clusters with equal cardinality are distinguished only by their standard deviation, set at 3.5 and 2.

5.000 observations and 10 variables For the dataset consisting of 5.000 observations and 10 variables, the same proportional logic used in the previous case is adopted, consistently doubling the cluster cardinalities. In the case of 3 clusters, the main cluster contains 2.500 observations, while the two remaining clusters each contain 1.250 observations. In the case of 5 clusters, the dataset is divided into a main cluster of 2.000 observations, two clusters of 1.000 observations, and two clusters of 500 observations. Finally, in the case of 7 clusters, the main cluster contains 1.800 observations, followed by two clusters of 900 observations, two clusters of 450 observations, and two clusters of 250 observations. In all configurations, the structure of the standard deviations remains unchanged: the main cluster has a standard deviation of 5, while each subsequent pair of clusters has standard deviations of 3.5 and 2.

10.000 observations and 5 variables In the case of the dataset consisting of 10.000 observations and 5 variables, the subdivision of the observations again follows the same modular and proportional structure. In the case of 3 clusters, the main cluster contains 5.000 observations, while the two remaining clusters each contain 2.500 observations. In the case of 5 clusters, the main cluster contains 4.000 observations, followed by two clusters of 2.000 observations and two clusters of 1.000 observations.

Finally, in the case of 7 clusters, the main cluster is composed of 3.600 observations, followed by two clusters of 1.800 observations, two clusters of 900 observations, and two clusters of 500 observations. Even in this scenario, the standard deviation of the main cluster is set to 5, while for each subsequent pair of clusters standard deviations equal to 3.5 and 2 are assigned, ensuring consistency and comparability between all the configurations considered.

Table 2: Modular structure of the clusters in the different simulated scenarios.

Macro-scenario	No. of clusters	Cluster structure (size and σ)
2500×15	3	1250 ($\sigma = 5$) 625 ($\sigma = 3.5$), 625 ($\sigma = 2$)
	5	1000 ($\sigma = 5$) 500 ($\sigma = 3.5$), 500 ($\sigma = 2$) 250 ($\sigma = 3.5$), 250 ($\sigma = 2$)
	7	900 ($\sigma = 5$) 450 ($\sigma = 3.5$), 450 ($\sigma = 2$) 225 ($\sigma = 3.5$), 225 ($\sigma = 2$) 125 ($\sigma = 3.5$), 125 ($\sigma = 2$)
5000×10	3	2500 ($\sigma = 5$) 1250 ($\sigma = 3.5$), 1250 ($\sigma = 2$)
	5	2000 ($\sigma = 5$) 1000 ($\sigma = 3.5$), 1000 ($\sigma = 2$) 500 ($\sigma = 3.5$), 500 ($\sigma = 2$)
	7	1800 ($\sigma = 5$) 900 ($\sigma = 3.5$), 900 ($\sigma = 2$) 450 ($\sigma = 3.5$), 450 ($\sigma = 2$) 250 ($\sigma = 3.5$), 250 ($\sigma = 2$)
10000×5	3	5000 ($\sigma = 5$) 2500 ($\sigma = 3.5$), 2500 ($\sigma = 2$)
	5	4000 ($\sigma = 5$) 2000 ($\sigma = 3.5$), 2000 ($\sigma = 2$) 1000 ($\sigma = 3.5$), 1000 ($\sigma = 2$)
	7	3600 ($\sigma = 5$) 1800 ($\sigma = 3.5$), 1800 ($\sigma = 2$) 900 ($\sigma = 3.5$), 900 ($\sigma = 2$) 500 ($\sigma = 3.5$), 500 ($\sigma = 2$)

Summary Table 2 summarizes the modular structure adopted for the subdivision of observations across the different simulated scenarios. It is possible to observe how the cluster distribution follows a proportional and hierarchical logic, which is kept constant as both the number of clusters and the overall dataset size vary, in order to guarantee controlled and interpretable comparisons.

4.5 Noise and Correlation Scaling Factor

Now that the composition of each cluster has been defined, we focus on their spatial arrangement and the factors that most influence their shape and orientation. Specifically, we analyse the noise added to the final dataset and the correlation scaling factor. Noise is modelled as a Gaussian random variable with zero mean and user-selected standard deviation, and is applied only after the dataset has been fully generated. The correlation scaling factor, however, does not directly represent the observed correlation, but rather constitutes a control parameter that affects the linear dependence structure between the variables, introduced during data construction. The analysis is divided into two phases: first, the noise level is kept constant and the correlation scaling factor is varied to assess its impact on the internal structure of the clusters. Subsequently, the correlation scaling factor is fixed and the algorithms’ performance is studied as the noise level varies. These scenarios allow us to simulate increasingly complex and realistic conditions, similar to those encountered in real-world application contexts, such as biomedical research, where datasets are often characterized by high dimensionality, strong dependence between variables, and the presence of significant noise [33].

Correlation scaling factor The correlation scaling factor $\alpha \in [0, 1]$ is used to control the strength of linear dependence between variables within each cluster. It does not represent the empirical correlation observed in the simulated dataset, but rather a tuning parameter that uniformly scales the off-diagonal terms of a base correlation matrix, as described in the covariance construction procedure. Two distinct scenarios

will be considered. The first scenario represents a favourable condition, characterized by low linear dependence between variables, corresponding to a factor value $\alpha = 0.1$. In this context, each feature provides a high amount of independent information, reducing redundancy and preserving the geometric meaning of distances in feature space. This results in greater separation between clusters, a particularly favourable condition for distance-based clustering algorithms such as k-means and PAM. The second scenario instead considers a less favourable situation, characterized by a high correlation scaling factor $\alpha = 0.9$, which induces a strong linear dependence between the variables. Although this configuration makes separation between clusters more difficult, it does not necessarily imply clustering failure. Techniques such as Principal Component Analysis (PCA) are particularly effective, as they transform the original space into a system of uncorrelated coordinates, reducing information redundancy and improving the geometric conditionality of the data. Following PCA, clustering algorithms should be able to operate in a more isotropic and separable space, improving the stability and compactness of the clusters. Finally, we will also consider an intermediate case with a correlation scaling factor $\alpha = 0.5$. In this first analysis, the noise level is kept constant at 7.

Noise We conclude by analysing the effect of additive Gaussian noise, which is injected after the full dataset has been generated. In the proposed implementation, noise is sampled independently for each feature as:

$$\varepsilon \sim \mathcal{N}(0, \sigma_{\text{noise}}^2),$$

where σ_{noise} denotes the noise standard deviation. We consider three noise levels, $\sigma_{\text{noise}} \in \{5, 7, 9\}$, moving from a favourable low-noise scenario to progressively more challenging settings. As expected, increasing σ_{noise} increases dispersion, reduces cluster compactness, and deteriorates the performance of both dimensionality reduction and clustering algorithms. In this second phase, the correlation scaling factor is kept fixed at $\alpha = 0.5$.

Chapter 5

Results and Discussion

This chapter provides an in-depth analysis and discussion of the results obtained from the clustering algorithms.

5.1 Procedure

Initially, each dataset is standardized and directly subjected to the clustering algorithms without any prior transformation. Subsequently, the same clustering methods are applied to datasets projected onto lower-dimensional spaces obtained through the dimensionality reduction techniques. For t-SNE, the number of components is kept constant and set equal to 2. Empirical analyses show that t-SNE is sensitive to the choice of the embedding dimensionality, with better results obtained when using two components instead of three. UMAP, on the other hand, proves to be less sensitive to the number of components; therefore, a default value of three components is chosen. All the remaining parameters were set to their default values: t-SNE perplexity = 30, UMAP k neighbors = 15 and $min-dist = 0.1$. This choice ensures reproducibility and provides a standard baseline configuration. In the case of PCA, the number of components is determined automatically by selecting only those with eigenvalues greater than one, according to the Kaiser criterion.¹ The remaining main hyperparameters of the employed algorithms can be found in Appendix at Table 10.

¹The threshold of 1 is adopted because, when data are standardized, each original variable has unit variance. Consequently, a principal component with an eigenvalue lower than 1 explains less variance than a single original variable.

Final tables The clustering algorithms are applied to three macro-datasets differing in the number of observations and variables. Each dataset is generated with different parameters, including the number of clusters ranging from 3 to 5 and 7, a correlation scaling factor, and noise. In the first phase, noise σ is fixed at 7 while the correlation scaling factor α varies between 0.1, 0.5, and 0.9. In the second phase, the correlation scaling factor α is fixed at 0.5, and the noise level σ varies between 5, 7, and 9.

5.2 Study on Correlation Scaling Factor

In this section, we analyse the case in which the noise level in the dataset remains constant, while the correlation scaling factor between the variables changes. Visually, as shown in Figure 12 and Figure 13, it is possible to see how the correlation scaling factor affects the shape and orientation of the clusters and, consequently, the data

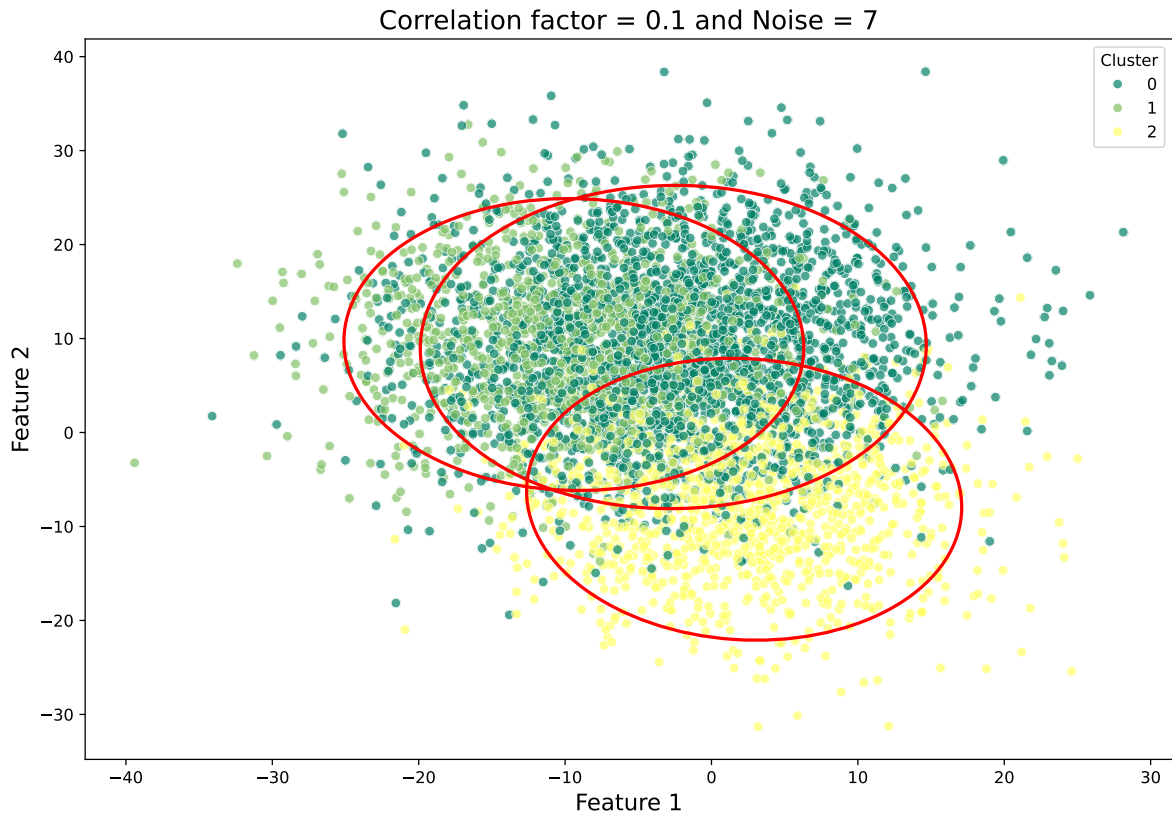


Figure 12: Scatter plot of the first and second variables of the dataset with 5.000 observations, 10 variables, 3 clusters, $\alpha = 0.1$ and $\sigma = 7$.

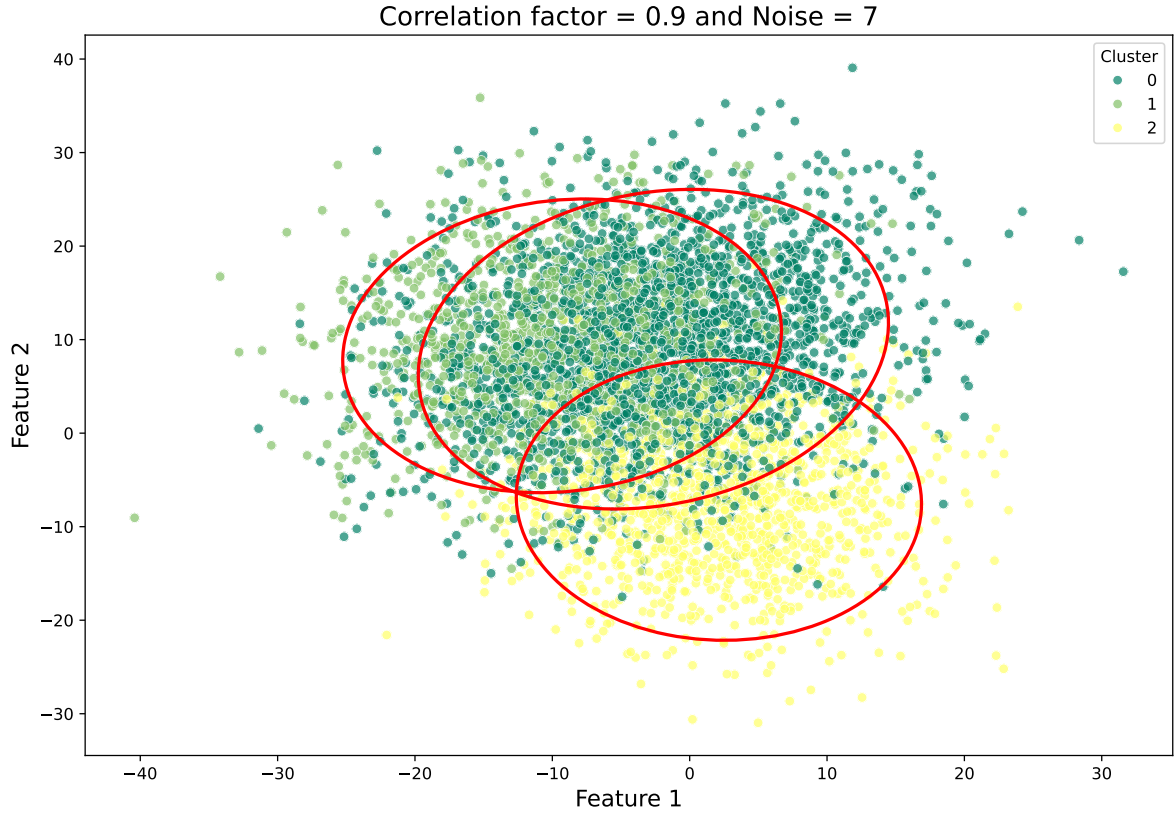


Figure 13: Scatter plot of the first and second variables of the dataset with 5.000 observations, 10 variables, 3 clusters, $\alpha = 0.9$ and $\sigma = 7$.

modelling. Specifically, the figures represent the case with 5.000 observations, 10 variables, and 3 clusters: we observe the presence of a larger main cluster characterized by greater dispersion around the centroid, surrounded by two more compact and less noisy clusters, in line with what was discussed in the previous chapter. Although not particularly pronounced, one difference concerns the geometry of the clusters. In cases with a high correlation scaling factor, the clusters appear more elongated and oriented in the same direction, especially the two largest clusters, highlighting an anisotropic structure.

Correlated variables High correlation between variables could reduce clustering performance for several reasons. First, highly correlated variables introduce information redundancy: they do not give truly new knowledge, but rather replicate content that already exists in other dimensions. Second, many clustering algorithms rely on

distance metrics. In the presence of collinearity, some directions in the data space are overweighted compared to others, altering the geometry of the distance space and leading to potentially overlapping clusters. Furthermore, correlation tends to generate elliptical and directional clusters rather than spherical ones. However, algorithms like k-means implicitly assume isotropic clusters, meaning they are characterized by similar variance along all dimensions. Violating this assumption results in less accurate and more unstable results. Finally, the common orientation of clusters in the presence of high correlation can increase the areas of overlap between groups, making it more difficult to correctly distinguish observations and amplifying the algorithms' sensitivity to initial values and residual noise.[34]

A general overview

The main goal of this analysis is to provide an in-depth description of the tables containing the evaluation metrics of clustering algorithms. This chapter presents the tables evaluated using the ARI index, while Appendix 7.2 reports the same tables evaluated using the NMI metric.

Cardinality and dimensionality of the dataset An initial descriptive analysis shows that clustering performance tends to drop as the number of observations increases while the number of variables decreases: the algorithms achieve better results on average for 2.500 observations and 15 variables, while performance progressively declines in the scenarios with 5.000 observations and 10 variables and, even more markedly, with 10.000 observations and only 5 variables. This behaviour appear to be driven by the joint effect of multiple structural factors. As the sample size increases, the number of available features simultaneously decreases, meaning that a larger number of observations must be separated using a smaller set of informative variables. This reduction in dimensionality implies a loss of discriminative power: the clusters become less distinct, the overlap between groups increases and the distance between centroids becomes less strong relative to the within-cluster variability.

Number of clusters Another relevant factor concerns the number of clusters: moving from 3 to 5 and then to 7 clusters, a progressive drop of the ARI indices is observed in most cases. Increasing the number of clusters makes the classification problem more complex: the algorithms must distinguish a greater number of structures, the assignment alternatives increase, and classification errors become more likely, especially in the presence of noise and correlation between variables. In addition, greater variability among algorithms is observed: as the number of clusters increases, some methods, such as PAM and GMM, experience a more marked drop than others. In the former

Observations and Features:	2500 observations and 15 features								
Number of Clusters:	3 clusters			5 clusters			7 clusters		
Correlation Factor:	0,1	0,5	0,9	0,1	0,5	0,9	0,1	0,5	0,9
k-means	0,890	0,897	0,414	0,570	0,795	0,718	0,814	0,626	0,605
PCA + k-means	0,882	0,413	0,876	0,566	0,551	0,656	0,606	0,587	0,551
t-SNE + k-means	0,811	0,802	0,760	0,696	0,517	0,567	0,548	0,468	0,491
UMAP + k-means	0,901	0,892	0,906	0,577	0,837	0,571	0,759	0,591	0,611
Birch	0,819	0,843	0,835	0,684	0,520	0,533	0,740	0,537	0,534
PCA + Birch	0,852	0,883	0,895	0,808	0,530	0,513	0,720	0,543	0,492
t-SNE + Birch	0,808	0,822	0,743	0,632	0,578	0,496	0,613	0,393	0,511
UMAP + Birch	0,901	0,892	0,906	0,788	0,662	0,674	0,594	0,699	0,600
GMM	0,914	0,928	0,448	0,750	0,679	0,835	0,515	0,504	0,619
PCA + GMM	0,900	0,901	0,914	0,704	0,480	0,787	0,520	0,605	0,551
t-SNE + GMM	0,384	0,820	0,871	0,639	0,557	0,576	0,426	0,472	0,486
UMAP + GMM	0,895	0,899	0,918	0,603	0,835	0,572	0,700	0,629	0,571
PAM	0,563	0,427	0,708	0,485	0,473	0,458	0,418	0,421	0,358
PCA + PAM	0,884	0,889	0,879	0,810	0,533	0,545	0,606	0,591	0,549
t-SNE + PAM	0,816	0,808	0,782	0,554	0,516	0,567	0,556	0,475	0,571
UMAP + PAM	0,900	0,894	0,905	0,835	0,559	0,848	0,609	0,613	0,621
Spectral Clustering	0,909	0,915	0,907	0,846	0,853	0,872	0,827	0,825	0,628
PCA + Spectral Clustering	0,896	0,909	0,906	0,835	0,806	0,563	0,618	0,593	0,564
t-SNE + Spectral Clustering	0,727	0,763	0,590	0,485	0,475	0,561	0,480	0,443	0,579
UMAP + Spectral Clustering	0,890	0,898	0,910	0,837	0,836	0,854	0,813	0,623	0,582

Figure 14: Results evaluated according to ARI in the case of 2500 observations and 15 variables and keeping constant noise at 7

Observations and Features:	5000 observations and 10 features								
Number of Clusters:	3 clusters			5 clusters			7 clusters		
Correlation Factor:	0,1	0,5	0,9	0,1	0,5	0,9	0,1	0,5	0,9
k-means	0,512	0,472	0,418	0,363	0,344	0,343	0,380	0,364	0,324
PCA + k-means	0,520	0,456	0,415	0,343	0,278	0,261	0,257	0,269	0,240
t-SNE + k-means	0,362	0,434	0,382	0,316	0,322	0,305	0,263	0,254	0,244
UMAP + k-means	0,488	0,446	0,399	0,453	0,342	0,434	0,408	0,307	0,293
Birch	0,459	0,433	0,405	0,336	0,291	0,291	0,202	0,239	0,232
PCA + Birch	0,372	0,306	0,399	0,308	0,276	0,267	0,232	0,242	0,192
t-SNE + Birch	0,440	0,304	0,354	0,304	0,361	0,288	0,276	0,207	0,234
UMAP + Birch	0,490	0,439	0,356	0,393	0,289	0,270	0,380	0,388	0,303
GMM	0,380	0,416	0,421	0,360	0,415	0,318	0,261	0,267	0,203
PCA + GMM	0,519	0,345	0,436	0,260	0,283	0,237	0,220	0,261	0,203
t-SNE + GMM	0,286	0,377	0,331	0,294	0,291	0,318	0,254	0,263	0,271
UMAP + GMM	0,456	0,371	0,343	0,302	0,305	0,390	0,364	0,259	0,242
PAM	0,463	0,393	0,343	0,290	0,305	0,285	0,261	0,289	0,243
PCA + PAM	0,522	0,457	0,414	0,353	0,261	0,260	0,258	0,233	0,231
t-SNE + PAM	0,423	0,433	0,390	0,314	0,324	0,295	0,271	0,235	0,240
UMAP + PAM	0,496	0,438	0,403	0,463	0,461	0,437	0,406	0,324	0,300
Spectral Clustering	0,552	0,446	0,390	0,492	0,449	0,415	0,492	0,444	0,406
PCA + Spectral Clustering	0,505	0,448	0,419	0,333	0,273	0,216	0,250	0,215	0,191
t-SNE + Spectral Clustering	0,319	0,371	0,380	0,294	0,279	0,292	0,252	0,221	0,237
UMAP + Spectral Clustering	0,494	0,397	0,381	0,434	0,438	0,431	0,471	0,442	0,385

Figure 15: Results evaluated according to ARI in the case of 5000 observations and 10 variables and keeping constant noise at 7

case, the high dependence on medoids makes it more difficult to identify truly significant representatives when the clusters become numerous, while in the latter, the probabilistic nature of the model leads to an increase in the number of parameters to be estimated. Furthermore, we can note that the fall of the ARI index is not always monotonic and linear. In particular, increasing the number of clusters tends to amplify the negative effects of internal correlation: in the presence of numerous clusters and correlated variables, the structures become more extended and oriented in the same directions, increasing the likelihood of overlap between groups.

Observations and Features:	10000 observations and 5 features								
Number of Clusters:	3 clusters			5 clusters			7 clusters		
Correlation Factor:	0,1	0,5	0,9	0,1	0,5	0,9	0,1	0,5	0,9
k-means	0,319	0,299	0,260	0,206	0,209	0,216	0,190	0,186	0,198
PCA + k-means	0,307	0,272	0,254	0,213	0,208	0,157	0,150	0,145	0,169
t-SNE + k-means	0,273	0,251	0,229	0,205	0,189	0,195	0,186	0,178	0,162
UMAP + k-means	0,304	0,304	0,267	0,223	0,226	0,204	0,187	0,204	0,183
Birch	0,303	0,287	0,225	0,165	0,132	0,160	0,137	0,169	0,108
PCA + Birch	0,289	0,255	0,242	0,171	0,186	0,120	0,215	0,204	0,172
t-SNE + Birch	0,198	0,078	0,270	0,141	0,191	0,119	0,164	0,169	0,148
UMAP + Birch	0,290	0,252	0,266	0,231	0,192	0,204	0,175	0,225	0,188
GMM	0,233	0,264	0,221	0,141	0,187	0,139	0,171	0,082	0,150
PCA + GMM	0,049	0,214	0,209	0,173	0,148	0,144	0,195	0,207	0,155
t-SNE + GMM	0,112	0,222	0,073	0,207	0,153	0,162	0,138	0,185	0,158
UMAP + GMM	0,310	0,270	0,400	0,249	0,193	0,130	0,165	0,196	0,184
PAM	0,318	0,349	0,250	0,228	0,200	0,230	0,190	0,188	0,184
PCA + PAM	0,312	0,341	0,271	0,213	0,204	0,164	0,162	0,153	0,168
t-SNE + PAM	0,279	0,251	0,233	0,208	0,192	0,151	0,186	0,180	0,162
UMAP + PAM	0,307	0,300	0,269	0,221	0,226	0,205	0,187	0,211	0,185
Spectral Clustering	0,334	0,310	0,257	0,218	0,204	0,216	0,184	0,194	0,199
PCA + Spectral Clustering	0,311	0,293	0,258	0,208	0,196	0,153	0,153	0,142	0,164
t-SNE + Spectral Clustering	0,203	0,000	0,085	0,176	0,167	0,184	0,152	0,195	0,150
UMAP + Spectral Clustering	0,324	0,313	0,272	0,221	0,232	0,207	0,166	0,196	0,193

Figure 16: Results evaluated according to ARI in the case of 10000 observations and 5 variables, keeping noise constant at 7

In-depth analysis

Regarding the correlation scaling factor within the variables, a general tendency, although not always perfectly monotonic, is observed for performance to drop as the correlation scaling factor increases. However, this behaviour is not universal: especially in the dataset with 2.500 observations and 15 variables, in some scenarios increasing correlation does not lead to an immediate deterioration, suggesting an interaction with dimensionality and the number of clusters. We will now analyse the behaviour of each single algorithm as the various parameters vary.

k-means Overall, we get excellent performance of the basic k-means algorithm. analysing the 2.500 observations and 15 variables dataset, an interesting behaviour emerges: with 5 clusters increasing correlation scaling factor leads to improved performance, apparently counter to the general trend. This can happen because, in certain configurations, the correlation highlights directions of predominant variance that make the structures more easily separable even for an algorithm that assumes isotropic clusters. In combination with PCA, k-means does not represent a universally superior solution: in fact, in most cases it tends to worsen the already excellent performances obtained by k-means, presenting an advantage only in the case of high correlation with 3 clusters. Combinations with t-SNE are often less effective, while UMAP combined with k-means proves to be the most robust solution on average. In the datasets with 5.000 observations and 10 variables and 10.000 observations and 5 variables, the dynamics become less extreme and UMAP remains the most reliable configuration.

Birch In the dataset with 2.500 observations and 15 variables, Birch shows good clustering results even when applied alone, improving only when combined with UMAP that ensure greater stability compared to the base algorithm. This result appears consistent with the nature of the method: as discussed in the theoretical section, Birch builds a hierarchical structure, called CF-Tree, that progressively synthesizes the data and benefits from compact and structurally consistent representations, such as those provided by UMAP. In contrast, t-SNE and Birch exhibits lower performance and greater volatility. Similar behaviours emerge in the dataset with 5.000 observations and 10 variables, where UMAP and Birch represent the best solution even in scenarios with 7 clusters. Also in the case with 10.000 observations and 5 variables, the pattern remains similar: UMAP continues to be an effective support, while t-SNE confirms its difficulties, especially in the low cluster scenarios.

GMM In the dataset with 2.500 observations and 15 variables, GMM achieves very high performance even on its own, further improved by the integration with UMAP.

This behaviour is fully consistent with the probabilistic nature of the model: GMM represents the data as a combination of Gaussian distributions estimated via EM and benefits from spaces in which the structures are more compact and well-separated. In the dataset with 5.000 observations and 10 variables, a similar behaviour is observed, with one significant critical issue: t-SNE in combination with GMM shows a marked deterioration in the case of low correlation and 3 clusters, a theoretically favourable scenario. In the case of 10.000 observations and 5 variables, UMAP still represents the most effective solution, while PCA and t-SNE encounter greater difficulties.

PAM PAM benefits significantly from the integration with the dimensionality reduction algorithms, achieving a significant improvement over the baseline algorithm. This result is consistent with the theoretical discussion: PAM, while more robust than k-means thanks to the use of medoids, remains sensitive to geometric deformations induced by correlation and benefits from more compact and less redundant representations. Of particular importance is the case with 3 clusters, where the influence of algorithms such as UMAP is particularly evident as the ARI index is almost doubled compared to the PAM algorithm alone. In the dataset with 5.000 observations and 10 variables, UMAP emerges as one of the best solutions overall, while in the more complex case with 10.000 observations and 5 variables, the differences between the configurations are markedly reduced.

Spectral Clustering Spectral clustering remains among the algorithms with the highest average performance and stability, a result consistent with theory: as discussed in chapter 3, the construction of the similarity graph and the use of the spectral properties of the Laplacian allow the identification of even non-spherical clusters with complex structures. Integration with PCA and UMAP does not provide radical improvements, suggesting that the spectral transformation is already an effective form of structural reduction. In contrast, t-SNE introduces greater instability, especially in low correlation cases. In the dataset with 5.000 observations and 10 variables, Spectral

Clustering continues to provide superior results compared to the other algorithms even without preprocessing, while in the case with 10.000 observations and 5 variables, it remains one of the most stable algorithms, despite not benefiting significantly from further transformations of the data space.

Dimensionality reduction algorithms Generally speaking, PCA can be an effective tool for mitigating the effects of collinearity and improving performance; however, in this setting it does not lead to clear improvements to the basic clustering algorithm. The same goes for t-SNE. Its performance is highly sensitive to initialization and hyper-parameter choices, such as perplexity and embedding dimensionality. Moreover, t-SNE primarily focuses on preserving local neighborhood relationships, which can distort global inter-cluster structure and negatively affect clustering quality. In contrast, UMAP emerges as the most stable and robust solution, consistently providing effective support for clustering across different configurations. UMAP differs from t-SNE as it is explicitly designed to preserve both local and global aspects of the data manifold. This characteristic makes it more suitable as a preprocessing step for clustering algorithms, particularly in complex scenarios. Overall, the results confirm that increasing structural complexity, namely a larger number of clusters, stronger correlations among variables and reduced discriminative information, tends to degrade clustering performance. Nevertheless, appropriate dimensionality reduction techniques, with UMAP in particular, can partially mitigate these effects and improve robustness and stability.

Comparative algorithm summary

The results highlight that the choice of algorithm cannot be guided just by computational considerations, but must take into account the structure of the dataset. In scenarios characterized by low correlation and a limited number of clusters, algorithms such as k-means, PAM, and GMM represent efficient solutions, especially when supported by UMAP. As structural complexity increases (higher correlation, more clusters, greater overlap between groups), more robust approaches such as Spectral

Clustering become preferable. Birch occupies an intermediate position, proving efficient in favourable contexts but less stable in highly challenging configurations. In general, dimensionality reduction techniques can improve the quality of clustering, but must be used wisely: PCA can be beneficial in specific scenarios, while UMAP consistently provides the most reliable improvements. Finally, t-SNE as preprocessing must be used carefully when the primary goal is not visualization, but rather the stability and interpretability of the resulting partitions.

5.3 Study on Noise

We now analyse clustering performance when the noise level varies, keeping the correlation scaling factor constant at 0.5. Figure 17 and Figure 18 visually demonstrate

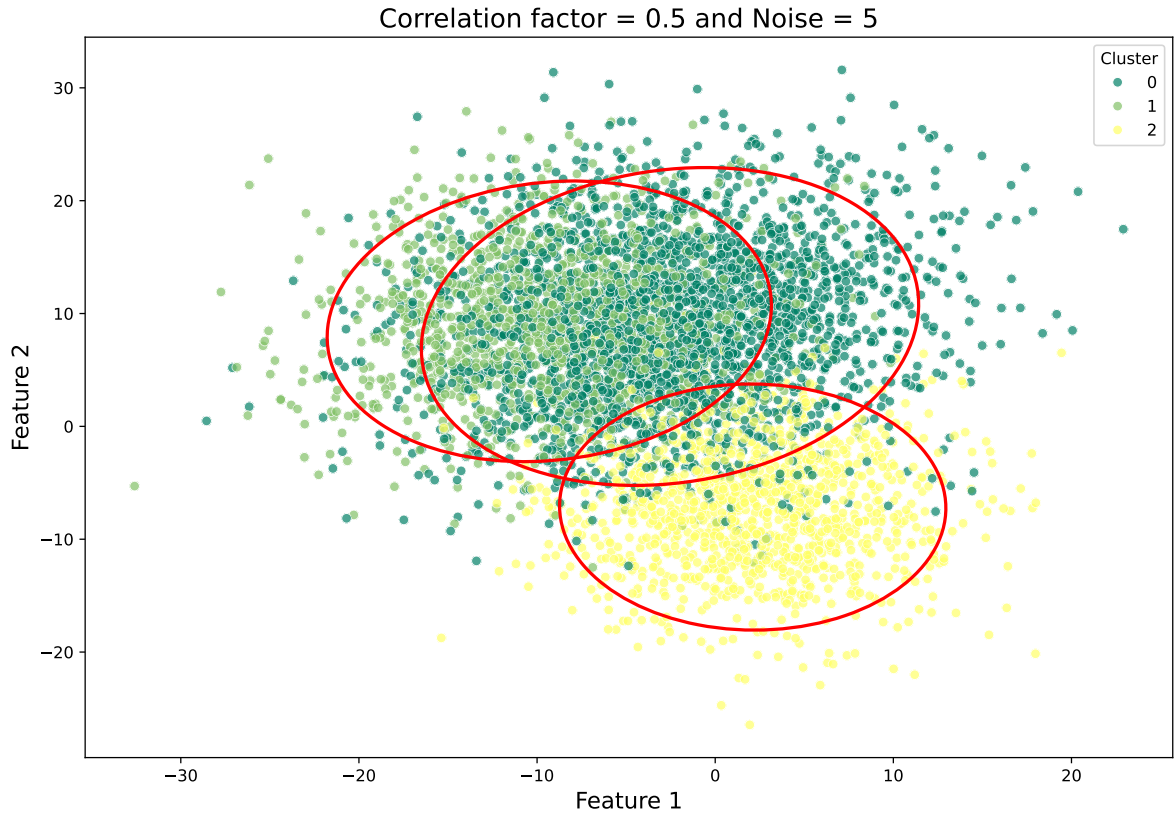


Figure 17: Scatter plot of the first and second variables of the dataset with 5.000 observations, 10 variables, 3 clusters, $\alpha = 0.5$ and $\sigma = 5$.

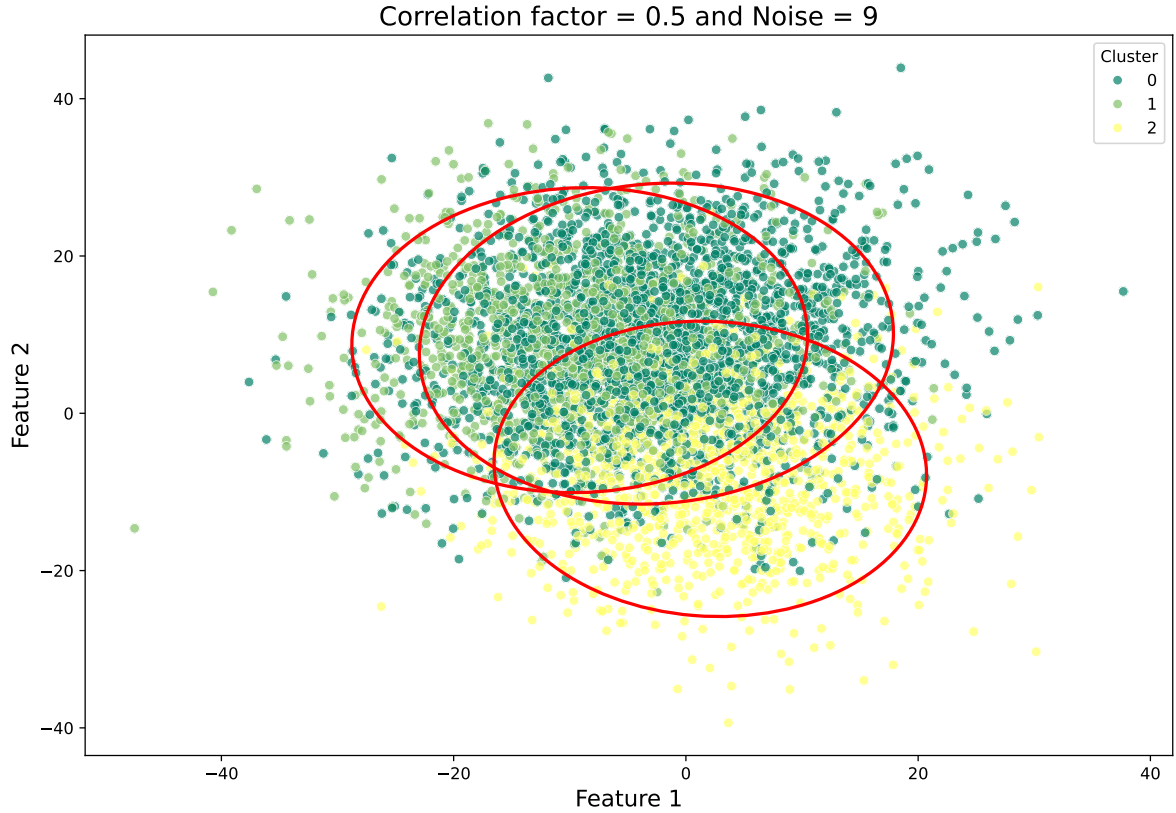


Figure 18: Scatter plot of the first and second variables of the dataset with 5.000 observations, 10 variables, 3 clusters, $\alpha = 0.5$ and $\sigma = 9$

how noise affects the dispersion of observations around their respective centroids. Although the difference is not immediately evident, a closer look reveals a progressive increase in the spread of points and a greater average distance of observations from the reference centroids. In particular, we observe how the boundaries between clusters become less clear and the potential areas of overlap between different groups increase.

Noisy dataset The introduction of noise into the data has a clear impact on clustering performance, generally resulting in a progressive drop of quality indices as the level of noise increases. This occurs because noise makes clusters less compact, increasing internal dispersion and blurring the boundaries separating them. In Euclidean distance-based methods, such as k-means and PAM, this phenomenon translates into greater instability in the estimation of centroids or medoids and, as a consequence,

less reliable assignments. In probabilistic models, such as GMM, noise increases uncertainty in estimating distributions, widening the regions of overlap between Gaussian components and worsening the distinction between clusters. In this study, noise is introduced in the form of Gaussian additive noise, applied in a controlled manner to the observations. This choice allows for a gradual increase in problem complexity while preserving the basic structure of the clusters.

A general overview

The following tables report the ARI values obtained under different experimental settings, allowing a systematic comparison of how noise affects clustering quality. As in the previous analysis, Appendix 7.2 provides the corresponding results evaluated using the NMI metric, enabling a more comprehensive interpretation of the outcomes.

Cardinality and dimensionality of the dataset Similar to what was observed in the study on the correlation scaling factor, performance worsen as the number of observations increases and the number of informative variables decreases. In particular, scenarios with 10.000 observations and 5 variables represent the most critical case: in the presence of high levels of noise, the algorithms struggle to maintain separation between clusters, with ARI values often very low and in some cases close to zero. This behaviour is consistent with statistical theory: fewer significant variables imply less discriminating information, while a large number of observations amplifies the effects of noise, making it more difficult to recognize the ground truth structure.

Number of clusters A clear trend also emerges with respect to the number of clusters. Across all three dimensional configurations, increasing the number of clusters leads to a progressive decline in performance. As the number of clusters grows, the problem becomes intrinsically more complex: the number of possible assignment alternatives increases, and the separation between clusters decreases, particularly when observations are affected by high noise.

Observations and Features:	2500 observations and 15 features								
Number of Clusters:	3 clusters			5 clusters			7 clusters		
Noise:	5	7	9	5	7	9	5	7	9
k-means	0,962	0,897	0,785	0,646	0,795	0,475	0,723	0,626	0,492
PCA + k-means	0,96	0,413	0,778	0,881	0,551	0,466	0,693	0,587	0,465
t-SNE + k-means	0,95	0,802	0,637	0,643	0,517	0,416	0,715	0,468	0,327
UMAP + k-means	0,961	0,892	0,792	0,949	0,837	0,466	0,73	0,591	0,372
Birch	0,925	0,843	0,676	0,92	0,52	0,561	0,794	0,537	0,4
PCA + Birch	0,963	0,883	0,731	0,637	0,53	0,397	0,658	0,543	0,268
t-SNE + Birch	0,94	0,822	0,67	0,658	0,578	0,418	0,706	0,393	0,315
UMAP + Birch	0,961	0,892	0,791	0,862	0,662	0,551	0,862	0,699	0,554
GMM	0,976	0,928	0,441	0,655	0,679	0,521	0,656	0,504	0,267
PCA + GMM	0,972	0,901	0,56	0,92	0,48	0,473	0,633	0,605	0,444
t-SNE + GMM	0,959	0,82	0,478	0,699	0,557	0,411	0,886	0,472	0,341
UMAP + GMM	0,961	0,899	0,792	0,953	0,835	0,493	0,735	0,629	0,293
PAM	0,887	0,427	0,4	0,899	0,473	0,24	0,647	0,421	0,222
PCA + PAM	0,961	0,889	0,813	0,613	0,533	0,446	0,688	0,591	0,459
t-SNE + PAM	0,95	0,808	0,662	0,643	0,516	0,424	0,717	0,475	0,321
UMAP + PAM	0,961	0,894	0,793	0,95	0,559	0,609	0,729	0,613	0,482
Spectral Clustering	0,971	0,915	0,794	0,957	0,853	0,685	0,949	0,825	0,674
PCA + Spectral Clustering	0,972	0,909	0,789	0,92	0,806	0,62	0,705	0,593	0,457
t-SNE + Spectral Clustering	0,961	0,763	0,582	0,938	0,475	0,33	0,946	0,443	0,287
UMAP + Spectral Clustering	0,445	0,898	0,796	0,954	0,836	0,565	0,703	0,623	0,444

Figure 19: Results evaluated according to ARI in the case of 2500 observations and 15 variables and keeping correlation scaling factor constant at 0.5

In-depth analysis

In general, we observe that increasing noise in the dataset leads to a progressive deterioration in the performance of clustering algorithms. However, this effect is not uniformly valid for all algorithms and does not always manifest itself monotonically: the behaviour depends on the interaction between noise, the number of clusters, and the structure of the dataset. To better understand these dynamics, it is therefore necessary to analyse the behaviour of individual algorithms and evaluate how different dimensionality reduction techniques contribute to stabilizing or worsening the results.

k-means In the case with 2.500 observations and 15 variables, we generally observe a progressive drop in k-means performance as noise increases, with the only exception of 5 clusters and a noise level equal to 7, where an exceptionally high value is observed. This anomalous behaviour suggests that, under specific configurations, the cluster structure may remain sufficiently separable despite the presence of noise. The integration with dimensionality reduction algorithms shows that UMAP slightly mitigates the effects of noise, confirming its relative stability among the methods considered for preprocessing. PCA and t-SNE, on the other hand, do not exhibit the same level of robustness and tend to introduce additional variability. In the cases with 5.000 and

Observations and Features:	5000 observations and 10 features								
Number of Clusters:	3 clusters			5 clusters			7 clusters		
Noise:	5	7	9	5	7	9	5	7	9
k-means	0,547	0,472	0,378	0,575	0,344	0,284	0,544	0,364	0,239
PCA + k-means	0,533	0,456	0,367	0,389	0,278	0,267	0,36	0,269	0,182
t-SNE + k-means	0,536	0,434	0,28	0,522	0,322	0,253	0,458	0,254	0,177
UMAP + k-means	0,549	0,446	0,355	0,622	0,342	0,334	0,473	0,307	0,198
Birch	0,618	0,433	0,244	0,401	0,291	0,261	0,447	0,239	0,214
PCA + Birch	0,397	0,306	0,285	0,352	0,276	0,24	0,331	0,242	0,169
t-SNE + Birch	0,52	0,304	0,222	0,403	0,361	0,157	0,359	0,207	0,179
UMAP + Birch	0,549	0,439	0,345	0,449	0,289	0,301	0,527	0,388	0,237
GMM	0,683	0,416	0,287	0,49	0,415	0,341	0,528	0,267	0,166
PCA + GMM	0,491	0,345	0,209	0,392	0,283	0,183	0,39	0,261	0,194
t-SNE + GMM	0,44	0,377	0,253	0,536	0,291	0,238	0,421	0,263	0,157
UMAP + GMM	0,494	0,371	0,251	0,511	0,305	0,231	0,47	0,259	0,237
PAM	0,396	0,393	0,298	0,389	0,305	0,225	0,479	0,289	0,177
PCA + PAM	0,544	0,457	0,368	0,328	0,261	0,261	0,345	0,233	0,17
t-SNE + PAM	0,537	0,433	0,291	0,399	0,324	0,252	0,45	0,235	0,18
UMAP + PAM	0,573	0,438	0,337	0,613	0,461	0,263	0,584	0,324	0,203
Spectral Clustering	0,56	0,446	0,354	0,61	0,449	0,317	0,647	0,444	0,247
PCA + Spectral Clustering	0,54	0,448	0,367	0,325	0,273	0,248	0,327	0,215	0,161
t-SNE + Spectral Clustering	0,547	0,371	0,33	0,348	0,279	0,217	0,454	0,221	0,152
UMAP + Spectral Clustering	0,558	0,397	0,323	0,632	0,438	0,278	0,473	0,442	0,233

Figure 20: Results evaluated according to ARI in the case of 5000 observations and 10 variables and keeping correlation scaling factor constant at 0.5

Observations and Features:	10000 observations and 5 features								
Number of Clusters:	3 clusters			5 clusters			7 clusters		
Noise:	5	7	9	5	7	9	5	7	9
k-means	0,393	0,299	0,218	0,31	0,209	0,145	0,278	0,186	0,122
PCA + k-means	0,407	0,272	0,219	0,306	0,208	0,143	0,214	0,145	0,106
t-SNE + k-means	0,411	0,251	0,187	0,284	0,189	0,116	0,259	0,178	0,121
UMAP + k-means	0,5	0,304	0,186	0,34	0,226	0,154	0,301	0,204	0,124
Birch	0,33	0,287	0,184	0,208	0,132	0,098	0,293	0,169	0,095
PCA + Birch	0,245	0,255	0,181	0,292	0,186	0,107	0,28	0,204	0,127
t-SNE + Birch	0,329	0,078	0,144	0,23	0,191	0,133	0,212	0,169	0,133
UMAP + Birch	0,344	0,252	0,199	0,322	0,192	0,132	0,303	0,225	0,167
GMM	0,361	0,264	0,179	0,224	0,187	0,103	0,231	0,082	0,096
PCA + GMM	0,322	0,214	0,1	0,159	0,148	0,102	0,246	0,207	0,115
t-SNE + GMM	0,34	0,222	0,075	0,253	0,153	0,108	0,219	0,185	0,106
UMAP + GMM	0,501	0,27	0,115	0,301	0,193	0,159	0,299	0,196	0,127
PAM	0,438	0,349	0,229	0,314	0,2	0,144	0,306	0,188	0,116
PCA + PAM	0,446	0,341	0,217	0,312	0,204	0,134	0,215	0,153	0,112
t-SNE + PAM	0,417	0,251	0,186	0,284	0,192	0,118	0,266	0,18	0,113
UMAP + PAM	0,494	0,3	0,213	0,341	0,226	0,163	0,293	0,211	0,121
Spectral Clustering	0,464	0,31	0,221	0,321	0,204	0,137	0,302	0,194	0,129
PCA + Spectral Clustering	0,423	0,293	0,215	0,311	0,196	0,154	0,191	0,142	0,106
t-SNE + Spectral Clustering	0,336	0	0	0,334	0,167	0,145	0,267	0,195	0,129
UMAP + Spectral Clustering	0,554	0,313	0,198	0,334	0,232	0,138	0,275	0,196	0,129

Figure 21: Results evaluated according to ARI in the case of 5000 observations and 10 variables, keeping correlation scaling factor constant at 0.5

10.000 observations, the overall dynamics are similar: k-means maintains acceptable performance across noise levels and benefits primarily from combination with UMAP, while t-SNE and PCA do not tend to improve the already strong performance of k-means. In some cases, they lead to a significant drop, as in the case of 3 clusters and 7 noise, where the combination of PCA and k-means fails.

Birch Birch shows solid performance especially in the case with 2.500 observations and 15 variables, with better results when preceded by UMAP, which stabilizes its behaviour. Also PCA and t-SNE performs well, but tend to worsen the already optimal

results of Birch when noise is high. In the case with 5.000 observations and 10 variables, the combination with PCA is the least effective, while UMAP continues to provide the best support for the model, improving the robustness and stability of the clustering. The same behaviour is observed in the case with 10.000 observations and 5 variables, where Birch shows increasing difficulty and UMAP remains the best ally, while PCA and t-SNE present more marked difficulties than the previous datasets.

GMM In the dataset with 2.500 observations, GMM achieves very high performance, but struggles when the noise increases, particularly in the cases with 3 and 7 clusters. In this context, PCA and t-SNE perform well. However, the GMM in combination with UMAP proves to be the most effective: not only significantly improve performance, but it also makes the trend more regular and less volatile as the noise increases. Only in rare cases does PCA perform better, for example, 7 clusters and noise equal to 9. Moving to 5.000 observations, the same dynamic occurs again, although in this case it is precisely the PCA that presents the greatest difficulties. Even with 10.000 observations and 5 variables, the behaviour remains consistent, and UMAP continues to represent the best support for GMM especially in the case with 3 clusters

PAM PAM shows difficulties starting with the case of 2.500 observations and 15 variables, proving more sensitive to noise than k-means. In this context, dimensionality reduction techniques play a fundamental role since they help stabilize performance, bringing it to values on average comparable to other algorithms. More precisely, the same pattern found previously is repeated, i.e. the greater stability brought about by UMAP. With 5.000 observations, the pattern repeats, with UMAP once again representing the best solution. In the case of 10.000 observations, PAM tends to stabilize, obtaining results in line with or sometimes superior to the other algorithms, reducing the need for particularly intensive preprocessing.

Spectral Clustering Spectral Clustering remains the most stable and performing algorithm on average, particularly with 2.500 observations, where it maintains high

values even in conditions of high noise. In many cases, dimensionality reduction techniques do not lead to further improvements, especially t-SNE. UMAP shows some isolated difficulties in this scenario, for example in the case with 3 clusters and low noise, but overall it maintains consistent behaviour. With 5.000 observations, Spectral Clustering slightly aligns with the other algorithms, while maintaining superior performance: PCA and t-SNE tend to reduce the output values, while UMAP again represents the best choice, without significantly improving the algorithm's already good performance. In the case with 10.000 observations and 5 variables, the difficulty of t-SNE emerges markedly, producing values close to zero, as also highlighted by the NMI in Figure 40. Of all the algorithms, Spectral Clustering remains one of the most stable and least dependent on additional preprocessing.

Dimensionality reduction algorithms Overall, UMAP emerges as the most effective and robust dimensionality reduction technique, capable of improving or stabilizing the performance of most algorithms, especially in the presence of high noise. PCA shows greater variability than correlation, proving effective in some contexts but less stable in others. t-SNE, consistent with the observations in the previous analysis, remains the least reliable tool for clustering: while valid for exploratory and visualization purposes, it often introduces instability, especially in conditions of high noise or a large number of clusters.

Comparative algorithm summary

From the analysis we can see that increasing noise leads to a progressive drop in clustering results, but with different effects depending on the algorithm. Spectral Clustering appears to be the most robust method on average, maintaining good performance even in highly noisy scenarios. GMM is effective as long as the noise remains moderate, but becomes more unstable as the overlap between components increases. Among partition-based algorithms, k-means maintains reasonable results, while PAM appears more vulnerable, especially in particularly high noise conditions and requires

the help of dimensionality reduction algorithms. Birch occupies an intermediate position, proving valid when noise is low but less reliable in more complex scenarios. Dimensionality reduction techniques play a key role: UMAP remains the most stable and useful solution for mitigating the effects of noise, while PCA and t-SNE represent good solutions but are subject to more variations and are generally less efficient than UMAP.

5.4 High-Dimensional Case Study

We now conclude our analyses by considering an extreme case, in order to evaluate the actual effectiveness of dimensionality reduction algorithms. As shown in the previous experiments, an increasing number of clusters, higher correlation among variables, and stronger noise levels tend to negatively affect clustering performance, effects that can often be mitigated through the use of appropriate preprocessing techniques. To this end, we construct a dataset similar to the one presented in Figure 14, consisting of 2,500 observations, while increasing the number of variables from 15 to 100 and thereby introducing a markedly higher level of structural complexity. We focus exclusively on the most critical configuration, namely the one with 7 clusters, a high noise level of 9, and a correlation scaling factor of 0.9. On this dataset, we apply the same experimental procedure adopted in the previous analyses and evaluate clustering performance using ARI, NMI, and the Silhouette score, in order to assess not only the agreement with the reference labels but also the geometric structure of the resulting clusters. The results of the analysis are reported in Figure 22.

Interpretations The results obtained in this high-dimensional scenario clearly highlight the crucial role played by dimensionality reduction algorithms in supporting clustering techniques. Without preprocessing, many algorithms exhibit limited performance: k-means, GMM, and PAM exhibit moderate or low ARI and NMI values, accompanied by Silhouette scores close to zero, indicating a poorly compact and highly overlapping geometric structure of the clusters. Even Birch, while showing slightly

better results in terms of alignment with reference labels, suffers from poor geometric separation in the original data space. These behaviours are consistent with the expected effects of high dimensionality, strong correlation, and high noise, which degrade discriminative information and make Euclidean distances less significant. The introduction of dimensionality reduction techniques radically changes this picture. PCA provides generally moderate but consistent improvements, particularly for GMM and PAM, suggesting that removing collinearity and noise helps to stabilize model estimation and improve cluster separability. However, PCA is not sufficient to fully recover cluster structure in the most complex cases, as demonstrated by the only partially

Algorithms	ARI	NMI	Silhouette Score
k-means	0,669	0,835	0,07
PCA + k-means	0,676	0,837	0,257
t-SNE + k-means	0,999	0,998	0,758
UMAP + k-means	0,999	0,998	0,919
Birch	0,811	0,908	0,093
PCA + Birch	0,762	0,906	0,293
t-SNE + Birch	0,766	0,905	0,638
UMAP + Birch	0,999	0,998	0,919
GMM	0,557	0,664	0,012
PCA + GMM	0,908	0,933	0,266
t-SNE + GMM	0,999	0,998	0,758
UMAP + GMM	0,999	0,998	0,919
PAM	0,528	0,636	0,059
PCA + PAM	0,745	0,868	0,295
t-SNE + PAM	0,76	0,904	0,65
UMAP + PAM	0,999	0,998	0,919
Spectral Clustering	1	1	0,101
PCA + Spectral Clustering	1	1	0,349
t-SNE + Spectral Clustering	0,488	0,757	0,209
UMAP + Spectral Clustering	0,318	0,647	0,217

Figure 22: Results evaluated according to ARI, NMI and Silhouette Score for a dataset composed of 2.500 observations, 100 variables, 7 clusters, correlation scaling factor α equal to 0.9 and noise σ equal to 9.

improved Silhouette values. A much more pronounced effect emerges with the use of nonlinear techniques, particularly t-SNE and UMAP. When combined with various clustering algorithms, both lead to ARI and NMI values close to unity, indicating a near-perfect match with the reference labels. This result suggests that these techniques are able to project the data into a low-dimensional space in which the cluster structure is highly separable. However, the Silhouette score analysis highlights substantial differences between the two approaches: while t-SNE produces well-defined clusters but with a less compact geometry, UMAP consistently delivers very high Silhouette scores, indicating clusters that are not only correctly separated but also highly geometrically cohesive. This behaviour confirms the greater stability and reliability of UMAP as a preprocessing tool for clustering in complex contexts. A particularly interesting case is Spectral Clustering, which shows perfect performance in the original data space in terms of ARI and NMI, but with a very low Silhouette score. This is expected, since the Silhouette score is computed in the original Euclidean space, while Spectral Clustering relies on graph-based similarities that may not correspond to compact Euclidean clusters. The application of PCA partially improves this aspect, while the use of t-SNE and UMAP significantly compromises performance, highlighting the high sensitivity of the spectral method to nonlinear transformations of the distance space.

5.5 Final Conclusions

Overall, the algorithms achieved higher performance in settings characterized by a higher feature dimensionality and a smaller sample size, while performance deteriorated when the number of observations increased and the number of informative variables decreased. This behaviour may be imputed to the reduction in discriminant information, which makes cluster separation more challenging, while a large number of observations enhance the effect of noise and overlap. Finally, it was confirmed that increasing the number of clusters naturally makes the problem more complex by expanding the number of possible assignments and consequently increasing the probability of misclassification. This effect was particularly evident for algorithms such

as GMM and PAM, whereas Spectral Clustering showed greater stability. Moreover, the results confirm that dimensionality reduction can significantly influence clustering outcomes. In this sense, nonlinear approaches such as UMAP proved particularly effective in preserving meaningful structure under challenging conditions.

On correlation scaling factor The analysis on the correlation scaling factor highlighted how a high correlation between variables generally led to a drop in clustering performance. Collinearity introduces information redundancy, alters the geometry of the distance space, and generates anisotropic structures, making cluster separation more complex, especially for Euclidean distance-based algorithms such as k-means and PAM. However, it has been shown that this effect is not always monotonic and may depend on the interaction with other factors, such as dimensionality and number of clusters. In this context, PCA and UMAP have proven particularly effective in mitigating the effects of correlation, especially in more complex configurations.

On noise The study on the effect of noise confirms a clear and consistent trend: as the level of noise in the data increases, clustering performance progressively declines on average. Noise increases the dispersion within clusters, reduces their compactness, widens the areas of overlap, and makes decision boundaries less clear. The impact of noise, however, is not uniform across algorithms. Spectral Clustering proves to be the most resilient approach overall, maintaining comparatively stable performance even under high noise levels. In contrast, GMM, k-means, and above all PAM are more strongly affected by the progressive loss of informative structure. Within this context, UMAP continues to stand out as the most effective and stable dimensionality reduction technique, consistently supporting clustering performance under noisy conditions. PCA and t-SNE, on the other hand, do not bring great improvements to the clustering algorithms, often resulting in slightly worsening the results.

Practical implications From an application perspective, the results show that the choice of clustering algorithm cannot be based only on computational criteria or standard preferences, but must be guided by an informed knowledge of the data structure. In relatively simple scenarios, characterized by low correlation, moderate noise, and a limited number of clusters, algorithms such as k-means, PAM, and GMM represent efficient and reliable solutions, especially when supported by appropriate preprocessing techniques like UMAP. As structural complexity increases, more robust methods such as Spectral Clustering become preferable. Dimensionality reduction techniques play a crucial role: as said before UMAP is a reliable and often crucial tool in improving the quality of partitions, while the use of PCA and t-SNE as a preprocessing tools must be evaluated more accurately. Finally, it is worth noting that, in terms of computational efficiency, PAM and Spectral Clustering tended to be the slowest algorithms, especially in the setting with 10.000 observations and 5 variables.

High-dimensional case Overall, these results confirm that, in the presence of high dimensionality, noise, and strong correlation, dimensionality reduction can significantly improve clustering performance, although its effectiveness strongly depends on both the technique employed and the clustering algorithm considered. Consistently with the previous analyses, UMAP emerges as the most robust and consistent solution, providing stable and significant improvements across several methods, including k-means, GMM, PAM, and Birch. PCA yields more moderate and conservative benefits, particularly for model-based and distance-based approaches affected by collinearity, while t-SNE, despite its strong ability to enhance cluster separability, shows limitations in terms of stability and reliability when used beyond exploratory purposes. Finally, the observed behaviour of Spectral Clustering highlights that not all algorithms benefit equally from dimensionality reduction, emphasizing the importance of jointly selecting preprocessing and clustering strategies in complex data settings.

Chapter 6

An Application on a Real Dataset

This chapter presents the analysis conducted on a real dataset relating to the energy consumption of US households.

6.1 Introduction

The dimensionality reduction and clustering techniques considered so far are now applied to a real-world dataset related to household energy consumption in the United States. Unlike the synthetic case, this dataset does not provide a known ground truth, that is, a unique label associated with each observation. Therefore, cluster quality must be assessed exclusively through internal validation measures, such as the Silhouette score, the Davies-Bouldin index, and the Calinski-Harabasz index. These metrics evaluate clustering performance based on intrinsic geometric properties of the data, including within cluster variance, inter cluster separation and overall structural coherence. In this setting, a direct comparison between clustering results obtained under different preprocessing strategies is not always meaningful. More specifically, comparisons across algorithmic combinations remain useful when using Principal Component Analysis (PCA), since PCA is a linear transformation that preserves the global metric structure of the data while maximizing the explained variance. By contrast, nonlinear dimensionality reduction techniques such as t-SNE and UMAP are designed to preserve local neighborhood relationships and, in doing so, distort global distances and cluster geometry. Consequently, internal validation indices computed in these

embedding spaces strongly depend on the structure caused by the embedding itself rather than on intrinsic properties of the original data. For this reason, comparisons of clustering outcomes across different nonlinear embeddings, or between nonlinear embeddings and the original feature space, should be interpreted with care, as they are not directly comparable in a metric sense.

The dataset The dataset used in this analysis is drawn from the 2020 Residential Energy Consumption Survey (RECS), a national study conducted by the U.S. Energy Information Administration (EIA) aimed at collecting detailed information on energy consumption in U.S. primary residences. The 2020 edition represents the fifteenth release of the survey and integrates information on structural characteristics of homes, energy-related behaviours, actual energy consumption, and associated financial expenditures, for a total of 799 variables. The study is based on a representative sample of approximately 18,500 households, designed to statistically represent about 123.5 million primary residences across the United States. The selection of this dataset is motivated by a strong interest in the analysis of energy consumption patterns, combined with the availability of a large and well-structured dataset in terms of both variables and observations.

6.2 Pre-processing Analysis

The primary goal of this analysis is to investigate energy consumption patterns among U.S. households, identify recurring structures that can be summarized through clustering, and conduct further analyses to better understand the differences among households. In parallel, this study aims to assess whether the findings of the previous chapter remain consistent when applied to real-world data, which are characterized by more complex and less controlled structures than synthetically generated datasets. However, as discussed in the introduction, direct comparisons between clustering algorithms are not always feasible, since the interpretation of the results necessarily relies on internal validation metrics. For this reason, a more qualitative evaluation of the resulting

cluster structures must also be carried out. Before applying the clustering algorithms, the data face a multi-stage preprocessing procedure, aimed at ensuring consistency, comparability, and quality of the analysed variables. The overall workflow can be summarized in the following diagram presented in Figure 23, which summarizes the main steps from data acquisition to final standardization.

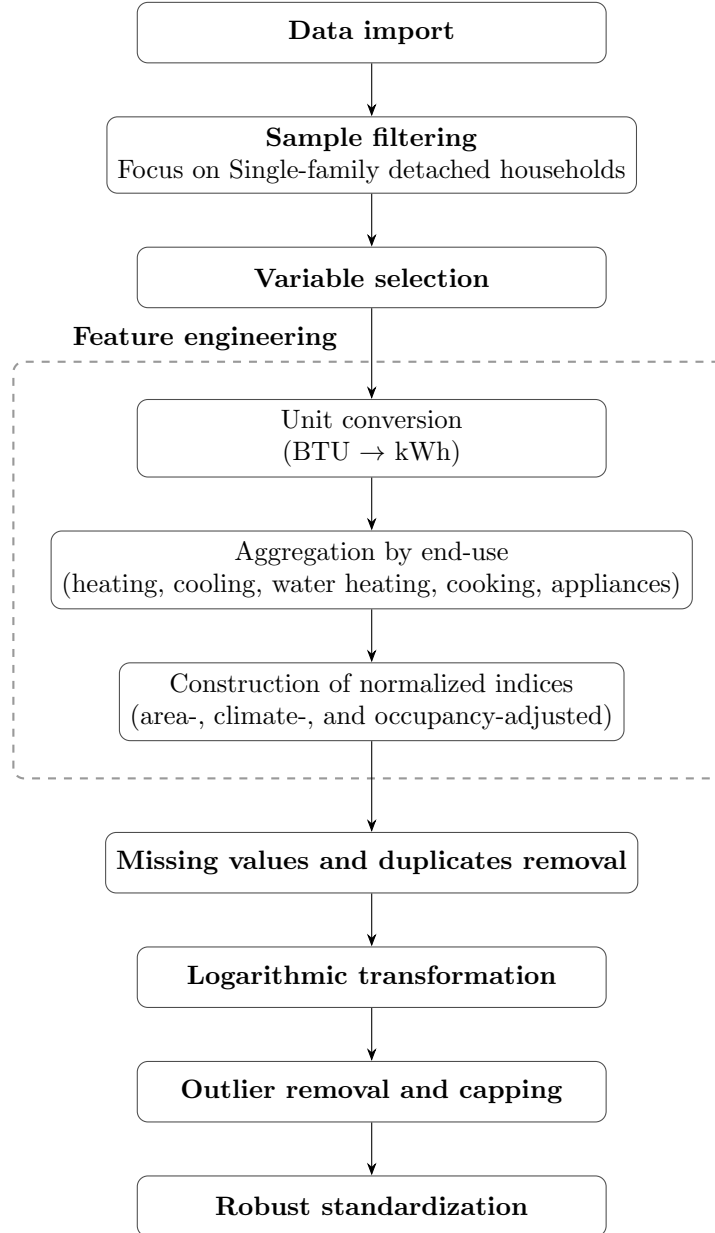


Figure 23: Preprocessing and feature engineering workflow prior to clustering

Sample filtering Once data is imported we can start to process and study them. Preliminary analyses shown in Figure 24 reveal a clear prevalence of the single-family detached housing type. Since this category is significantly more represented than the others, the analysis is restricted exclusively to this housing type. This choice reduces structural heterogeneity within the sample and ensures a more homogeneous and fair comparison between statistical units, allowing the clustering results to better reflect differences in energy use patterns rather than structural characteristics of the residential units. As a result of this filtering procedure, the dataset consists of 12.319 observations.

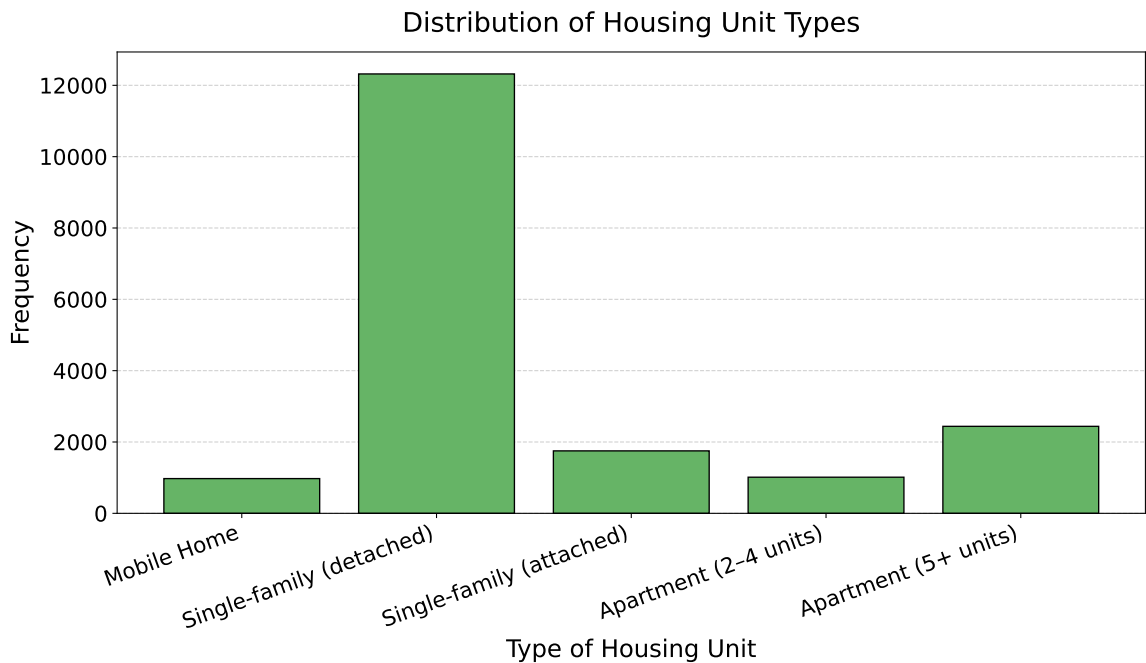


Figure 24: Distribution of housing types in the dataset

Variable selection The RECS 2020 dataset includes a total of 799 variables describing residential energy consumption in the United States. These features cover a wide range of topics, including detailed information on energy use, household and building characteristics, geographic and climatic conditions and billing information. For clustering reasons, a targeted variable selection and transformation procedure was carried

out. First, the analysis was restricted to quantitative variables measuring calibrated annual energy consumption. Secondly, the retained variables describing energy use and expressed in thousands of BTU¹ were converted into equivalent kilowatt-hours.² to ensure full comparability across energy sources. Finally, variables referring to the same household energy function but originating from different fuels were aggregated, allowing the creation of five synthetic variables, each capturing the total annual energy consumption associated with a specific functional category:

- Space Heating: includes all energy consumption associated with main and secondary space heating systems, combining electricity, natural gas, propane, and fuel oil/kerosene used for heating purposes, as well as electricity used by auxiliary components such as furnace fans and boiler pumps,
- Cooling: aggregates electricity consumption related to space cooling, including central air conditioning systems, individual cooling units, evaporative coolers, and associated ventilation components,
- Water Heating: obtained by combining electricity, natural gas, propane, and fuel oil/kerosene consumption used for domestic water heating, both for primary and secondary systems,
- Cooking: aggregates energy consumption associated with food preparation activities, including electricity and gas usage for cook-tops, ovens, ranges, and microwave appliances,
- Appliances: constructed by aggregating electricity and fuel consumption related to refrigerators, freezers, dishwashers, clothes washers and dryers, televisions, and other household equipment.

¹The *BTU* (British Thermal Unit) is a unit of thermal energy defined as the amount of heat required to raise the temperature of one pound of water by one degree Fahrenheit. For energy conversion, the relationship $1 \text{ BTU} = 0,000293071 \text{ kWh}$ holds, implying that $1,000 \text{ BTU} = 1 \text{ kBTU} = 0,293071 \text{ kWh}$. In practice, conversion from thousands of BTU to kilowatt-hours is obtained through the formula $\text{kWh} = \text{kBTU} \times 0,293$.

²The *kWh* (kilowatt-hour) is a unit of energy measurement and represents the energy consumed or produced by a power of one kilowatt sustained for one hour.

Each synthetic indicator reflects the overall energy requirement of a specific end use, independently of the fuel mix used to satisfy it, shifting the focus of the analysis from energy sources to household energy-use profiles. All aggregated consumption variables were then normalized according to their underlying nature. Heating and cooling consumption were normalized by both the energy-consuming floor area and climate severity, using Heating Degree Days and Cooling Degree Days, accounting for differences in house size and climate. These indicators measure the cumulative deviation of outdoor temperatures from a reference threshold of 65°F, corresponding to 18°C, and provide a standard proxy for heating and cooling consumption driven by local climatic conditions. By contrast, consumption associated with water heating, cooking, and household appliances was normalized by the number of household members, reflecting the fact that these end uses are primarily driven by occupancy-related behaviour rather than building characteristics. To ensure numerical stability during the normalization process, a small constant was added to the denominators to avoid divisions by zero. Subsequently, zero-valued observations were replaced with the 0.01 quantile of the corresponding distributions. Overall, this normalization strategy enhances comparability across observations, reduces the dominant influence of structural factors on absolute consumption levels, and enables clustering algorithms to emphasize differences in usage behaviour and relative efficiency rather than scale effects.

Final steps As we proceed with the analyses, once we have discovered that the dataset does not contain missing values, duplicates, or rows composed exclusively of zeros, we proceed with the logarithmic transformation. This step is essential to reduce the impact of observations significantly distant from the sample mean and to make the distribution of variables more symmetric. In particular, the logarithmic transformation improves the suitability of the data for distance-based clustering algorithms, such as k-means and PAM, by stabilizing variances and reducing the dominance of outlier values. Moreover, it improves the applicability of probabilistic models such as Gaussian Mixture Models, where reduced skewness and more regular feature distributions

support more reliable parameter estimation. Once the logarithmic transformation is complete, we proceed with managing outliers. A careful approach is necessary at this stage, as indiscriminately deleting extreme elements could result in the loss of informative families relevant for identifying specific groups. However, controlling for such observations is essential for clustering algorithms to operate properly. A significant choice is to cap the outliers, namely to limit the influence of extreme observations by replacing values beyond a specified threshold with a fixed quantile. Through this procedure, the most extreme observations are assigned the value corresponding to the 99th quantile, resulting in a more symmetric and consistent distribution, as can be seen in Figure 25. It is worth noting that, even after capping, the distribution retains a pronounced right tail due to the large number of extreme observations. However, this approach avoids the loss of data points that could be crucial for the formation of meaningful clusters.

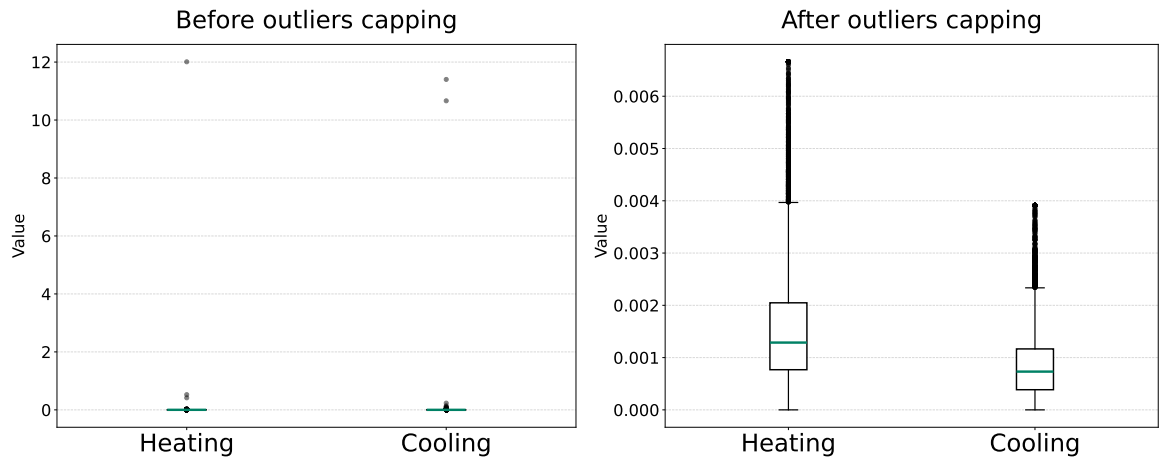


Figure 25: Distribution of the Heating and Cooling variables before and after the application of the outlier capping procedure (quantile 0.99)

Proceeding with the remaining three variables, which are more closely related to consumption and analysed separately because they have a different scale than those related to home heating and cooling, a marked asymmetry of the distribution toward the left tail can be observed. In particular, a significant portion of observations with a value of zero is highlighted, as shown in Figure 26. Specifically, there are 67

observations with zero hot water consumption and 27 observations with zero energy consumption related to household appliances. These homes are excluded from the dataset, as it is plausible that they are abandoned homes, second homes for occasional use, or measurement errors, elements that could introduce significant bias in subsequent analyses. After that, the left tail is clipped at the 0.01 quantile, in order to make any extremely low values more consistent with the rest of the distribution.

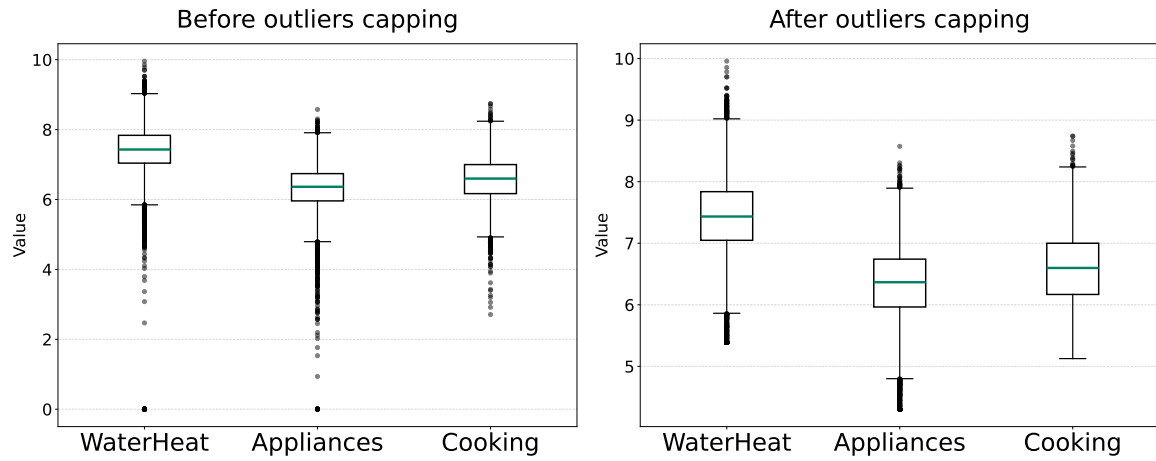


Figure 26: Distribution of the Water Heating, Appliances and Cooking variables before and after the application of the outlier capping procedure

Finally, the data are standardized using a robust scaling approach based on the 25th and 75th quantiles. Unlike standard normalization techniques based on the mean and standard deviation, robust scaling relies on the interquartile range, making it less sensitive to residual extreme values and distributional asymmetries that may persist after previous preprocessing steps. As a result, this transformation preserves the relative structure of the data while preventing a small number of observations from disproportionately influencing distance computations. This step represents the final preprocessing phase and is aimed at ensuring proper comparability among variables, as well as an adequate definition of the distance measures employed by the clustering algorithms. The final distributions of the standardized variables are shown in Figure 27. In this case, the distributions appear centred around zero and approximately symmetric, although a higher concentration of observations is still present in the distribution

tails—particularly on the right tail for Heating and Cooling, and on the left tail for Water Heating and Appliances. This behaviour reflects the intrinsic heterogeneity of the underlying consumption patterns rather than preprocessing artifacts.

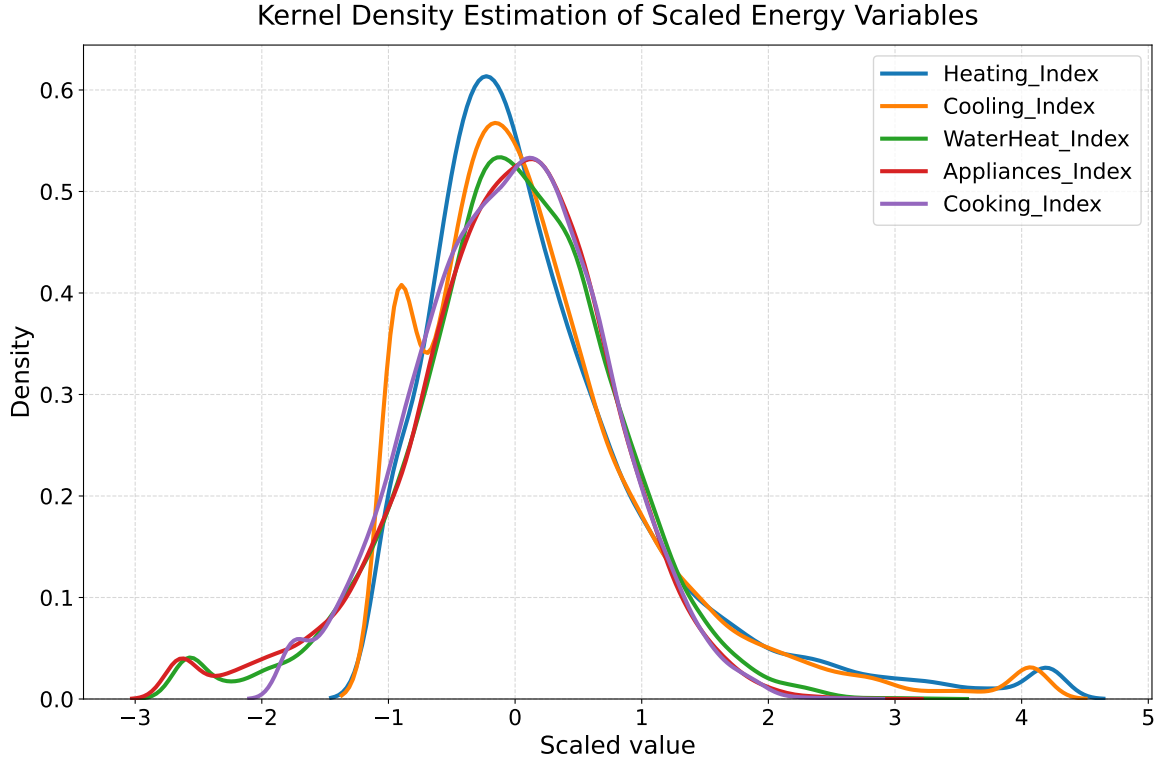


Figure 27: Final distributions of the variables after all preprocessing steps

Speaking of correlation, we can see in Figure 28 that the variables exhibit generally low pairwise correlations, which limits information redundancy and ensures that each variable contributes distinct information to the clustering process. Only the variables relating to energy consumption for household appliances and cooking are moderately correlated, with a coefficient equal to 0.49; however, this level of correlation is to be considered acceptable and not excessively invasive for the purposes of the clustering analysis. Having completed all preprocessing steps, the dataset is suitably prepared for the subsequent clustering analysis, enabling the application of a broad range of clustering algorithms under consistent and comparable conditions.

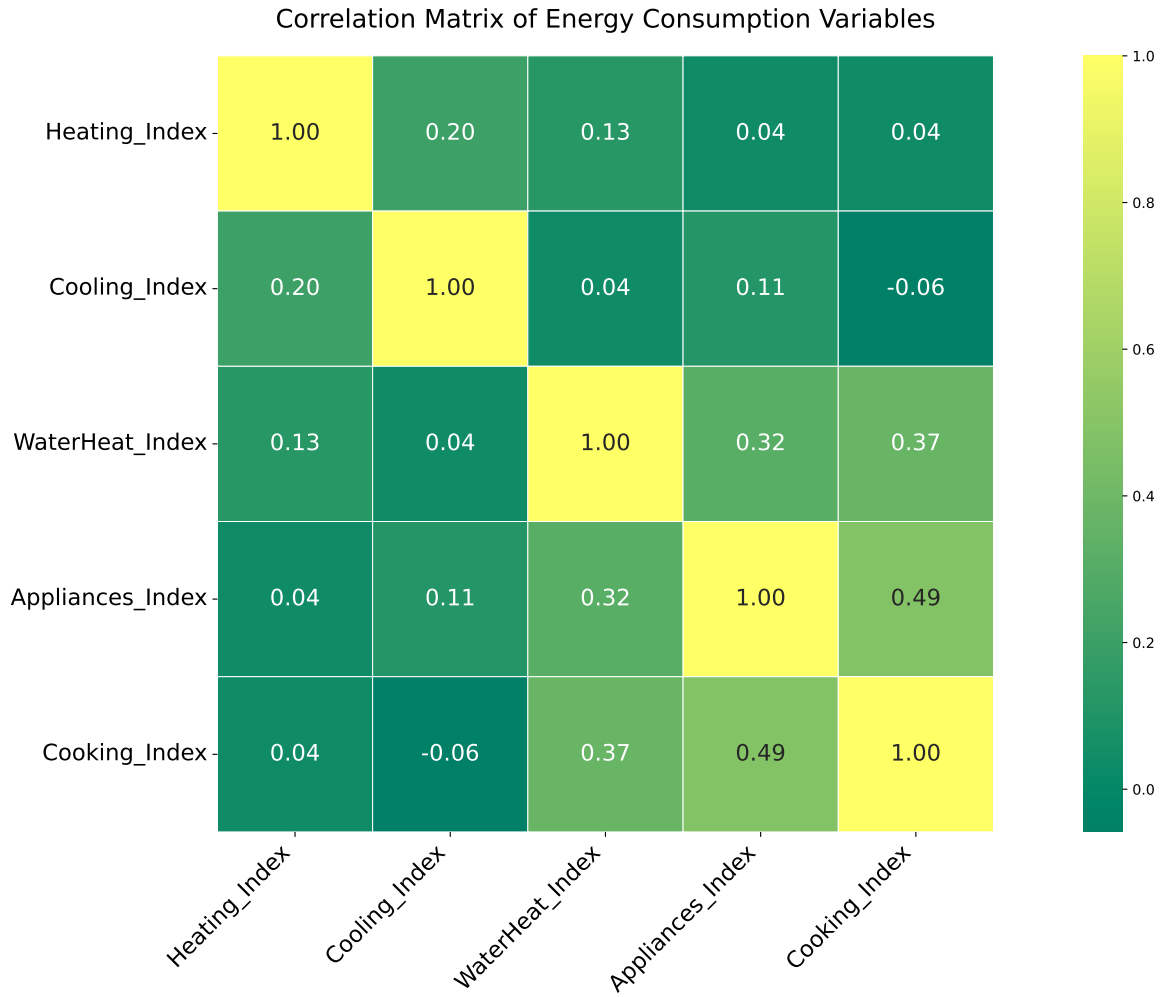


Figure 28: Correlation matrix of the five variables considered in this study.

Comparison with theoretical results At the end of this procedure, the resulting dataset consists of 12.228 observations and 5 variables, a configuration that closely mirrors the theoretical scenario analysed in the previous chapter, which considered a synthetic dataset of 10.000 observations and 5 variables. This setting is the one that exhibited the least benefit from the application of dimensionality reduction techniques, mainly due to the limited number of original variables; nevertheless, several relevant insights were still identified. The analysis of the results obtained by varying the correlation scaling factor, as reported in Figure 16, revealed generally low ARI values, particularly when t-SNE was employed as a preprocessing method. Overall, clustering

algorithms applied directly to the data or preceded by dimensionality reduction techniques showed comparable performance, with only marginal improvements observed, mainly when UMAP was used as a preprocessing step. Conversely, when variations in the noise level were considered, as shown in Figure 21, the differences induced by the preprocessing techniques became more pronounced. In this context, k-means, PAM, and Spectral Clustering emerged as the best-performing algorithms, especially when combined with UMAP. Therefore, it is reasonable to expect good overall performance also when considering a dataset drawn from a real-world context. It should be noted, however, that in this case the final evaluation metrics are not directly comparable. For this reason, and in light of the theoretical results highlighting the effectiveness of UMAP, the subsequent analysis will focus in particular on clustering solutions obtained by combining UMAP with a clustering algorithm.

6.3 Clustering and Results

We now move on to the clustering analyses. As discussed in the previous chapter, the procedure remains unchanged: the clustering algorithms are initially applied to the dataset without preprocessing and then to the dataset subjected to dimensionality reduction. To this end, we begin by describing the dimensionality reduction algorithms and how they were implemented in the analyses.

Dimensionality reduction algorithms We begin with Principal Component Analysis (PCA), for which the number of components is determined based on the Kaiser Criterion. Specifically, components with eigenvalues greater than one are selected, while requiring that at least 77.38% of the total variance be explained. This threshold derives from the relationship reported in Equation 6.3.1, which shows that the required cumulative variance is proportional to the number of variables considered p , equal to 5 in this case.

$$0.95^p \times 100 = 0.95^5 \times 100 = 77.38\% \quad (6.3.1)$$

Based on this criterion, three principal components are selected, which together explain approximately 80% of the total variance. For t-SNE and UMAP, we chose to reduce the dataset to two dimensions. This choice is motivated by the small number of original variables and, consequently, the reduced degree of complexity of the required dimensional reduction.

The number of clusters We now move on to selecting the optimal number of clusters, following the procedure introduced in Section 3.3 of chapter 3, based on the combined use of the Elbow Method and the Silhouette Score. The Elbow Method is based on the analysis of the inertia trend, which is the sum of the squared distances between each observation and the centroid of the cluster to which it belongs. This quantity represents the objective function minimized by k-means and provides a measure of the compactness of the clusters: lower inertia values indicate more compact clusters. However, since inertia decreases monotonically as the number of clusters increases, the Elbow Method criterion consists in identifying an inflection point, the so-called “elbow”, beyond which the addition of further clusters produces marginal benefits. The Silhouette Score provides a complementary measure to inertia, simultaneously evaluating the internal compactness of the clusters and their mutual separation. Higher values indicate a better assignment of observations, with well-separated clusters. Since clustering algorithms are applied both to the original space and to spaces transformed using dimensionality reduction techniques, it is essential to evaluate the number of clusters as a function of the representation space considered. Starting from the unreduced space, the graph reported in Figure 29 shows a comparison with the case of k-means applied to the reduced space via PCA. In both cases, the presence of a well-defined elbow is not particularly evident; however, a qualitative analysis of the inertia trend suggests a value of $k = 4$ as a plausible choice. This indication is further confirmed by the results reported in Table 3, where the highest Silhouette Score values correspond precisely to the case of four clusters, supporting the selection of this configuration as a reasonable trade-off between model complexity and cluster quality.

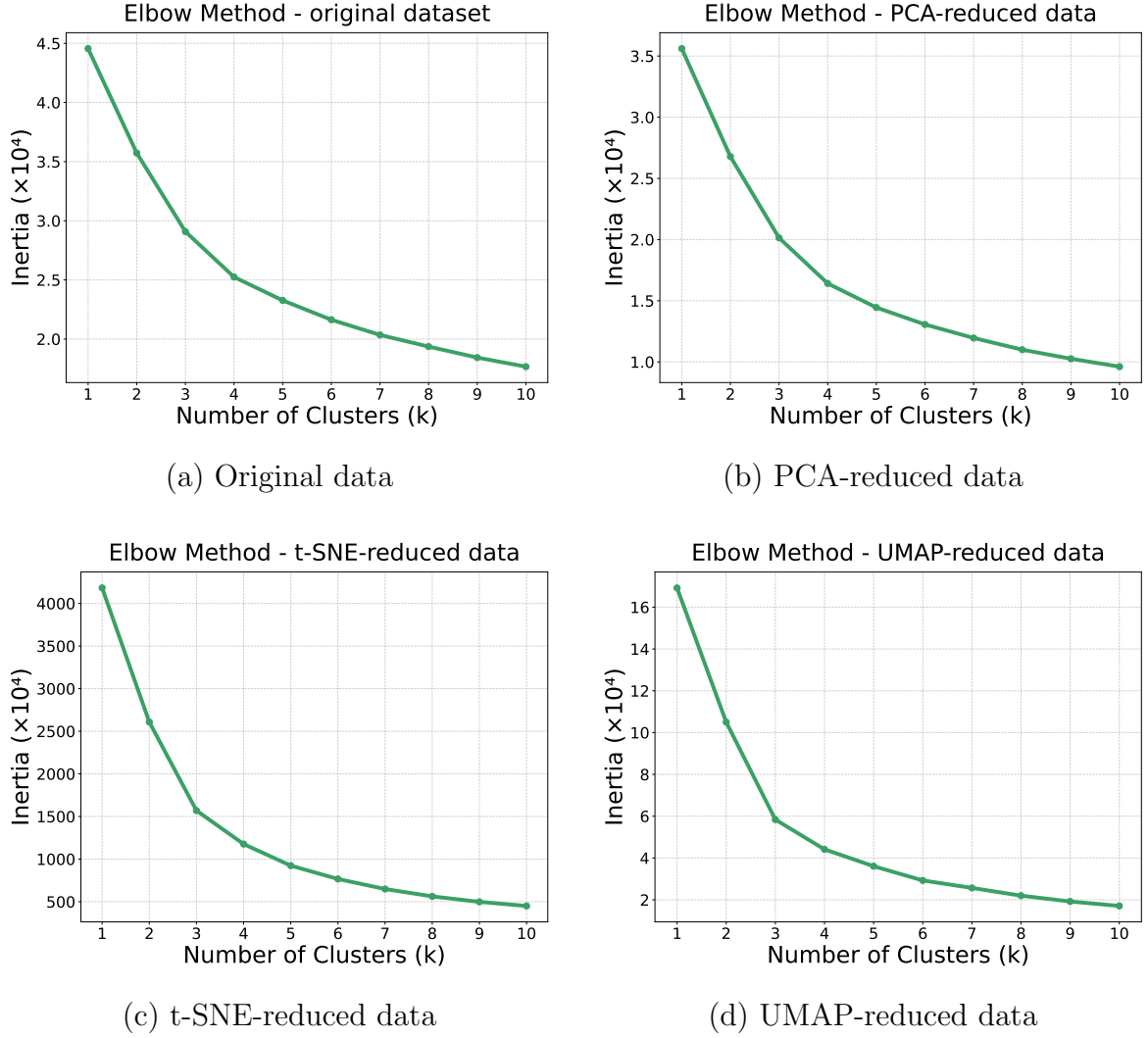


Figure 29: Elbow Method for k-means applied to different data representations: original feature space, PCA-reduced data, t-SNE-reduced data, and UMAP-reduced data

Number of clusters	Original data	PCA	t-SNE	UMAP
k = 2	0.187133	0.243000	0.351060	0.338353
k = 3	0.204440	0.274986	0.389815	0.418112
k = 4	0.220221	0.296382	0.358586	0.386613
k = 5	0.159879	0.236085	0.358471	0.360009
k = 6	0.156234	0.236673	0.382026	0.348928

Table 3: Silhouette scores for different numbers of clusters using various preprocessing methods

In the case of PCA, the four-cluster configuration also has the lowest inertia value compared to the other solutions considered, suggesting a good compromise between compactness and model complexity. The behaviour observed in the reduced spaces using t-SNE and UMAP, both with two components, is different. In these cases, the Elbow Method graph shows a much more marked and clearly identifiable elbow formation at $k = 3$. This result is consistent with what was observed in the Silhouette Score table, where the maximum value is reached for three clusters, followed by a progressive decrease as k increases. This evidence suggests that, in the nonlinear spaces learned by t-SNE and UMAP, the intrinsic structure of the data is captured more effectively by a smaller number of clusters. This observation reinforces the importance of jointly evaluating multiple criteria and adapting the selection of k to the geometric context induced by the different preprocessing techniques.

Results We now turn to a detailed examination of the results summarized in Figure 30. As discussed previously, due to the different nature of the spaces in which the clustering algorithms are applied, namely the original feature space and the reduced spaces, the results obtained are not always directly comparable across configurations that rely on different dimensionality reduction techniques. However, a consistent comparison can be carried out between the results obtained in the PCA-reduced space and those obtained in the original space, since PCA corresponds to a linear transformation of the variables and preserves a global structure that remains largely comparable. In particular, by comparing the baseline results with those obtained after applying PCA, it can be observed that the PCA and k-means configuration achieves the best performance across all three internal validation indices considered, although with relatively small margins. Moreover, with the exception of PAM, the application of PCA tends to systematically improve the performance of the baseline clustering algorithms, confirming the trends observed in the experimental analyses presented in the previous chapter. Regarding the UMAP-based configurations, which were identified in the experimental settings as the most promising preprocessing strategy, we observe that the

best performance is obtained by combining UMAP with k-means and with PAM. Since the results of these two combinations are essentially equivalent in terms performances, PAM is selected as the reference algorithm, favouring a medoid-based solution that is potentially more robust to the presence of residual extreme observations. Consequently, the final analyses will focus on two main configurations: PCA followed by k-means with $k = 4$ clusters, and UMAP followed by PAM with $k = 3$ clusters.

Algorithms	Silhouette Score	Davies Bouldin	Calinski Harabasz
k-means	0,22	1,36	3116,279
PCA + k-means	0,296	1,097	4764,273
t-SNE + k-means	0,39	0,84	10175,179
UMAP + k-means	0,418	0,799	11601,247
Birch	0,195	1,495	1494,252
PCA + Birch	0,232	1,275	2170,844
t-SNE + Birch	0,329	0,986	7808,326
UMAP + Birch	0,354	0,895	8608,939
GMM	0,173	2,103	1535,094
PCA + GMM	0,211	1,462	2428,52
t-SNE + GMM	0,374	0,891	9374,913
UMAP + GMM	0,394	0,84	10368,128
PAM	0,21	1,393	3048,933
PCA + PAM	0,188	1,333	3882,545
t-SNE + PAM	0,39	0,841	10174,595
UMAP + PAM	0,418	0,798	11600,022
Spectral Clustering	0,221	1,32	2894,931
PCA + Spectral Clustering	0,242	1,227	4131,494
t-SNE + Spectral Clustering	-0,135	1,068	17,297
UMAP + Spectral Clustering	0,415	0,807	11383,167

Figure 30: Comparison of internal validation metrics for the different clustering configurations considered

6.4 In-depth Analysis

We have reached the final part of the chapter, which focuses on a detailed analysis of the composition of the clusters identified by the two selected configurations: PCA followed by k-means, and UMAP followed by PAM. For both cases, the typical consumption patterns associated with each cluster will be examined, and the analysis

will be further improved by considering additional variables available in the original dataset. The main objective of this section is to identify the dominant patterns characterizing each cluster to understand how they differ from one another and to assess whether they are informative with respect to the variables considered. Finally, the clusters will be mapped onto a geographical representation of the United States in order to identify the dominant cluster in each state.

6.4.1 PCA and k-means Configuration

We begin by analysing the mean and median values characterizing each household cluster identified by the PCA followed by k-means. Since all variables have been previously standardized, positive values involve consumption above the sample average, while negative values show lower consumption. Cluster 0 could be referred as the "Cooling-dominant energy-use profile", since is characterized by a very high value of cooling-related consumption, compared to values close to the average for all other consumption components. This profile suggests households predominantly located in warmer climatic areas, where cooling represents the main driver of residential energy demand. In contrast, Cluster 1, could be labeled as the "Heating-dominant energy-use profile", because a high consumption for domestic heating can be seen, with mean and median values well above the sample average, while the remaining components remain close to zero. This pattern is typical of colder climatic regions, where heating constitutes the main source of household energy consumption. Taken together, these first two clusters highlight how, despite the climate normalization of the variables, a strong environmental component persists. However, this subdivision is only partially informative for the purposes of behavioural energy analysis, as it primarily reflects geographic and climatic differences rather than structural patterns of household energy use. Most likely, this configuration is not able to properly handle the two peaks observed in the tails of the distributions in Figure 27, thus resulting in the formation of two additional sub-clusters. Fortunately, together they represent approximately 23% of the total observations.

Label	Size	Heating	Cooling	Water Heating	Appliances	Cooking
0	1519	0.395499	1.919594	0.067413	0.097402	-0.173165
1	1289	2.301500	0.311234	0.185385	-0.058017	0.063677
2	5225	-0.029590	-0.116931	0.384095	0.398738	0.417010
3	4195	-0.147321	-0.142652	-0.603405	-0.685828	-0.524391

Table 4: Mean values of the clusters obtained by PCA and k-means

Label	Size	Heating	Cooling	Water Heating	Appliances	Cooking
0	1519	0.347342	1.634694	0.061760	0.109736	-0.189198
1	1289	2.027652	0.213441	0.178466	-0.009816	0.073604
2	5225	-0.087883	-0.133289	0.362709	0.387416	0.400438
3	4195	-0.209140	-0.160948	-0.517110	-0.562082	-0.523841

Table 5: Median values of the clusters obtained by PCA and k-means

Of much greater interest are the remaining two clusters, which constitute the majority of the sample and are considerably more informative from a behavioural perspective. Cluster 2, could be identified as the "High overall energy-intensity profile", is characterized by mean and median values close to zero for heating and cooling consumption, while displaying pronounced positive deviations for domestic hot water, appliances, and cooking. This configuration indicates households with high domestic energy use that is largely independent of climatic conditions, maybe driven by lifestyle or usage habits. In contrast, Cluster 3, that could be labeled as the "Low non-climatic energy-use profile", presents strongly negative mean and median values for all domestic consumption components, while heating and cooling consumption remains close to the sample average. This cluster represents households characterized by consistently low levels of non-climatic energy use, suggesting more efficient or parsimonious behaviours that are independent of external climate conditions. Clusters 2 and 3 therefore represent the core of the analysis. They account for approximately 77% of the total observations and capture fundamental differences in household energy-use behaviour that go beyond climatic and geographic factors. As such, these clusters provide the most meaningful insights into structural consumption patterns and form the basis for the subsequent geographic and socioeconomic analyses.

Economic interpretation of clusters The analysis of the socioeconomic profiles reported in Table 9 allows us to improve the interpretation of energy clusters by joining it with demographic and economic information. It is worth noting, however, that for clusters 0 and 1, the main drivers of energy consumption are linked to cooling and heating, so the observed socioeconomic differences reflect the climatic and geographic context rather than specific consumption behaviours. Specifically, cluster 0 is characterized by households that are on average younger, with income levels lower than the sample average and a prevalence of full-time employment, followed by relatively high levels of education. These contexts are often associated with economically dynamic regions, characterized by high labour mobility and the presence of significant universities, such as the state of California. In these areas, it is plausible to find a higher concentration of young individuals entering or consolidating their position in the labour market. Cluster 1, on the other hand, is predominantly composed of older families, largely retired from the labour market, with income levels below the sample average. This profile appears consistent with areas characterized by colder or continental climates, often linked by lower population density and a more stable and older population. This cluster can also be assigned to inland or mountainous regions of the United States, which are generally less economically dynamic. Clusters 2 and 3 are of particular interest, as they form the predominant portion of the sample and

Label	Age	Income	Employment	Education	Race
0	-4.4	-1371.7	Full-time (56%) Retired (24%)	College (32%) Bachelor (25%)	White (90%) Black (5%)
1	3.9	-16467.4	Retired (44%) Full-time (37%)	College (30%) High school(25%)	White (78%) Black (15%)
2	5.0	1780.7	Retired (45%) Full-time (41%)	College (30%) Bachelor (27%)	White (92%) Black (4%)
3	-5.8	3338.7	Full-time (56%) Retired (23%)	Bachelor (29%) College (28%)	White (84%) Asian (7%)

Table 6: Socioeconomic profile of the clusters obtained using PCA and k-means. For categorical variables, the two most frequent modalities within each cluster are reported together with their relative frequencies; numeric variables are expressed as standardized mean deviations from the sample average.

represent the most informative groups from the perspective of analysing energy behaviour. Cluster 2 is characterized predominantly by retired households, with a high average age and slightly above-average income levels. The presence of a significant proportion of economically stable families suggests a greater level of home-based living and, consequently, a more intensive use of household appliances. It is plausible to associate this cluster with middle-class American families living in older and therefore less energy-efficient homes, for whom consumption related to water heating, the use of appliances, and meal preparation is higher. At the same time, since these households are older than those in the remaining clusters and are more frequently to be classified as having retired from the labour market, they likely spend more time at home. In contrast, Cluster 3 is primarily made up of families employed full-time, who are on average younger and have the highest income levels among the clusters considered. The higher education level observed in this group suggests a greater awareness of resource management, which is reflected in overall more efficient energy behaviours, or in living in homes characterized by a high degree of energy efficiency. In contrast to cluster 2, this cluster includes a larger share of young, predominantly employed individuals, which likely corresponds to lower energy consumption given reduced time spent at home.

Map We now represent the results of the clustering analyses on a geographical map of the United States, focusing specifically on the last two clusters related to energy consumption. As discussed in the previous sections, these clusters primarily describe the intensity of consumption related to daily home use, rather than needs that are strongly influenced by climate factors, such as heating or cooling. By limiting the analysis to these two clusters, the final map is based on a subset of 9.420 observations. As shown in Figure 31, a clear territorial division of the country emerges. Most American states are associated with a high household energy consumption profile, especially in the North and Central parts, while a smaller number of states, concentrated mainly in the Southern part of the country, have a low consumption profile.

Legend:

- High energy-use profile
- Low energy-use profile

This finding does not appear to be an isolated result or one specifically dependent on the clustering procedure. The same trend is indeed confirmed by Figure 32, which reports a map obtained independently of the clusters across the entire sample. In this second case, the geographical representation is constructed starting from a continuous index of household consumption, defined as the average of the indices relating to water heating, the use of household appliances, and consumption for cooking. Starting from this index, a threshold corresponding to the 60th percentile of the overall distribution is identified. Aggregating this information at the state level, the share of households with high household consumption is obtained for each state. The comparison between the two maps is particularly informative since both yield a very similar geographical pattern. This strengthens the robustness of the results and suggests that the territorial division reflects structural differences in household consumption behaviour across different areas of the country. In particular, the Central and Northern states appear to be characterized by a greater prevalence of households with high household consumption, as described previously in Cluster 2, while the Southern states show, on average,

a lower incidence of such behaviours. This geographic difference could be attributable to structural factors such as a greater percentage of households living at home, an average older building stock, and lifestyles more oriented toward domestic use in the Central and Northern states. Conversely, in the Southern states, energy consumption appears to be more driven by climate factors, while structural household consumption appears to be lower, even in the presence of high levels of overall energy demand.

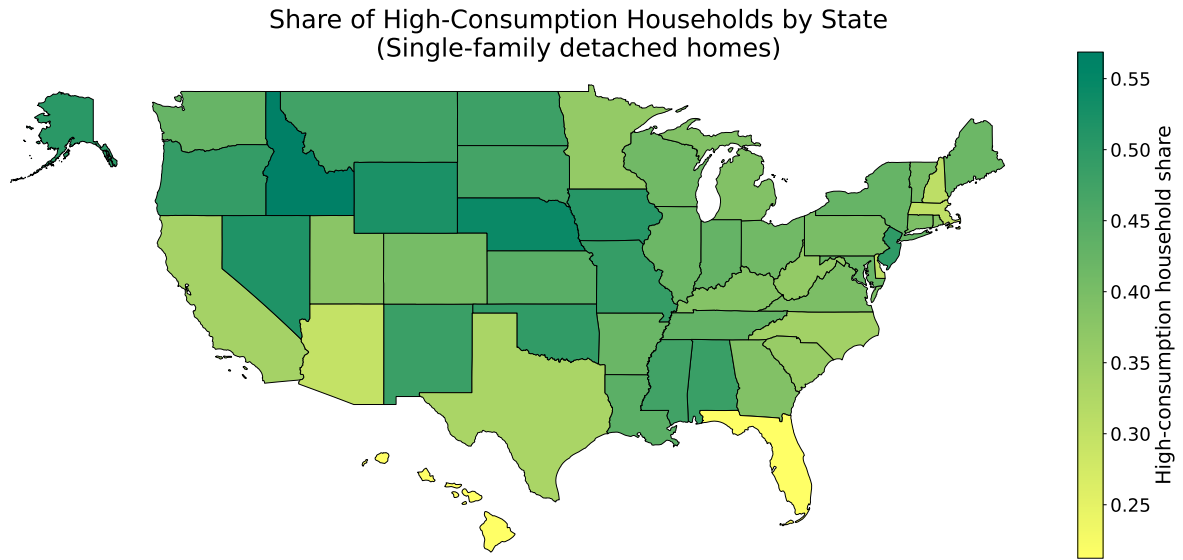


Figure 32: Share of households with high household consumption by state in detached single-family dwellings. A household is classified as high-consumption if its average household consumption index (hot water, appliances, and cooking) exceeds the 60th percentile (quantile 0.60) of the overall distribution; the map shows the corresponding percentage incidence at the state level.

6.4.2 UMAP and PAM Configuration

We now perform the same analyses using a different combination of algorithms, namely UMAP combined with PAM, which in this case implies the definition of a number of clusters equal to 3. The procedure followed is similar to the one adopted so far: we therefore start by analysing the mean and median values of each variable for each cluster, reported respectively in Table 7 and Table 8. Observing the results, we note how this configuration clearly reproduces two groups of households linked to domestic consumption, already appeared in the previous analysis. In particular, the cluster 0

presents positive mean values for the components linked to domestic consumption, such as water heating, appliances and especially cooking, suggesting a profile characterised by high consumption in daily activities. On the opposite side, cluster 1 shows strongly negative values for the same variables, highlighting lower-than-average consumption and thus representing a group composed of more saving households. A notable point is that, for both of these clusters, the values for heating and cooling are close to zero: this makes the comparison between the two typologies even more explicit as the separation between the groups is mainly driven by components related to domestic consumption. In other words, the clustering distinguishes primarily patterns attributable to consumption habits or differences in the equipment and appliances, rather than to climatic or geographical factors like the first two cluster in the previous analysis. This behaviour is also consistent when considering the median values, which confirm the same differences between the two groups.

Label	Size	Heating	Cooling	Water Heating	Appliances	Cooking
0	4428	-0.174681	-0.194792	0.397305	0.349561	0.459898
1	4237	-0.126559	-0.067914	-0.608248	-0.660801	-0.571670
2	3563	1.151977	0.914383	0.178179	0.149169	0.051597

Table 7: Mean values of the clusters obtained by UMAP and PAM

Label	Size	Heating	Cooling	Water Heating	Appliances	Cooking
0	4428	-0.194732	-0.226283	0.395962	0.351142	0.428940
1	4237	-0.201947	-0.111386	-0.505361	-0.531390	-0.559843
2	3563	0.954528	0.719121	0.160073	0.175065	0.063780

Table 8: Median values of the clusters obtained by UMAP and PAM

Finally, moving on to cluster 2, we observe that the household components are overall moderately aligned with the mean, while a simultaneous high consumption for heating and cooling clearly emerges. This pattern suggests the presence of households characterized by high energy needs, compatible with potentially less energy-efficient homes, for example with insufficient insulation or low-performance systems, so heating and cooling consumption remains high regardless of the intensity of daily household

activities. An important difference compared to the previous analysis based on PCA and k-means with 4 clusters is that, in this configuration, the so-called climate factor seems to disappear, meaning that a specific group associated with a trade-off between heating and cooling does not emerge. In other words, the clusters do not show a segmentation that can be interpreted as an effect of geographical location, but instead appear more coherently attributable to two macro-categories: on the one hand, high or low household consumption for clusters 0 and 1, and on the other, high consumption related to heating and cooling in cluster 2. This result can be interpreted as a consequence of the combination of UMAP and PAM, which tends to capture local and nonlinear structures in the data and produce more compact and more interpretable clusters than configurations based only on linear reductions such as PCA.

Economic interpretation of clusters The analysis of the socioeconomic profiles reported in Table 9 allows us to upgrade the interpretation of energy clusters, considering consumption data with demographic and economic information. Unlike the previous configuration based on PCA and k-means, in which some clusters could be primarily explained by climatic and territorial dynamics, the segmentation obtained through UMAP and PAM produces groups that are more easily interpreted from a behavioural perspective. Cluster 0 is composed of households that are on average older than the sample average, with a positive value for age and income levels slightly above average. From an employment perspective, a clear prevalence of retired individuals emerges, followed by a significant share of full-time workers. In terms of education, the cluster has a good human capital base, with a significant presence of college and bachelor’s degrees. This profile appears consistent with relatively stable families, a significant elderly population, and a moderately solid economic situation. These characteristics may favour a greater permanence in the home and a more intensive use of household services, especially those related to daily needs, thus helping to explain the high household consumption observed in the energy analysis. If we compare this with the previous PCA and k-means analyses, this cluster would correspond to cluster 2.

Label	Age	Income	Employment	Education	Race
0	5.2	1335.5	Retired (47%) Full-time (40%)	College (29%) Bachelor (28%)	White (92%) Black (3%)
1	-6.5	4080.8	Full-time (57%) Retired (22%)	Bachelor (29%) College (28%)	White (83%) Asian (7%)
2	1.2	-6512.5	Full-time (46%) Retired (36%)	College (31%) Bachelor (24%)	White (86%) Black (9%)

Table 9: Socioeconomic profile of the clusters obtained using UMAP and PAM. For categorical variables, the two most frequent modalities within each cluster are reported together with their relative frequencies; numeric variables are expressed as standardized mean deviations from the sample average.

Cluster 1, on the other hand, is characterized by households that are on average younger, as shown by the negative value for age, with above-average income levels. This group is dominated by families with full-time employment, while the retired component is much smaller than in Cluster 0. The educational profile is also high, with a significant presence of university degrees. From an ethnic perspective, greater heterogeneity is observed compared to the other groups, with a significant share of Asians. This cluster appears consistent with more dynamic social and economic contexts, in which the working-age population is predominant and has good earning capacity. The combination of relatively high income and overall lower energy consumption suggests that low consumption is not necessarily attributable to economic constraints, but may instead reflect more efficient and rational behaviour, or the presence of more modern and energy-efficient homes on average. This cluster corresponds to cluster 3 of the previous analyses. Finally, Cluster 2 presents an intermediate profile in terms of age, being slightly above average, but with an average income lower than the sample. The occupational composition highlights a strong presence of full-time workers, alongside a significant share of retired people, making a potentially more heterogeneous group in terms of family structure. Education levels are generally good and addressed primarily in college and Bachelor’s degrees, while in terms of ethnicity, the cluster shows a relatively higher share of Black people compared to the other groups. Considering that this cluster, in the energy analysis, is characterized by high consumption related

to both heating and cooling, one possible interpretation is that the dominant driver lies not so much in daily behaviour, but in structural factors related to the home. In this scenario, it is plausible that families with more limited economic resources are more frequently associated with less efficient homes, resulting in greater vulnerability to high energy consumption despite an overall level of domestic demand that is not necessarily above average. Overall, the integration of energy results and socioeconomic profiles shows how the clustering obtained through UMAP and PAM allows us to create three main household types: an older and relatively stable group, potentially associated with higher domestic consumption; a younger group, with relatively high incomes and lower consumption, likely linked to more efficient housing conditions or less time spent at home; and finally, a cluster with high heating and cooling consumptions, which appears to more closely reflect a structural or technological component of the home and could represent a priority category for energy efficiency interventions.

Map We now present the results of the clustering analysis through a map of the United States. Figure 33 shows the geographic distribution of the dominant energy profile by state, where each state is assigned to the most frequent cluster among the observed households. The map reveals marked territorial heterogeneity: a substantial share of states is associated with a high energy-use profile, primarily in the northern part of the country, whereas a smaller number of states exhibit a prevalence of low energy-use profiles, especially in the South. These results are highly consistent with those obtained in the previous PCA and k-means analysis, further confirming the territorial divide between northern and southern regions. To assess the robustness of this evidence, Figure 34 provides an alternative representation based exclusively on a binary classification between high and low energy-use profiles, excluding the cluster associated with low-efficiency houses. The comparison between the two maps is particularly informative, as it suggests that territorial differences are not solely attributable to climatic conditions, but also reflect the structural and behavioural characteristics of households.

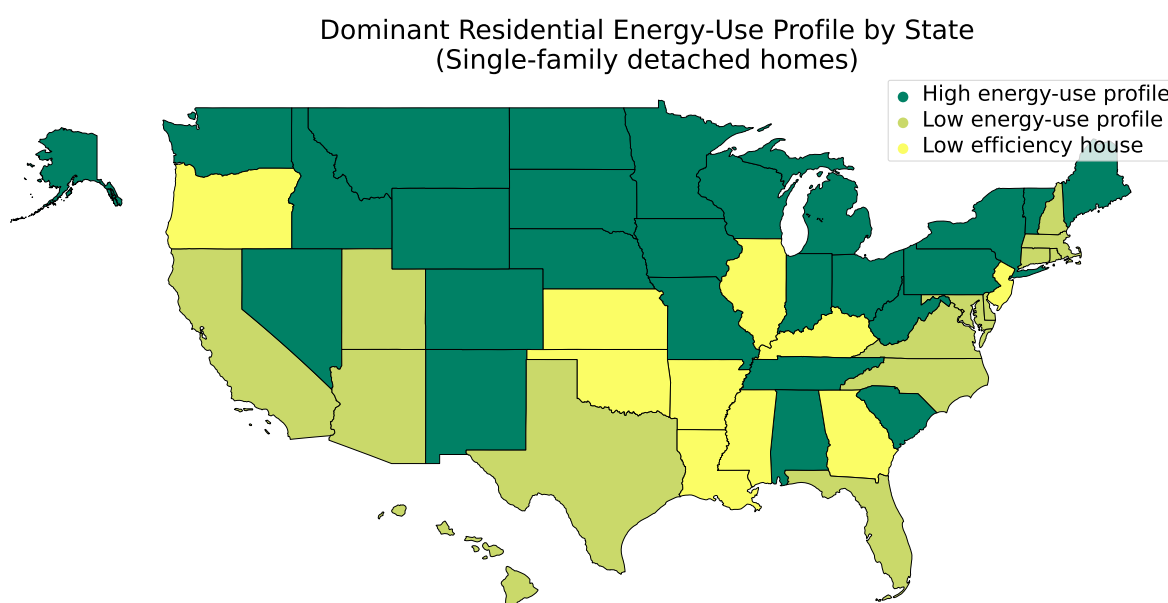


Figure 33: Geographical distribution of the dominant energy profile by state, considering all the clusters for detached single-family homes. Each state is assigned to the most frequent cluster among the observed households

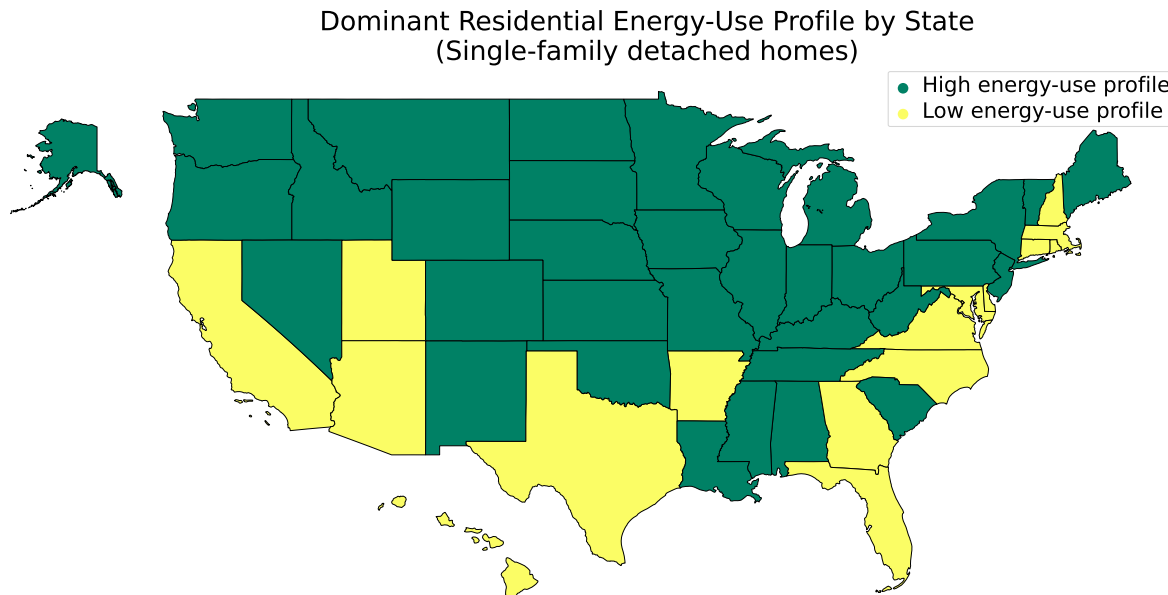


Figure 34: Geographical distribution of the dominant energy profile by state, considering only the clusters related to actual domestic consumption (hot water, appliances, and cooking) for detached single-family homes. Each state is assigned to the most frequent cluster among the observed households ($N = 8.665$).

Moreover, the third cluster, corresponding to low-efficiency housing, appears geographically condensed across central states. This pattern suggests that in some regions the structural component, namely high heating and cooling requirements, represents a predominant driver of residential energy consumption. These states may therefore be characterized by relatively low levels of energy efficiency, a result consistent with conditions that tend to be more rural and less modernized compared to larger or more economically central areas.

6.5 General Conclusions

To summarize what we have seen so far, the cluster analysis shows that the two selected configurations, one based on PCA and k-means and the other on UMAP and PAM, identify similar patterns, but with different levels of interpretability. In the PCA and k-means case, two macro-groups emerge:

- two clusters with little information, dominated by climatic and geographic factors, with heating or cooling values significantly above the average, which explain a limited portion of the sample;
- two clusters with much more significant behavioural significance, distinguishing households with high domestic consumption, i.e., hot water, appliances, and cooking, from households with low non-climatic consumption.

Considering only these last two groups, significant differences also emerge in socio-economic terms. The high-consumption cluster is mainly composed of older families, with a higher incidence of people retired from the labour market and income levels slightly above the sample average. This profile is consistent with a greater level of residence at home and a more intensive use of household services, such as hot water, appliances, and meal preparation. Conversely, the low-consumption cluster, which is non-climate-related, is characterized by families who are younger on average, more often employed full-time, and with higher-than-average incomes. Furthermore, this

group also has a higher average education level, suggesting a possible greater focus on energy efficiency or a greater likelihood of living in more modern and high-performance homes. Focusing on the last two clusters, the maps confirm a clear territorial division: the high-consumption profile is more frequent in the Northern and Central states, while the low-consumption profile is more common in the South. One possible interpretation of this finding may also be partly linked to climate and lifestyle factors: in warmer areas, it is plausible that families spend less time at home and consequently reduce the intensity of consumption related to daily household activities. With UMAP and PAM, the segmentation is cleaner and more interpretable: the climate clusters are effectively eliminated and replaced by a more informative structure, highlighting how UMAP, and consequently PAM, is better able to manage the presence of longer queues and extreme values compared to the previous configuration. The two main clusters related to domestic consumption remain, but a third additional cluster also appears, characterized by high simultaneous heating and cooling consumption, which can be interpreted as a sign of structural inefficiency in homes. This cluster appears particularly concentrated in the Central states, suggesting potential priority areas for targeted energy efficiency interventions. Regarding the first two clusters, the North-South division is again confirmed, further strengthening the robustness of the territorial pattern observed with PCA and k-means. We can therefore conclude that UMAP, even in the experimental case, proves to be a robust preprocessing algorithm, capable of reducing the influence of extreme values and generating a geometric representation of the data more suitable for segmentation by clustering algorithms.

Chapter 7

Conclusions and Future Work

This concluding chapter summarizes the main findings of the thesis and presents some possible research directions for future developments and insights.

7.1 Final Conclusions

The primary goal of this thesis was to analyse the behaviour of different clustering algorithms under multiple data settings, in order to understand which methods perform best and whether they can benefit from the application of dimensionality reduction techniques. For this reason, the work was structured into two main sections. The first experimental phase was based on synthetic datasets, generated using a function capable of reproducing different structural conditions, with a ground truth label that allowed the simulation to be repeatable and objective. For example, parameters relating to the number of observations, variables, number of clusters, intra-cluster correlation and noise were varied. In a second application and validation phase, we considered a real dataset relating to US residential energy consumption, in which the quality of the clusters was assessed using internal metrics.

Discussion of results The results highlighted some recurring trends. It emerged that, algorithm performance tends to be better for datasets with a relatively larger number of variables and fewer observations, as in the case of 2.500 observations and 15 variables, while it progressively worsens as the number of observations increases

and the available variables decrease. This behaviour suggests that the loss of discriminative information due to low informative dimensionality may have a greater impact than the benefits of a larger sample size, especially in the presence of noise and overlapping structures. Increasing the number of clusters from 3 to 5 and then to 7 almost always led to a worsening of the quality indices, namely ARI and NMI, with an increase in variability between algorithms. This result is consistent with the increase in problem complexity: more clusters imply less clear decision boundaries and a greater likelihood of misclassification, particularly when the clusters are unbalanced in size and dispersion. Increasing the correlation scaling factor generally reduced clustering quality, especially for algorithms that rely heavily on Euclidean distances, such as k-means and PAM. Correlation introduces information redundancy and generates anisotropic and oriented clusters, implicitly violating the spherical cluster assumption on which many partition-based techniques rely. In some cases, however, a perfectly monotonic relationship between correlation and performance was not observed, suggesting that the impact of collinearity interacts with the dimensionality and effective separability of the structures. Adding additive Gaussian noise resulted in a progressive deterioration in performance in most scenarios. Noise increases the internal dispersion of clusters, reduces compactness, increases overlap, and blurs the separation between groups. Among the algorithms considered, Spectral Clustering proved to be among the most robust, keeping relatively stable results even with high levels of noise, while methods based on centroids or medoid such as k-means and PAM proved to be more sensitive when not preceded by dimensionality reduction algorithms.

Role of dimensionality reduction One of the most significant findings of the work concerns the comparative behaviour of dimensionality reduction methods as a support for clustering. PCA showed moderate benefits, especially in the presence of correlation between variables, confirming its ability to reduce redundancy and improve the geometric settings of the space. However, in the experimental framework adopted, PCA is not a universally helpful solution: in several cases, it does not improve, and sometimes

worsens, already very good results obtained from clustering on the original space. t-SNE showed high variability and often unstable results when used as a preprocessing tool for clustering. This behaviour is consistent with its primary function: preserving local relationships, often at the expense of the global structure between clusters. Consequently, although it can produce visually effective representations, t-SNE is less reliable in a context where the goal is cluster stability and not just graphical exploration. Furthermore, the algorithm was found to be highly dependent on the initial parameter initialization, more precisely on the number of components into which the original space is reduced, showing much worse results in the case of 3 components instead of 2. UMAP proved to be the most robust and consistent method in almost all the configurations analysed. In particular, UMAP often improved the performance or stabilized algorithms sensitive to noise and collinearity, showing more regular behaviour than PCA and t-SNE. This result was also confirmed in the high-dimensional case, where UMAP was able to very effectively reconstruct the separability between clusters. The analysis on an extreme case of 2.500 observations, 100 variables, 7 clusters, high correlation scaling factor and high noise clearly highlighted that dimensionality reduction can become an essential step. Without preprocessing, many clustering algorithms are ineffective, while the use of UMAP and, to some extent, PCA allows for the discovery of latent structures that are difficult to separate in the original space. Furthermore, the simultaneous use of ARI, NMI and Silhouette score showed that a good alignment with the labels does not necessarily guarantee a compact geometry, emphasizing the importance of multi-metric evaluations.

Validation on a real dataset We now discuss the application on real data, which allowed us to verify whether the experimental findings remained consistent in an uncontrolled context. In the absence of ground truth labels, the evaluation was conducted using internal metrics such as Silhouette, Davies-Bouldin, and Calinski-Harabasz and through qualitative interpretation of the obtained profiles. The analysis showed that PCA-based solutions, which, we recall, are comparable to the original space, tend to

moderately improve the quality of the clustering. UMAP-based solutions often produce more interpretable clusters, more clearly separating patterns related to domestic consumption related to hot water, appliances and cooking, compared to segmentations dominated exclusively by climate factors such as heating and cooling. In fact, the best combination was UMAP and PAM, capable of generating a cluster structure that is more behaviourally informative and more consistent with the applied segmentation objectives. Furthermore, the geographical representation of the clusters highlighted a clear territorial heterogeneity consistent across different clustering strategies, suggesting that the identified profiles are not simple methodological artifacts, but reflect systematic differences in energy behaviours or in the structural conditions of homes.

7.2 Future work

The results obtained open several possible lines of development, both methodological and applicative. A first natural extension would be to enrich the synthetic generator by introducing non-Gaussian structures that better reflect real-world behaviour, such as skewed or heavy-tailed distributions as Student-t or log-normal. The geometric structure of the clusters could also be further diversified by considering multimodal distributions, i.e., subgroups within the same class, or manifold-based patterns such as spirals, concentric rings, and non-convex clusters. These extensions would allow testing the robustness of the proposed methodologies in scenarios where classical modelling assumptions are strongly violated. A second development concerns hyper-parameter management, which is particularly important for nonlinear techniques such as t-SNE and UMAP. Future work could include systematic tuning via grid search, stability-based selection criteria across multiple random seeds and dedicated sensitivity analyses on key parameters, such as perplexity for t-SNE and the number of nearest neighbors for UMAP. The hyperparameters were indeed set at the beginning of the workflow and were not tuned during model selection. They are reported in Appendix at Table 10. This would make the benchmarking framework more rigorous and help distinguish clear methodological limitations from suboptimal parameter configurations. A further

direction would be to extend the comparison to clustering methods that better handle noise and variable density, such as DBSCAN. This density-based technique would be particularly relevant in real-world settings, where cluster structures are often non-convex and characterised by non-uniform density. Another important issue concerns the scalability of the proposed pipelines and their applicability to larger datasets. In this regard, future developments could include a systematic evaluation of computational costs through detailed study of runtime and memory usage for each algorithmic combination. Finally, an important extension would be to explicitly consider mixed-type data, so datasets combining continuous numerical variables with categorical or binary features. This setting is extremely common in applied domains such as social sciences, healthcare, and energy-related surveys. In such cases, the Euclidean distance adopted in this thesis is no longer appropriate, as it cannot correctly capture similarities between heterogeneous variables. A more suitable approach would involve dissimilarity measures such as the Gower distance, which provides a unified framework for combining continuous, ordinal, and categorical information into a single pairwise metric. From a clustering perspective, this would also motivate the adoption of algorithms designed for dissimilarity matrices like hierarchical clustering or PAM directly on the Gower matrix, potentially leading to more realistic and interpretable segmentations. Incorporating mixed-type data would therefore represent a crucial step toward increasing the external validity and practical usability of the proposed clustering pipelines.

Application validation and interpretability In the real data case, a possible study involves the interpretability of the identified clusters and their validation through integration with external variables available in the dataset. In particular, it would be useful to analyse the extent to which cluster membership is associated with structural characteristics of the house, such as the quality of the building envelope, the level of insulation, and the year of construction, as well as the type of heating and cooling systems used. A further step could be to include more detailed climatic and territorial

variables to assess any regional or environmental differences that influence consumption profiles. Finally, a more comprehensive validation could include an estimate of the economic impact associated with the different identified groups, making it possible to also discuss potential implications for energy efficiency policies. Such analyses would allow clustering to be transformed from a purely descriptive exercise to an operational segmentation tool and support for policymaking.

Final considerations This thesis has shown that clustering and dimensionality reduction must be considered as interdependent components of the same analytical pipeline and that the optimal choice depends heavily on the structural properties of the data. In particular, it has been highlighted how factors such as correlation, noise, number of clusters, internal heterogeneity, and the ratio of observations to variables directly influence the effectiveness of unsupervised methodologies. The overall results confirm that dimensionality reduction is not always necessary, but can become crucial in complex contexts. In this scenario, UMAP emerges as the most robust and consistent tool, demonstrating consistent benefits both in controlled environments and in real-world applications, improving the quality and, above all, the interpretability of the partitions produced by clustering algorithms.

Appendix

Synthetic data generation function

```
1 class CorrelatedClusterGenerator:
2     """
3     Generator of synthetic datasets with:
4     - correlated features
5     - controllable variances
6     - separated cluster centers
7     - realistic covariance structures
8     """
9
10    def __init__(
11        self,
12        n_samples,
13        n_features,
14        n_clusters,
15        cluster_std=1.0,
16        correlation=0.5,
17        separation_factor=3.0,
18        random_state=None,
19        sample_distribution=None,
20    ):
21        """Initialize generator parameters."""
22        self.n_samples = n_samples
23        self.n_features = n_features
24        self.n_clusters = n_clusters
25        self.cluster_std = cluster_std
26        self.correlation = correlation
27        self.separation_factor = separation_factor
28        self.random_state = random_state
29        self.sample_distribution = sample_distribution
30
31    def generate_clusters(self):
32        """Generate dataset, labels, and cluster centers."""
33        np.random.seed(self.random_state)
34        centers = self._generate_separated_centers()
35        X, y = [], []
36        samples_per_cluster = self._distribute_samples()
37
```

```

38         for i, (center, n_samples_cluster) in enumerate(zip(centers
39             , samples_per_cluster)):
40             std = self._get_cluster_std(i)
41             cov_matrix = self._generate_covariance_matrix(std)
42             cluster_samples = np.random.multivariate_normal(
43                 mean=center,
44                 cov=cov_matrix,
45                 size=n_samples_cluster
46             )
47             X.append(cluster_samples)
48             y.append(np.full(n_samples_cluster, i))
49
50     X = np.vstack(X)
51     y = np.hstack(y)
52     shuffle_idx = np.random.permutation(len(X))
53     return X[shuffle_idx], y[shuffle_idx], centers
54
55     def _get_cluster_std(self, cluster_idx):
56         """Return standard deviation for a specific cluster."""
57         if isinstance(self.cluster_std, (list, np.ndarray)):
58             return self.cluster_std[cluster_idx]
59         return self.cluster_std
60
61     def _generate_separated_centers(self):
62         """Generate cluster centers with guaranteed minimum
63         separation."""
64         if isinstance(self.cluster_std, (list, np.ndarray)):
65             max_std = max(self.cluster_std)
66         else:
67             max_std = self.cluster_std
68
69         min_distance = self.separation_factor * max_std
70         centers = []
71
72         for _ in range(self.n_clusters):
73             attempts = 0
74             while attempts < 1000:
75                 center = np.random.uniform(-10, 10, self.n_features
76                     )
77                 if len(centers) == 0:
78                     centers.append(center)
79                     break
80                 distances = [np.linalg.norm(center - c) for c in
81                     centers]
82                 if min(distances) >= min_distance:
83                     centers.append(center)
84                     break
85                 attempts += 1
86             return np.array(centers)
87
88     def _generate_covariance_matrix(self, std):

```

```

85     """Generate PSD covariance matrix with controlled
86         correlation."""
87     try:
88         base_cov = make_spd_matrix(
89             n_dim=self.n_features,
90             random_state=self.random_state
91         )
92         diag_sqrt = np.sqrt(np.diag(base_cov))
93         corr_matrix = base_cov / np.outer(diag_sqrt, diag_sqrt)
94
95         if isinstance(self.correlation, (int, float)):
96             corr_factor = self.correlation
97         else:
98             corr_factor = np.mean(self.correlation)
99
100         scaled_corr = (corr_matrix - np.eye(self.n_features)) *
101             corr_factor + np.eye(self.n_features)
102         final_cov = scaled_corr * (std**2)
103         return final_cov
104
105     except (ImportError, ValueError, np.linalg.LinAlgError) as
106         e:
107         print(f"Warning: Using diagonal covariance matrix for
108             std={std}. Error: {e}")
109         return np.eye(self.n_features) * (std ** 2)
110
111 def _distribute_samples(self):
112     """
113     Distribute samples across clusters using:
114     - equal split (default)
115     - percentage distribution
116     - exact counts
117     """
118     if self.sample_distribution is None:
119         base = self.n_samples // self.n_clusters
120         remainder = self.n_samples % self.n_clusters
121         samples = [base] * self.n_clusters
122         for i in range(remainder):
123             samples[i] += 1
124         return samples
125
126     if all(isinstance(x, float) and 0 <= x <= 1 for x in self.
127         sample_distribution):
128         total = sum(self.sample_distribution)
129         if not np.isclose(total, 1.0, atol=1e-6):
130             raise ValueError(f"Percentages must sum to 1.0, got
131                 {total}")
132
133         samples_per_cluster = [int(self.n_samples * pct) for
134             pct in self.sample_distribution]
135         total_assigned = sum(samples_per_cluster)
136         diff = self.n_samples - total_assigned

```

```

130
131         if diff > 0:
132             remainders = [
133                 (self.n_samples * pct) - int(self.n_samples *
134                     pct)
135                 for pct in self.sample_distribution
136             ]
137             sorted_indices = np.argsort(remainders[::-1])
138             for i in range(diff):
139                 samples_per_cluster[sorted_indices[i]] += 1
140             return samples_per_cluster
141
142         if sum(self.sample_distribution) != self.n_samples:
143             raise ValueError(
144                 f"Sample counts must sum to {self.n_samples}, got {
145                     sum(self.sample_distribution)}"
146             )
147         return list(self.sample_distribution)
148
149     def create_cluster_dataset(
150         n_clusters=3,
151         n_samples=10000,
152         n_features=20,
153         cluster_std=1.0,
154         correlation=0.2,
155         separation_factor=3.0,
156         random_state=42,
157         noise_std=0.0,
158         sample_distribution=None,
159     ):
160         """Create correlated-cluster dataset and return it as DataFrame
161         ."""
162         gen = CorrelatedClusterGenerator(
163             n_samples=n_samples,
164             n_features=n_features,
165             n_clusters=n_clusters,
166             cluster_std=cluster_std,
167             correlation=correlation,
168             separation_factor=separation_factor,
169             random_state=random_state,
170             sample_distribution=sample_distribution,
171         )
172
173         X, y, _ = gen.generate_clusters()
174
175         if noise_std > 0:
176             X = X + np.random.normal(scale=noise_std, size=X.shape)
177
178         df = pd.DataFrame(X, columns=[f"var_{i+1}" for i in range(X.
179             shape[1])])
180         df["target"] = y
181         return df

```

Observations and Features:	2500 observations and 15 features								
Number of Clusters:	3 clusters			5 clusters			7 clusters		
Correlation Factor:	0,1	0,5	0,9	0,1	0,5	0,9	0,1	0,5	0,9
k-means	0,831	0,839	0,535	0,675	0,743	0,691	0,756	0,703	0,680
PCA + k-means	0,819	0,535	0,803	0,669	0,651	0,642	0,696	0,668	0,635
t-SNE + k-means	0,768	0,765	0,737	0,655	0,624	0,641	0,630	0,579	0,594
UMAP + k-means	0,835	0,827	0,850	0,659	0,762	0,672	0,706	0,669	0,684
Birch	0,728	0,763	0,752	0,629	0,562	0,620	0,642	0,594	0,588
PCA + Birch	0,793	0,811	0,825	0,740	0,615	0,628	0,669	0,552	0,550
t-SNE + Birch	0,737	0,741	0,684	0,632	0,657	0,583	0,621	0,536	0,584
UMAP + Birch	0,835	0,827	0,850	0,721	0,678	0,693	0,601	0,647	0,612
GMM	0,851	0,874	0,579	0,721	0,630	0,771	0,532	0,563	0,606
PCA + GMM	0,841	0,832	0,852	0,693	0,617	0,727	0,623	0,596	0,637
t-SNE + GMM	0,501	0,751	0,803	0,648	0,662	0,653	0,586	0,582	0,602
UMAP + GMM	0,826	0,834	0,862	0,669	0,763	0,667	0,666	0,679	0,632
PAM	0,600	0,505	0,637	0,560	0,571	0,543	0,527	0,504	0,456
PCA + PAM	0,827	0,827	0,808	0,745	0,623	0,606	0,684	0,662	0,618
t-SNE + PAM	0,770	0,770	0,748	0,636	0,619	0,643	0,635	0,586	0,633
UMAP + PAM	0,834	0,830	0,849	0,756	0,661	0,772	0,689	0,686	0,694
Spectral Clustering	0,846	0,854	0,846	0,771	0,779	0,803	0,765	0,746	0,695
PCA + Spectral Clustering	0,832	0,842	0,836	0,765	0,728	0,663	0,695	0,664	0,638
t-SNE + Spectral Clustering	0,718	0,739	0,655	0,586	0,597	0,671	0,618	0,571	0,656
UMAP + Spectral Clustering	0,821	0,833	0,852	0,759	0,762	0,786	0,743	0,697	0,683

Figure 35: Results evaluated according to NMI in the case of 2500 observations and 15 variables and keeping constant noise at 7

Observations and Features:	5000 observations and 10 features								
Number of Clusters:	3 clusters			5 clusters			7 clusters		
Correlation Factor:	0,1	0,5	0,9	0,1	0,5	0,9	0,1	0,5	0,9
k-means	0,518	0,502	0,462	0,432	0,423	0,425	0,466	0,453	0,433
PCA + k-means	0,520	0,490	0,456	0,413	0,371	0,362	0,382	0,375	0,362
t-SNE + k-means	0,376	0,466	0,397	0,402	0,399	0,383	0,384	0,375	0,386
UMAP + k-means	0,485	0,485	0,456	0,472	0,444	0,478	0,462	0,422	0,416
Birch	0,420	0,391	0,370	0,356	0,347	0,354	0,327	0,344	0,327
PCA + Birch	0,431	0,446	0,452	0,391	0,385	0,377	0,355	0,368	0,326
t-SNE + Birch	0,434	0,404	0,391	0,373	0,399	0,372	0,383	0,349	0,379
UMAP + Birch	0,485	0,482	0,458	0,447	0,412	0,404	0,424	0,430	0,376
GMM	0,476	0,432	0,484	0,401	0,428	0,419	0,392	0,415	0,378
PCA + GMM	0,523	0,439	0,441	0,379	0,375	0,346	0,357	0,378	0,343
t-SNE + GMM	0,318	0,360	0,442	0,382	0,398	0,411	0,381	0,374	0,407
UMAP + GMM	0,469	0,469	0,459	0,411	0,430	0,480	0,452	0,410	0,386
PAM	0,427	0,423	0,417	0,370	0,393	0,391	0,360	0,374	0,344
PCA + PAM	0,518	0,491	0,456	0,410	0,370	0,362	0,384	0,355	0,353
t-SNE + PAM	0,451	0,468	0,405	0,406	0,400	0,388	0,392	0,370	0,388
UMAP + PAM	0,488	0,483	0,456	0,479	0,497	0,482	0,465	0,429	0,422
Spectral Clustering	0,524	0,492	0,470	0,496	0,491	0,479	0,496	0,493	0,486
PCA + Spectral Clustering	0,507	0,492	0,455	0,398	0,358	0,337	0,372	0,344	0,335
t-SNE + Spectral Clustering	0,334	0,444	0,389	0,394	0,388	0,385	0,377	0,361	0,387
UMAP + Spectral Clustering	0,490	0,469	0,450	0,467	0,486	0,478	0,483	0,484	0,470

Figure 36: Results evaluated according to NMI in the case of 5000 observations and 10 variables and keeping constant noise at 7

Observations and Features:	10000 observations and 5 features								
Number of Clusters:	3 clusters			5 clusters			7 clusters		
Correlation Factor:	0,1	0,5	0,9	0,1	0,5	0,9	0,1	0,5	0,9
k-means	0,334	0,321	0,288	0,255	0,258	0,257	0,270	0,264	0,268
PCA + k-means	0,323	0,297	0,282	0,263	0,250	0,224	0,255	0,248	0,239
t-SNE + k-means	0,287	0,274	0,255	0,264	0,237	0,243	0,270	0,263	0,248
UMAP + k-means	0,319	0,322	0,295	0,278	0,269	0,256	0,268	0,279	0,258
Birch	0,287	0,307	0,263	0,205	0,187	0,194	0,237	0,241	0,171
PCA + Birch	0,316	0,286	0,278	0,201	0,250	0,157	0,264	0,238	0,223
t-SNE + Birch	0,235	0,106	0,291	0,213	0,230	0,199	0,248	0,263	0,234
UMAP + Birch	0,292	0,286	0,304	0,265	0,245	0,253	0,269	0,281	0,259
GMM	0,336	0,297	0,213	0,208	0,250	0,206	0,231	0,193	0,270
PCA + GMM	0,166	0,266	0,242	0,224	0,205	0,210	0,252	0,246	0,236
t-SNE + GMM	0,150	0,251	0,169	0,258	0,227	0,194	0,247	0,263	0,240
UMAP + GMM	0,326	0,326	0,366	0,273	0,244	0,217	0,250	0,260	0,245
PAM	0,335	0,340	0,272	0,267	0,238	0,261	0,268	0,265	0,253
PCA + PAM	0,324	0,346	0,299	0,263	0,247	0,227	0,259	0,251	0,246
t-SNE + PAM	0,291	0,274	0,259	0,265	0,248	0,214	0,269	0,263	0,247
UMAP + PAM	0,323	0,322	0,296	0,276	0,270	0,257	0,276	0,281	0,259
Spectral Clustering	0,343	0,333	0,291	0,249	0,246	0,253	0,263	0,269	0,269
PCA + Spectral Clustering	0,320	0,313	0,288	0,249	0,241	0,220	0,250	0,244	0,241
t-SNE + Spectral Clustering	0,159	0,067	0,139	0,242	0,200	0,228	0,256	0,251	0,242
UMAP + Spectral Clustering	0,335	0,329	0,302	0,270	0,253	0,255	0,265	0,269	0,262

Figure 37: Results evaluated according to NMI in the case of 10000 observations and 5 variables and keeping constant noise at 7

Observations and Features:	2500 observations and 15 features								
Number of Clusters:	3 clusters			5 clusters			7 clusters		
Noise:	5	7	9	5	7	9	5	7	9
k-means	0,931	0,839	0,704	0,752	0,743	0,545	0,838	0,703	0,547
PCA + k-means	0,926	0,535	0,695	0,836	0,651	0,54	0,802	0,668	0,519
t-SNE + k-means	0,918	0,765	0,612	0,768	0,624	0,454	0,831	0,579	0,401
UMAP + k-means	0,93	0,827	0,704	0,915	0,762	0,505	0,845	0,669	0,469
Birch	0,875	0,763	0,57	0,876	0,562	0,463	0,835	0,594	0,389
PCA + Birch	0,93	0,811	0,642	0,766	0,615	0,48	0,731	0,552	0,373
t-SNE + Birch	0,902	0,741	0,587	0,761	0,657	0,452	0,821	0,536	0,392
UMAP + Birch	0,93	0,827	0,703	0,87	0,678	0,543	0,868	0,647	0,483
GMM	0,953	0,874	0,457	0,773	0,63	0,485	0,724	0,563	0,382
PCA + GMM	0,941	0,832	0,599	0,88	0,617	0,519	0,759	0,596	0,492
t-SNE + GMM	0,924	0,751	0,518	0,801	0,662	0,474	0,869	0,582	0,407
UMAP + GMM	0,93	0,834	0,7	0,919	0,763	0,506	0,846	0,679	0,45
PAM	0,841	0,505	0,452	0,848	0,571	0,331	0,729	0,504	0,311
PCA + PAM	0,928	0,827	0,722	0,738	0,623	0,52	0,79	0,662	0,508
t-SNE + PAM	0,918	0,77	0,626	0,771	0,619	0,466	0,832	0,586	0,401
UMAP + PAM	0,93	0,83	0,704	0,915	0,661	0,534	0,844	0,686	0,516
Spectral Clustering	0,943	0,854	0,7	0,926	0,779	0,6	0,917	0,746	0,572
PCA + Spectral Clustering	0,943	0,842	0,696	0,874	0,728	0,554	0,806	0,664	0,508
t-SNE + Spectral Clustering	0,928	0,739	0,585	0,905	0,597	0,402	0,915	0,571	0,402
UMAP + Spectral Clustering	0,618	0,833	0,706	0,922	0,762	0,521	0,828	0,697	0,506

Figure 38: Results evaluated according to NMI in the case of 2500 observations and 15 variables and keeping correlation scaling factor constant at 0.5

Observations and Features:	5000 observations and 10 features								
Number of Clusters:	3 clusters			5 clusters			7 clusters		
Noise:	5	7	9	5	7	9	5	7	9
k-means	0,598	0,502	0,388	0,654	0,423	0,335	0,656	0,453	0,32
PCA + k-means	0,591	0,49	0,379	0,462	0,371	0,33	0,492	0,375	0,268
t-SNE + k-means	0,569	0,466	0,278	0,593	0,399	0,303	0,592	0,375	0,264
UMAP + k-means	0,609	0,485	0,365	0,67	0,444	0,359	0,614	0,422	0,292
Birch	0,59	0,391	0,266	0,513	0,347	0,259	0,546	0,344	0,228
PCA + Birch	0,564	0,446	0,354	0,47	0,385	0,31	0,5	0,368	0,271
t-SNE + Birch	0,546	0,404	0,257	0,534	0,399	0,262	0,523	0,349	0,256
UMAP + Birch	0,61	0,482	0,361	0,588	0,412	0,338	0,635	0,43	0,281
GMM	0,675	0,432	0,281	0,634	0,428	0,288	0,601	0,415	0,262
PCA + GMM	0,583	0,439	0,23	0,488	0,375	0,287	0,489	0,378	0,277
t-SNE + GMM	0,576	0,36	0,288	0,62	0,398	0,294	0,551	0,374	0,254
UMAP + GMM	0,597	0,469	0,307	0,647	0,43	0,331	0,621	0,41	0,297
PAM	0,494	0,423	0,306	0,534	0,393	0,287	0,578	0,374	0,282
PCA + PAM	0,596	0,491	0,379	0,454	0,37	0,329	0,509	0,355	0,278
t-SNE + PAM	0,569	0,468	0,286	0,526	0,4	0,302	0,586	0,37	0,266
UMAP + PAM	0,615	0,483	0,358	0,668	0,497	0,327	0,669	0,429	0,297
Spectral Clustering	0,611	0,492	0,382	0,663	0,491	0,357	0,694	0,493	0,299
PCA + Spectral Clustering	0,594	0,492	0,376	0,46	0,358	0,311	0,47	0,344	0,266
t-SNE + Spectral Clustering	0,582	0,444	0,297	0,499	0,388	0,299	0,598	0,361	0,251
UMAP + Spectral Clustering	0,611	0,469	0,358	0,67	0,486	0,336	0,621	0,484	0,313

Figure 39: Results evaluated according to NMI in the case of 5000 observations and 10 variables and keeping correlation scaling factor constant at 0.5

Observations and Features:	10000 observations and 5 features								
Number of Clusters:	3 clusters			5 clusters			7 clusters		
Noise:	5	7	9	5	7	9	5	7	9
k-means	0,441	0,321	0,226	0,375	0,258	0,175	0,383	0,264	0,182
PCA + k-means	0,451	0,297	0,227	0,365	0,25	0,174	0,333	0,248	0,187
t-SNE + k-means	0,436	0,274	0,199	0,357	0,237	0,161	0,367	0,263	0,19
UMAP + k-means	0,5	0,322	0,198	0,397	0,269	0,189	0,403	0,279	0,189
Birch	0,396	0,307	0,175	0,298	0,187	0,131	0,373	0,241	0,153
PCA + Birch	0,351	0,286	0,206	0,357	0,25	0,143	0,333	0,238	0,162
t-SNE + Birch	0,417	0,106	0,155	0,329	0,23	0,15	0,337	0,263	0,187
UMAP + Birch	0,437	0,286	0,224	0,382	0,245	0,171	0,402	0,281	0,191
GMM	0,469	0,297	0,217	0,33	0,25	0,131	0,355	0,193	0,156
PCA + GMM	0,448	0,266	0,177	0,241	0,205	0,112	0,344	0,246	0,18
t-SNE + GMM	0,33	0,251	0,117	0,35	0,227	0,166	0,367	0,263	0,184
UMAP + GMM	0,494	0,326	0,122	0,362	0,244	0,172	0,391	0,26	0,176
PAM	0,445	0,34	0,232	0,366	0,238	0,183	0,39	0,265	0,18
PCA + PAM	0,473	0,346	0,225	0,368	0,247	0,178	0,342	0,251	0,186
t-SNE + PAM	0,437	0,274	0,198	0,358	0,248	0,161	0,367	0,263	0,179
UMAP + PAM	0,5	0,322	0,221	0,397	0,27	0,188	0,402	0,281	0,186
Spectral Clustering	0,487	0,333	0,23	0,377	0,246	0,165	0,395	0,269	0,188
PCA + Spectral Clustering	0,454	0,313	0,221	0,358	0,241	0,179	0,321	0,244	0,18
t-SNE + Spectral Clustering	0,309	0,067	0,002	0,384	0,2	0,162	0,364	0,251	0,183
UMAP + Spectral Clustering	0,518	0,329	0,217	0,391	0,253	0,176	0,391	0,269	0,183

Figure 40: Results evaluated according to NMI in the case of 10000 observations and 5 variables, keeping correlation scaling factor constant at 0.5

Algorithm	Main hyperparameters
PCA	n_components: Kaiser Criterion
t-SNE	n_components=2; perplexity=30; learning_rate="auto"; random_state=42
UMAP	n_components=3; n_neighbors=15; min_dist=0.1; metric="euclidean"; random_state=42
k-means	n_clusters=N_clusters; init="k-means++"; max_iter=300; tol=1e-4; random_state=42
Birch	n_clusters=N_clusters; threshold=1.5; branching_factor=50
GMM	n_components=N_clusters; covariance_type="full"; max_iter=100; tol=1e-3; random_state=42
PAM	n_clusters=N_clusters; metric="euclidean"; tolerance=1e-4
Spectral Clustering	n_clusters=N_clusters; affinity="nearest_neighbors"; n_neighbors=10; assign_labels="kmeans"; random_state=42

Table 10: Main hyperparameters used for dimensionality reduction and clustering algorithms.

Bibliography

- [1] Grané Chavez Aurea, Riani Marco, and Salini Silvia. “A compared protocol to improve clustering procedures”. In: *Book of Short Papers: 11th International Conference IES 2023: Statistical Methods for Evaluation and Quality: Techniques, Technologies and Trends*. Il Viandante, 2023, pp. 298–303. ISBN: 979-12-803-3369-8.
- [2] David L. Banks and Stephen E. Fienberg. “Data Mining, Statistics”. In: *Encyclopedia of Physical Science and Technology (Third Edition)*. Ed. by Robert A. Meyers. Third Edition. New York: Academic Press, 2003, pp. 247–261. ISBN: 978-0-12-227410-7. DOI: <https://doi.org/10.1016/B0-12-227410-5/00164-2>. URL: <https://www.sciencedirect.com/science/article/pii/B0122274105001642>.
- [3] Leland McInnes, John Healy, and James Melville. “Umap: Uniform manifold approximation and projection for dimension reduction”. In: *arXiv preprint arXiv:1802.03426* (2018).
- [4] Ali Peshawa Jammal Muhammad and Faraj Rezhna Hassan. *Data Normalization and Standardization: A Technical Report*. Tech. rep. Technical Report. Koya University, 2014.
- [5] Gareth James et al. *An Introduction to Statistical Learning*. Springer, 2023.
- [6] Sergio Zani and Andrea Cerioli. *Analisi dei dati e data mining per le decisioni aziendali*. Giuffrè Editore Milano, 2007.
- [7] Martin Wattenberg, Fernanda Viégas, and Ian Johnson. “How to Use t-SNE Effectively”. In: *Distill* (2016). DOI: 10.23915/distill.00002. URL: <http://distill.pub/2016/misread-tsne>.
- [8] Sebastian Ruder. *An overview of gradient descent optimization algorithms*. 2017. arXiv: 1609.04747 [cs.LG]. URL: <https://arxiv.org/abs/1609.04747>.
- [9] Yeliz Karaca and Majaz Moonis. “Chapter 14 - Shannon entropy-based complexity quantification of nonlinear stochastic process: diagnostic and predictive spatiotemporal uncertainty of multiple sclerosis subgroups”. In: *Multi-Chaos, Fractal and Multi-Fractional Artificial Intelligence of Different Complex Systems*. Ed. by Yeliz Karaca et al. Academic Press, 2022, pp. 231–245. ISBN: 978-0-323-90032-4. DOI: <https://doi.org/10.1016/B978-0-323-90032-4>.

4.00018-3. URL: <https://www.sciencedirect.com/science/article/pii/B9780323900324000183>.

- [10] Yifei Zhang. “A better autoencoder for image: Convolutional autoencoder”. In: *ICONIP17-DCEC*. Available online: http://users.cecs.anu.edu.au/Tom.Gedeon/conf/ABCs2018/paper/ABCs2018-paper_58.pdf (accessed on 23 March 2017). 2018, p. 34.
- [11] Laurens van der Maaten and Geoffrey Hinton. “Visualizing data using t-SNE”. In: *Journal of machine learning research* 9.Nov (2008), pp. 2579–2605.
- [12] Julius Zilinskas. *Slides from the Multidimensional Data Visualization course*. Teaching materials, Vilnius University. 2025.
- [13] Andy Coenen and Adam Pearce. *Understanding UMAP*. 2019. URL: <https://pair-code.github.io/understanding-umap/>.
- [14] Dong et al. “Efficient k-Nearest Neighbor Graph Construction for Generic Similarity Measures.” In: *Proceedings of the 20th International Conference on World Wide Web* (2011).
- [15] Manfredo P. Do Carmo. *Differential Geometry of Curves and Surfaces*. Prentice-Hall, 1992.
- [16] Hui Yin et al. *A Rapid Review of Clustering Algorithms*. 2024. arXiv: 2401.07389 [cs.LG]. URL: <https://arxiv.org/abs/2401.07389>.
- [17] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning - Data Mining Inference and Prediction*. Springer, 2017.
- [18] Anil K Jain, M Narasimha Murty, and Patrick J Flynn. “Data clustering: a review”. In: *ACM computing surveys (CSUR)* 31.3 (1999), pp. 264–323.
- [19] David Arthur and Sergei Vassilvitskii. “K-Means++: The Advantages of Careful Seeding”. In: vol. 8. Jan. 2007, pp. 1027–1035. DOI: 10.1145/1283383.1283494.
- [20] Abiodun M. Ikotun et al. “K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data”. In: *Information Sciences* 622 (2023), pp. 178–210. ISSN: 0020-0255. DOI: <https://doi.org/10.1016/j.ins.2022.11.139>. URL: <https://www.sciencedirect.com/science/article/pii/S0020025522014633>.
- [21] Dan A Simovici and Chabane Djeraba. “Clustering”. In: *Mathematical Tools for Data Mining: Set Theory, Partial Orders, Combinatorics*. Springer, 2014, pp. 767–817.
- [22] Larry Wasserman. *All of statistics: a concise course in statistical inference*. Springer Science & Business Media, 2013.
- [23] CG Gomes, Jeff Boisvert, and Clayton V Deutsch. “Gaussian mixture models”. In: *Geostatistics Lessons* (2022).

- [24] Jiawei Han, Micheline Kamber, and Jian Pei. “10 - Cluster Analysis: Basic Concepts and Methods”. In: *Data Mining: Concepts and Techniques (Third Edition)*. Ed. by Jiawei Han, Micheline Kamber, and Jian Pei. Third Edition. The Morgan Kaufmann Series in Data Management Systems. Boston: Morgan Kaufmann, 2012, pp. 443–495. ISBN: 978-0-12-381479-1. DOI: <https://doi.org/10.1016/B978-0-12-381479-1.00010-1>. URL: <https://www.sciencedirect.com/science/article/pii/B9780123814791000101>.
- [25] Tian Zhang, Raghu Ramakrishnan, and Miron Livny. “BIRCH: an efficient data clustering method for very large databases”. In: *ACM sigmod record* 25.2 (1996), pp. 103–114.
- [26] Kamal Berahmand et al. “A comprehensive survey on spectral clustering with graph structure learning”. In: *arXiv preprint arXiv:2501.13597* (2025).
- [27] Xu Wang and Yusheng Xu. “An improved index for clustering validation based on Silhouette index and Calinski-Harabasz index”. In: *IOP Conference Series: Materials Science and Engineering*. Vol. 569. 5. IOP Publishing, 2019, p. 052024.
- [28] Caglar Cengizler and M Kerem-Un. “Evaluation of Calinski-Harabasz criterion as fitness measure for genetic algorithm based segmentation of cervical cell nuclei”. In: *Br. J. Math. Comput. Sci* 22.6 (2017), pp. 1–13.
- [29] Claire Simpson et al. “Lifting the curse from high-dimensional data: automated projection pursuit clustering for a variety of biological data modalities”. In: *Giga-Science* 14 (May 2025), g1af052. ISSN: 2047-217X. DOI: 10.1093/gigascience/g1af052. eprint: <https://academic.oup.com/gigascience/article-pdf/doi/10.1093/gigascience/g1af052/63398516/g1af052.pdf>. URL: <https://doi.org/10.1093/gigascience/g1af052>.
- [30] Shaeela Ayesha, Muhammad Kashif Hanif, and Ramzan Talib. “Overview and comparative study of dimensionality reduction techniques for high dimensional data”. In: *Information Fusion* 59 (2020), pp. 44–58. ISSN: 1566-2535. DOI: <https://doi.org/10.1016/j.inffus.2020.01.005>. URL: <https://www.sciencedirect.com/science/article/pii/S156625351930377X>.
- [31] Dehua Peng, Zhipeng Gui, and Huayi Wu. *Interpreting the Curse of Dimensionality from Distance Concentration and Manifold Effect*. Jan. 2024. DOI: 10.48550/arXiv.2401.00422.
- [32] Michael Steinbach, Levent Ertöz, and Vipin Kumar. “The Challenges of Clustering High Dimensional Data”. In: *Univ. Minnesota Supercomp. Inst. Res. Rep.* 213 (Jan. 2003). DOI: 10.1007/978-3-662-08968-2_16.
- [33] William Lippitt et al. “Limitations of clustering with PCA and correlated noise”. In: *Journal of statistical computation and simulation* 94.10 (2024), pp. 2291–2319.
- [34] Leonard Kaufman and Peter J Rousseeuw. *Finding groups in data: an introduction to cluster analysis*. John Wiley & Sons, 2009.